



NOVUS

Smart Editorial & Publishing Platform

AY 2026-2027

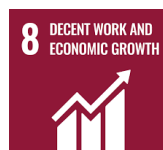
AUTHORS:

*Rei Khalil S. Galido
Roman Rico B. Albania
Joshmar L. Clavero
Joshua Concepcion*

CO-AUTHOR:

*Felino Caldero III
Carl Dominique Bueno*

CONTRIBUTES TO:



INDIRECTLY AIDS:

Table of Contents

Executive Summary	i
List of Figures	ii
List of Tables	iv
Introduction.....	1
Project Context.....	1
Statement of the Problem.....	1
Objectives	2
Significance of the Project	3
Scope and Limitations.....	3
Review of Related Literature / Systems.....	5
Technical Background	10
Current System.....	10
Proposed System.....	12
Project Feasibility	14
Requirements Analysis	15
Product Vision	15
Product Backlog / User Stories	16
Use Case Diagram.....	29
Use Case Full Description/Fully Dressed Use Case.....	29
Test Cases based on Fully Dressed Use Case	125
System Analysis and Design.....	125
System Context Diagram	125
Container Diagram.....	126
Component Diagram.....	127
Entity Relationship Diagram.....	128
Results and Discussion	130
Design Thinking Stages	130
Stage 1: Empathize	130
Stage 2: Define.....	141
Release Plan	142
Prototype	142
Use Classes and Characteristics	155
Conclusion	157

References.....	159
Appendices.....	162
Appendix A: Project Vision.....	162
Appendix B: Schedule	163
Appendix C: Product Roadmap	164
Appendix D: Team Meetings.....	165
Appendix E: Source Code.....	167

Executive Summary

This project presents NOVUS, a cloud-native, AI-assisted editorial and publishing platform developed to resolve critical operational bottlenecks and unlock digital revenue for independent media publications, specifically client partner BOSS Media Philippines Inc. (publishers of *BOSS Magazine PH*). Currently operating without dedicated digital infrastructure, the publication relies on fragmented consumer messaging applications (Facebook Messenger, Viber, Gmail) and manual workflows. This results in excessive editorial fatigue with the sole editor spending 31 to 40 hours per week handling 21 to 49 daily messages cascading publishing delays exceeding two weeks, zero organic search discoverability, and an inability to generate direct revenue from published content.

NOVUS addresses these challenges by making a publishing lifecycle into a unified, role-based platform. Built on a modular architecture utilizing Vue.js for the reactive web frontend and Django REST Framework for the backend API, the platform introduces four core capability pillars:

1. **Visual Pipeline Tracking:** A centralized workspace that replaces informal chat handoffs with real-time status tracking across all submission stages.
2. **AI-Assisted Editorial Assistant:** Claude AI API that evaluates submitted drafts and assigns an overall score from 0 to 100. Drafts must reach the passing mark of 70 to advance to the editor, aiming to cut the editor's active review time per article by at least half against a baseline measured in the pilot phase, while retaining a Human-in-the-Loop (HITL) override for editorial agency.
3. **Owned Reader Portal & Paywall:** A standalone public web application that frees the publication from total dependence on social media algorithms.
4. **Monetization Architecture:** Native integration with the PayMongo API to process localized e-wallet (GCash, Maya) and card payments for recurring reader subscriptions that unlock premium articles and digital issues.

With this NOVUS establishes an equitable working environment for newsroom staff while delivering the scalable infrastructure required for independent media survival. Developed across the Project-Based Learning (PBL) curriculum, the system's design, requirement specifications, UML/C4 modeling, database schemas, and high-fidelity wireframe prototypes are complete. Full software coding, API integration, testing, and production deployment will be executed in the final implementation phase (**MCSPROJ**).

List of Figures

Figure 1 Novus Project Lean Canvas	16
Figure 2 Use case diagram	29
Figure 3 Editorial Production	30
Figure 4 Issue Production and Publication	52
Figure 5 Editorial & Issue Oversight	70
Figure 6 Reader Engagement	77
Figure 7 Accounts Administration	88
Figure 8 UC-1.1: Commission an article	100
Figure 9 UC-1.2: Produce an Article	101
Figure 10 UC-1.2.1: Evaluate a Submitted Article	102
Figure 11 UC-1.3: Communicate About an Article	103
Figure 12 UC-1.3.1: Decide on a Submitted Article	104
Figure 13 UC-1.4: Correct an Editorial Decision	105
Figure 14 UC-1.5: Sign Off Editor-Authored Copy	106
Figure 15 UC-2.1: Schedule Articles into an Issue	107
Figure 16 UC-2.2: Produce a Magazine Layout	108
Figure 17 UC-2.3: Approve a Magazine Layout	109
Figure 18 UC-2.4: Close an Issue	110
Figure 19 UC-2.5: Publish an Issue	111
Figure 20 UC-2.6: Retire an Issue	112
Figure 21 UC-3.1: Review Publication Performance	113
Figure 22 UC-3.2: Monitor Editorial Progress	114
Figure 23 UC-3.3: Consult the Published Archive	115
Figure 24 UC-4.1: Read Published Content	116
Figure 25 UC-4.2: Save Articles for Later	117
Figure 26 UC-4.3: Subscribe to BOSS Magazine	118
Figure 27 UC-4.3.1: Read Subscriber-Only Content	119
Figure 28 UC-4.4: Manage a Subscription	120
Figure 29 UC-5.1: Register and Authenticate	121
Figure 30 UC-5.2: Manage Own Account	122
Figure 31 UC-5.3: Administer User Accounts	123
Figure 32 UC-5.4: Configure the Platform	124
Figure 33 System Context Diagram	125
Figure 34 Container Diagram	126
Figure 35 Component Diagram Vue.js Web Application	127
Figure 36 Component Diagram Django API Application	128
Figure 37 Entity Relationship Diagram	129
Figure 38 Design Thinking Stage 1 Meeting with our Client	130
Figure 39 Empathy Map Editor	134
Figure 40 Empathy Map Writers	135
Figure 41 Empathy map Publisher	137
Figure 42 Empathy Map Graphic Designer	139
Figure 43 Log in Page	143
Figure 44 Writer Dashboard	143
Figure 45 Article Submission Form	144
Figure 46 Article Editor	144
Figure 47 Submission status view	145
Figure 48 Revisions Inbox	145
Figure 49 Editor Dashboard	146

Figure 50 Article Review Screen	146
Figure 51 Revision notes composer	147
Figure 52 Approval Screen	147
Figure 53 Deadline management panel	148
Figure 54 Pipeline Overview	148
Figure 55 Layout handoff screen	149
Figure 56 Layout submission	149
Figure 57 Publisher pipeline view	150
Figure 58 Publishing confirmation	150
Figure 59 User homepage	151
Figure 60 Article listing page	151
Figure 61 Article read review	152
Figure 62 Subscription page	152
Figure 63 Payment checkout	153
Figure 64 Storyboard AI pre-screening & Editors sprint board	154
Figure 65 Storyboard Graphic designer Submission & Writer Centralized drafting workspace	154
Figure 66 Storyboard Writer deadline tracking & Writer Revision notes	155
Figure 67 Project schedule	163
Figure 68 Project schedule II	163
Figure 69 Project Roa	164
Figure 70 Meeting with team adviser	165
Figure 71 Consultation of Deliverables with team adviser	165
Figure 72 Sprint review with team adviser	166
Figure 73 GitHub repository	167

List of Tables

Table I Comparison of Publishing Systems	7
Table II List of Processes in the Current System of BOSS Media Philippines Inc.	10
Table III SWOT Analysis of BOSS Media Philippines Inc.	11
Table IV Product Backlog	16
Table V Client Interviews	131
Table VI User classes and characteristics	155

Introduction

The media industry in the Philippines is undergoing a critical shift. As audiences move away from traditional media, independent publications face mounting pressure to modernize their operations or risk being left behind: between 2020 and 2026, weekly television news use fell from 66 percent to 42 percent, and newspaper readership declined from 22 percent to 10 percent [1].

Project Context

BOSS Media Philippines Inc., publisher of BOSS Magazine, aims to scale into a globally recognized Filipino media brand with a sustainable digital revenue model. To achieve this, the organization is developing Novus, a dual-purpose platform designed to solve current operational bottlenecks and drive future growth.

Currently, the core production workflow between the Publisher, Editor, Writers, and Graphic Designer is highly fragmented. The team faces several critical challenges:

- **Informal Channels:** Assignments, submissions, and design files are routed through disjointed platforms like Messenger, Viber, and Gmail.
- **Lack of Standardization:** There are no formal design or editorial briefs, deadline tracking systems, or centralized review hubs.
- **Zero Pipeline Visibility:** The Publisher relies entirely on manual updates from the Editor and lacks real-time oversight of the production process.

Novus will directly address these issues by providing a centralized internal system to standardize the editorial pipeline, alongside a public-facing platform to expand audience reach and monetize their digital content.

Statement of the Problem

BOSS Magazine PH is a comprehensive Filipino lifestyle publication — covering a broad spectrum of topics including fashion, culture, modern trends, business, and society — that currently operates without any dedicated digital infrastructure to support its editorial, publishing, or revenue functions.

I. Fragmented Editorial Workflow and Inability to Scale

The first and most critical problem is the absence of a centralized editorial workflow system, which directly limits the magazine's ability to scale. The sole editor of BOSS Magazine PH manually manages 10 or more article submissions per publishing cycle, receiving drafts through 4 or more different communication platforms including Messenger, Viber, and email simultaneously. This results in 21 to 49 work-related messages per day, forces the editor to spend 31 to 40 hours per week on editorial tasks, and leaves no traceable record of submission history, revision rounds, or approval decisions. Furthermore, as the writer roster grows, this editorial workload grows proportionally with no additional support — resulting in cascade publishing delays where one late article pushes back the entire issue, and a publishing cycle that regularly exceeds 2 weeks to complete.

II. Time-Consuming Manual Content Reviews

The second problem is the lack of an automated content review mechanism. Every article submitted by a writer goes directly to the editor for a full manual review — covering grammar, spelling, tone, brand voice, and structure — before any feedback is given. Because there is no first-pass automated review tool, the editor spends more than 2 hours per day writing and explaining revision notes, and an additional 1 to 2 hours chasing writers for follow-ups and resubmissions. Only 3 to 4 out of every 10 submitted articles require no revision, meaning the majority of submissions add significant manual labor to an already overloaded workflow.

III. No Dedicated Digital Platform or Organic Reach

The third problem is the lack of a functional and reliable public digital presence. While BOSS Magazine PH previously attempted to establish a website using a third-party builder (Webtulz), the platform proved unviable, difficult to maintain, and is currently non-functional. As a result, the magazine has been forced to revert to publishing exclusively through Facebook. This limits the magazine's reach to existing social media algorithms, provides no mechanism to attract new readership organically through search engines, and prevents the publication from being taken seriously as a professional media entity by brand partners or international audiences.

IV. Zero Direct Revenue Generation

The fourth problem is the complete absence of a revenue model. Despite producing consistent editorial content, BOSS Magazine PH generates zero direct revenue from its published output. There is no subscription system, no digital issue sales mechanism, and no paywall — meaning the magazine bears the full operational cost of production with no financial return from its readership.

Objectives

Main Objective

To develop NOVUS, a centralized, AI-assisted digital publishing platform that streamlines editorial operations and introduces a subscription model, enabling BOSS Magazine PH to become a self-sustaining media enterprise.

Specific Objectives

To ensure the system's operational success and feasibility, the development of Novus is guided by the following measurable targets:

1. **Workflow & Capacity:** To streamline the editorial process of BOSS Magazine PH by implementing a centralized pipeline system that can handle a minimum of 50 article submissions per month, ensuring all updates and progress are tracked and visible to the team in one place.
2. **Workload Reduction:** To reduce the editor's active review time per article by at least 50% through the use of an AI-assisted pre-screening feature, measured against a baseline recorded 45 minutes average.
3. **Portal Performance:** To develop a reader-facing website for BOSS Magazine PH, meeting standard web performance benchmarks upon launch achieving a good rating having an average of 85 across its core evaluation metrics
4. **Monetization Readiness:** Establish the technical foundation for future digital sales by integrating a payment gateway and successfully validating it through simulated test transactions in a sandbox environment.

Significance of the Project

Primary Beneficiaries

The implementation of Novus directly benefits the core production team and its audience. For the editor, AI-assisted reviews and a centralized dashboard eliminate hours of manual administration, allowing a shift toward high-value strategic editing. The publisher gains real-time pipeline visibility and enforces deadlines while unlocking a critical digital revenue model through subscriptions and sales. Writers transition from fragmented chat handoffs to a dedicated portal, where instant AI feedback on grammar and tone significantly accelerates the revision cycle. Finally, readers benefit from a professional standalone website that offers premium subscriptions, digital editions, and category browsing and search for content discovery.

Improvement to the Current Situation

Novus drastically improves the magazine's current operational, editorial, commercial, and reputational standing. Operationally, it replaces a delayed, manual publishing cycle with a structured, trackable workflow. Editorially, automated AI pre-checks raise the baseline quality of submissions, drastically reducing manual revision rounds. Commercially, the platform transforms a zero-revenue operation into a monetized business with active subscriptions and digital storefronts. Ultimately, Novus elevates BOSS Magazine PH from a social media presence to an authoritative, professional digital publication.

Sustainable Development Goal (SDG) Alignment

SDG 8 — Decent Work and Economic Growth. Novus directly contributes to SDG 8 by enabling BOSS Magazine PH to build a sustainable digital business model that supports the livelihoods of its editorial team, writers, and publisher. By introducing a subscription-based revenue stream, the platform creates a foundation for the magazine to compensate its contributors fairly and grow as a viable media enterprise. Additionally, by reducing the editor's unsustainable 31–40-hour editorial workload through automation, Novus promotes healthier and more equitable working conditions within the organization.

SDG 9 — Industry, Innovation, and Infrastructure. Novus also aligns with SDG 9 by building the digital infrastructure that BOSS Magazine PH currently lacks — transforming an informal, chat-based operation into a technology-enabled publishing system. The integration of AI-assisted editorial review and oversight by human editors represents a direct application of innovation to a real industry challenge, demonstrating how small independent publishers in the Philippines can leverage modern technology to operate at a level previously only accessible to large media organizations.

Scope and Limitations

Scope of Novus

The scope of Novus encompasses a comprehensive, role-based platform designed to manage the entire editorial and publication lifecycle. It features a dedicated writer portal with an integrated AI-assisted review system that automatically evaluates submissions for readability and grammar. This feeds into a centralized editorial dashboard equipped with structured revision management and approval workflows, effectively eliminating the need for external chat applications. For public engagement and monetization, the platform includes a streamlined web interface alongside a localized payment gateway to handle subscriptions to premium articles and digital issues. Additionally, an admin dashboard provides high-level overview on production workflows and revenue metrics.

Limitations of the Project

The current iteration of Novus delivers a focused, fully web-based editorial and publishing platform for a single publication, BOSS Magazine PH.

To keep delivery within the team's academic schedule and budget, the following boundaries apply to this release:

- **Payments** - Payment processing is validated through simulated transactions in the payment gateway's sandbox environment; live transactions depend on the client's merchant account approval. Digital issues are accessed through an active subscription, and one-time purchases of individual issues are deferred to a future release.
- **AI evaluation** - The AI pre-screening assesses grammar, readability, and structure against the passing mark of 70 for English-language copy. It does not detect AI-generated or plagiarized text. Automated authorship detectors are not yet reliable enough to justify penalizing a writer on their verdict alone, so authorship judgment remains with the Editor, who can consult each article's recorded history of submissions and revisions and exercise the Human-in-the-Loop override.
- **AI provider** - Evaluation uses a single AI provider reached through one internal service interface; usage monitoring covers that provider only.
- **Platform** - Novus is a responsive web application; no native mobile application is included. It serves one publication and is not designed for multiple tenants.
- **Production** - Physical printing, print distribution, and the design work itself remain outside the system; Novus manages only the submission, versioning, and approval of design files.

Review of Related Literature / Systems

This chapter reviews the research, industry reports, and existing platforms relevant to Novus: Smart Editorial and Publishing Platform. It is organized around the platform's principal design decisions — a structured editorial workflow, AI-assisted screening under human oversight, reader revenue through an owned platform, and a secure and accessible architecture — and closes with a comparison of Novus against the publishing systems a small Philippine publisher would otherwise adopt.

A. Editorial Workflow and Content Management Systems

The baseline targets for the Novus content pipeline — a scalable minimum of 50 articles per month, and at least a 50% reduction in the editor's active review time for each submitted article — are informed by documented trends in editorial efficiency. Cflow reports that 68% of content teams struggle with workflow inefficiencies, leading to a 30% drop in productivity [2], and that automating editorial processes can cut content turnaround time by up to 50% [2]. Amnet observes that where no defined workflow exists, projects depend on memory or disorganized chat threads, and that the absence of standardized workflow management causes significant turnaround delays [3] — the condition under which BOSS Media Philippines Inc. currently operates. Bastianello, Tomlinson, and Adamou's PubLink shows the alternative: a customized, sustainable workflow that moves manuscripts through structured formats to publication while fitting within limited budgets [4]. For managing that flow, Brechner presents Kanban as a method for making work visible and limiting work in progress [5], and a systematic review by Ahmad, Markkula, and Oivo identifies improved lead time, improved communication and coordination, and more consistent delivery among Kanban's main reported benefits [6]. Novus applies these principles by giving every article an explicit status — from assignment through drafting, assessment, review, revision, approval, and scheduling to publication — and by grouping each dashboard according to who can act on the work now, supported by a calendar of deadlines and release targets.

B. AI-Assisted Editorial Screening and Human Oversight

Recent research supports using a large language model as a first-pass evaluator of written work. Zheng et al. found that strong language models used as judges matched human preferences with over 80% agreement — the same level of agreement found between human evaluators themselves [7] — while also identifying systematic weaknesses in such judges, including position, verbosity, and self-enhancement biases [7]. Earlier work on automated essay scoring points the same way: in a national demonstration, automated scoring engines performed well on five of seven measures and approximated human raters on the remaining two [8]. The feasibility of assessing text quality automatically rests on findings that readers' judgments of readability can be predicted from measurable lexical, syntactic, and discourse features [9], and on a long tradition of formal readability measurement dating to Dale and Chall's formula [10].

Novus applies this research conservatively. Its AI evaluation service returns grammar and readability scores from 0 to 100, an overall score, and specific suggestions. The overall score places each submission in one of three verdicts: 70 and above passes to the editor; 40 to 69 returns to the writer with revision notes and quick fixes for mechanical errors such as spelling and agreement; and below 40 returns with guidance for

rework. Before any call is made, the platform checks that a draft is genuinely ready — rejecting placeholder text, unchanged resubmissions, and drafts below a minimum length — which prevents wasted assessments.

Because automated judgments carry known biases [7], the verdict is advisory rather than final. Research on automation shows that people tend toward complacency, meaning insufficient monitoring and verification of automated functions [11]. Novus counters this by requiring an editor who overrides a verdict to write a justification, which is recorded against the assessment; by showing editors exactly which AI-suggested fixes a writer accepted; and by reporting the override rate so the passing mark can be tuned against real editorial decisions. This design also reflects audience expectations. Across six countries, only 12% of people are comfortable with news made entirely by AI, rising to 21% with human oversight, 43% when a human leads with some AI help, and 62% for news made entirely by human journalists [12]. The 2024 Digital News Report likewise concluded that the public wants humans in the driving seat when publishers adopt AI [13]. In Novus, the AI assesses and suggests; it never writes the article, and its quick fixes are limited to mechanical corrections so that authorship remains the writer's.

C. Digital Media Sustainability and Owned Platform Strategy

Moving small independent publishers away from dependence on third-party social networks is a critical requirement for long-term viability, because reach on those platforms is volatile and outside the publisher's control. In the Philippines, the share of people using Facebook as a news source fell from 72% to 61% in a single year, an 11-point drop against a 4-point average across markets [13]. By 2026, social media and video networks had overtaken both television and news websites as sources of news [1], while direct and branded routes to news continued to lose ground [1]. A publisher that reaches its audience only through a platform's feed controls neither its visibility nor its reader relationship, which is why independent media are establishing owned platforms. On revenue, Deployteq reports that 80% of publishers are investing in subscription and membership models for stable, recurring income, with The Economist Group attributing more than 60% of its profits to subscription and reader revenue [14]. In the Philippines, local payment methods are central to that revenue: digital wallets, led by GCash, accounted for 41% of e-commerce transaction value in 2025 [15]. Novus is therefore designed to integrate PayMongo, a local gateway that supports these methods, rather than accepting cards alone. Automated billing also protects revenue once it is won: Recurly reports that automatically retrying failed payments recovered 73% of at-risk transactions and reduced average involuntary churn from 6% to 1% [16]. Finally, quality alone does not convert readers into subscribers — Hamari, Hanner, and Koivisto found that improving a freemium service had surprisingly little direct effect on premium purchases [17] — so Novus places checkout inside the reader portal and keeps the path from reading to subscribing short.

D. Related Systems

A small publisher seeking to replace chat-based operations would typically consider three established platforms.

WordPress is the most widely used content management system. Its core defines eight default post statuses, of which Draft and Pending Review are the only states before publication [18]. Multi-stage editorial

workflows therefore require additional plugins such as PublishPress Statuses, which add custom statuses on top of the two built-in ones [19].

Ghost is an open-source publishing platform with built-in memberships and paid subscriptions. Stripe is its only natively supported payment provider; other providers can be connected only through external services or its API [20].

Substack offers newsletters and paid subscriptions with no setup cost, but takes 10% of paid subscription revenue and relies on Stripe as its sole payment provider [21].

Table I Comparison of Publishing Systems

Capability	WordPress (core)	Ghost	Substack	Novus
Editorial stages before publication	Draft and Pending Review; more via plugins	Draft and publish, with roles	Draft and publish	Assignment through scheduling, with per-role queues
AI pre-screening with human override	Not built in	Not built in	Not built in	Built in, with recorded justifications
Magazine issues and layout approval	Not built in	Not built in	Not built in	Built in; publication requires an approved layout
Protected digital edition	Plugin-dependent	Not built in	Not built in	Flipbook served through short-lived signed links
Paid subscriptions	Plugin-dependent	Built in	Built in	Built in
Local e-wallet payments	Plugin-dependent	Through external tools	Not supported natively	Designed for PayMongo
Platform fee on subscriptions	Plugin-dependent	None	10%	None

Each platform addresses part of the problem. None combines a multi-role editorial pipeline, AI-assisted screening under human oversight, magazine issue production, and locally payable subscriptions in a single system. That combination is the gap Novus is designed to fill.

E. Modular Monolith Architectures

The modular monolith has emerged in recent literature as a middle ground between a traditional monolith and microservices: a single deployable application divided into modules with clear internal boundaries. Su and Li's review of industry practice concludes that it serves as an alternative to microservices and potentially as an intermediate step before a system migrates to them [22]. Novus adopts this approach in part. Its backend is a single Django service with one PostgreSQL database, organized into domain modules for accounts, editorial production, AI evaluation, issues and publishing, content delivery, design, notifications, messaging, reports, and platform settings. The most consequential responsibilities are reached through a single entry point each: all communication with the AI provider passes through one service module, all

notifications through one function, all publication rules through one publishing service, and every access decision on paid content through one entitlement check. Other modules still share models directly, and the boundaries are maintained by convention rather than enforced by tooling. This keeps the transactional simplicity of one codebase, which suits a small team, while isolating the components most likely to change or to be separated later — AI evaluation in particular, whose provider can be replaced without altering the editorial workflow.

F. Decoupled Web Frameworks

The integration of a reactive frontend built with Vue.js and a backend API built on Django REST Framework (DRF) follows the decoupled architecture common in modern full-stack development. Vue.js, through its Composition API and component-based model, provides a reactive user interface that communicates with DRF endpoints over HTTP. DRF supports this separation by exposing RESTful APIs with built-in authentication, serialization, and pagination [23], while cross-origin rules restrict requests to the platform's own frontend. The frontend and backend can therefore evolve independently, with Vue managing presentation concerns such as role-based views and client-side routing, and Django handling domain logic, persistence, and security. For Novus, this supports distinct public, subscriber, and editorial interfaces over a single API layer, ensuring consistent data access across all of them.

G. Security and Protected Content Delivery

The OWASP Top 10:2025 ranks broken access control as the most serious web application security risk, followed by security misconfiguration, and lists authentication failures among the ten most critical risks [24]. Novus addresses each. Every API endpoint checks the caller's role on the server, and the frontend's route guards serve only as a convenience, never as the security boundary. Sessions use JSON Web Tokens [25] that expire after one hour and are renewed through a separate token that rotates on each use. Sign-in attempts are limited to five per minute for each account, keyed on the account rather than the caller's network address, which cannot be trusted behind a proxy. All traffic is served over HTTPS with Strict Transport Security, cross-origin requests are accepted only from the platform's own frontend, and credentials for the database, storage, and external services are held in environment configuration rather than in code. Magazine editions — the product subscribers pay for — are held in a private storage bucket and delivered through presigned URLs, which grant a single operation on a specific object without issuing credentials [26]. Novus issues these links only after checking the reader's entitlement, and they expire after fifteen minutes; the same mechanism lets designers upload print-resolution editions directly from the browser to storage.

H. Accessibility and Data Privacy

Web Content Accessibility Guidelines (WCAG) 2.2 define how web content can be made accessible to people with disabilities [27]. Novus applies its practices across the reader portal and workspace: controls are operable by keyboard, including page navigation in the digital edition; form fields and icon-only buttons carry text labels for screen readers; and both the digital edition's page-turn effect and the sign-in page's animated background respect the operating system's reduced-motion preference. Personal data is processed in line with the Philippine Data Privacy Act of 2012 [28]. At registration, consent to the terms of use is

recorded separately from consent to marketing, and the platform collects only the information its functions require.

I. Synthesis

The literature establishes four points on which Novus is built. First, structured workflows with visible status reduce delays compared with coordination through chat [2]–[6]. Second, language models can approximate human judgment of writing quality well enough to serve as a first screen, but not well enough to act alone, which makes human oversight a requirement rather than an option [7], [8], [11]–[13]. Third, sustainable independent publishing depends on an owned platform, recurring reader revenue, and payment methods readers actually use [13]–[17]. Fourth, existing platforms each address part of this need, but none combines editorial production, AI-assisted screening under human control, magazine issue production, and locally payable subscriptions in one system [18]–[21]. Novus is designed to close that gap.

Technical Background

The current system of BOSS Media Philippines Inc. operates entirely without dedicated software or digital infrastructure. All editorial and publishing processes are carried out through a combination of consumer-grade communication applications and general-purpose productivity tools. The primary platforms in use include Facebook Messenger, Viber, WhatsApp, Gmail, and Google Docs, which serve simultaneously as briefing channels, submission inboxes, revision communication tools, and publication coordinators. No editorial management software, content management system, project tracking tool, or publishing platform has been formally implemented.

In terms of hardware, the team operates using personal computers and mobile devices owned by individual staff members and writers. There is no shared server, internal network, or cloud workspace dedicated to editorial operations. File storage and version control are handled informally through Google Drive folders and shared Google Docs links, with no naming convention, folder structure, or access control policy in place.

The graphic designer works using Canva and other design applications on personal devices, with no shared design system or file handoff protocol. The publisher manages the physical printing and social media distribution manually, coordinating through the same consumer messaging platforms used throughout the rest of the workflow.

Current System

Table 1 lists all current processes in the editorial and publishing workflow of BOSS Media Philippines Inc. Each process is identified by a unique process ID, a process name, and a description of how the process is currently performed. These processes were documented through direct data gathering with the Editor and Publisher and validated through the writer survey conducted in March 2026.

TABLE I.

Table II List of Processes in the Current System of BOSS Media Philippines Inc.

Process ID	Process Name	Process Details
P001	Topic Assignment	The Editor assigns a topic or story angle to a Writer through Facebook Messenger, Viber, Gmail, or a verbal conversation. No standardized brief template or deadline system is used.
P002	Article Writing	The Writer drafts the article using Google Docs or a notes application. The Writer determines formatting independently as no writing guide is provided.
P003	Article Submission	The Writer submits the completed article through the same fragmented communication channels used for the assignment. No centralized submission portal exists.
P004	Editorial Review	The Editor manually reviews the submitted article for grammar, tone, structure, and brand alignment. This takes one to two days per article, with 61 to 80 percent of submissions requiring at least one revision.

P005	Feedback & Revision	The Editor sends revision notes through private messaging. The Writer revises and resubmits within three to five days on average. This loop repeats for two to three rounds per article.
P006	Article Approval	The Editor approves the final version and informs the Writer through chat. Only three to four out of every ten articles are approved on the first submission.
P007	Layout Production	Approved articles are forwarded to the Graphic Designer through chat or email. The Graphic designer arranges the magazine layout with no formal design brief or file-sharing system.
P008	Layout Review	The completed layout is sent back to the Editor for a final check. Revision notes are relayed again through informal messaging.
P009	Publishing	Once the layout is confirmed, the Publisher receives the final files and manually publishes the issue to social media platforms and coordinates physical printing. No publishing schedule system is used.

SWOT Analysis

The SWOT analysis below evaluates the internal strengths and weaknesses of BOSS Media Philippines Inc. as an organization, as well as the external opportunities and threats it faces in the current Philippine media landscape. Each point is derived from the data gathering conducted with the editorial team and from the broader industry context established in the literature review.

TABLE II.

Table III SWOT Analysis of BOSS Media Philippines Inc.

Strengths	Weaknesses
• Established publication identity with a Filipino-focused editorial voice and brand • Dedicated and committed core team willing to improve the current workflow • Existing readership base on Facebook and Instagram • Content covers a broad range of topics increasing discoverability potential	• No centralized editorial workflow or project tracking system in place • Single editor handling 10+ articles per cycle with no automation support • 61–80% of articles require revision; only 3–4 out of 10 are approved on first submission • Zero direct revenue generation from published content • No formal publishing schedule; delays regularly exceed 2 weeks
Opportunities	Threats
• Filipino Facebook news reliance dropped 11 points in 2024, creating demand for owned platforms (Newman et al., 2024) • 80% of publishers globally are investing in subscription models as a primary revenue stream (Deployteq, 2024) • Commercial LLM APIs now make AI-assisted editorial review accessible to small publications • Growing Filipino internet penetration and mobile-first consumption opens wider digital audiences	• Continued reliance on Facebook risks further reach reduction as algorithm changes deprioritize media content • Growing competition from digital-native Filipino media brands with existing platforms and revenue models • AI-generated content flooding the media space raises audience trust concerns across all publications • Operational burnout risk — single editor handling a growing workload with no system support

Proposed System

A. Technical Background & Architecture

Novus is a cloud-native editorial management system deployed on a managed platform environment, with the application, its relational database, and its object storage provisioned as independent managed services rather than assembled on self-administered infrastructure.

The data layer consolidates what is conventionally a multi-store configuration into a single relational database holding transactional user data, active document states, and append-heavy audit records, using native structured-text fields in place of a separate document store. File assets reside outside the database in object storage under two access policies: article photography is publicly served and cached, while magazine editions are private and retrievable only through short-lived signed URLs. The paywall therefore governs the file itself rather than the link to it.

The system presents a single web frontend serving two distinct audiences — a public reader portal and an unlinked staff workspace — and rests on four feature pillars.

- **Visual Pipeline Tracking.** A role-aware dashboard replaces fragmented messaging channels, giving each participant centralized visibility of article state across the full submission-to-publication pipeline, with every transition recorded against the article.
- **Unified AI-Powered Editorial Assistant.** A single generative language model, constrained to structured output, performs grammatical assessment and editorial brief extraction in one pass, limiting token consumption and latency relative to sequential calls. The model is reached through one internal service interface, making the provider a replaceable dependency rather than an architectural commitment.
- **Targeted Notification Protocol.** A server-side email service carries transactional messages exclusively — assignments, deadlines, and state transitions — deliberately excluding digest and activity notifications, the class of message that drove the previous workflow into personal messaging apps.
- **Streamlined Payment Infrastructure.** Subscription access is handled through a single localised payment gateway covering domestic e-wallets and local cards, matched to the readership rather than to maximum coverage.

B. First Pass Review: Scoring Framework (Writer Side)

Before an article reaches an editor, it is pre-screened by the AI evaluation service, which returns a grammar score, a readability score and an overall score, each from 0 to 100, together with suggestions for improvement.

- **Pass (70 and above):** the article advances to editorial review.
- **Revise (40 to 69):** the article returns to the writer with revision notes and quick fixes for mechanical errors such as spelling and agreement. Accepting a fix does not change the score; the article is scored again when resubmitted.
- **Rewrite (below 40):** the article returns with guidance for rework and no quick fixes.

- **Readiness check:** drafts that are too short, contain placeholder text, or are unchanged since their last assessment are refused before any assessment is made.

Human-in-the-Loop (HITL) Override: editors may override a verdict to accommodate intentional stylistic choices. Every override requires a written justification, which is recorded, and the override rate is reported so the passing mark can be adjusted over time.

C. Contextual Narrative Analysis (Editor's Side)

Articles clearing the first pass trigger a background evaluation producing a private **Editor's Brief**, attached to the article record and visible only to editors and publishers. The writer receives the composite score; the brief is an editorial instrument rather than feedback to be actioned directly.

Hook and reader engagement. Examines the opening paragraphs for a clearly stated main point and an identifiable stake for the reader.

Story flow and logic. Traces structural coherence, flagging timeline jumps, absent background, and weak transitions between sections.

Targeted content extraction. Surfaces two artefacts for downstream use — the *Marketing Highlight*, a standalone sentence suitable for social distribution and design treatment, and the *Editorial Bottleneck*, the single passage most in need of human rewriting.

D. Change Management and User Training

To ensure platform adoption, the implementation includes:

- **Onboarding Session:** Live walkthrough demonstrations for all platform users.
- **Digital Handbook:** Reference documentation detailing submission formats and revision protocols.
- **Pilot Phase:** Running Novus alongside existing workflows for at least one issue prior to full migration.

E. Evidence Collection & Validation Framework

The platform evaluates its operational, financial, and technical objectives using a multi-tiered data-collection framework:

- **Time-in-Status Tracking:** Logs chronological duration per pipeline stage to quantify reductions in review times and measure publication throughput.
- **Training Effectiveness:** Tracks post-training user error rates to validate digital onboarding materials.
- **Automated Override Rate:** Monitors HITL override frequencies against legacy baseline approvals to calibrate pre-screening algorithms.
- **System Usability:** Uses standardized surveys to measure cognitive load reduction compared to disjointed legacy messaging apps.
- **Revenue vs. Cost Analysis:** Tracks subscription revenue against unified cloud hosting and API expenses to prove economic sustainability.

- **Technical Audits:** Executes automated web performance tests to verify SEO, accessibility, and speed benchmarks.

To develop and deploy Novus, we have selected technologies including:

- **Vercel:** A platform that supports seamless deployment and global hosting services for our Vue.js frontend, providing automatic edge-caching, continuous integration, and secure SSL.
- **Render:** A modern cloud platform utilized to host our Django backend API and managed PostgreSQL database, offering automated builds and scalable infrastructure without heavy operational overhead.
- **Cloudflare R2:** An S3-compatible object storage solution chosen for the cost-effective and highly available storage of dynamic website assets, article media, and user uploads.
- **Vue.js:** A progressive JavaScript framework selected for the frontend to manage dynamic client-side rendering, user interfaces, and role-based routing as a Single Page Application (SPA).
- **Django & Django REST Framework:** A high-level Python web framework chosen for the backend API to enforce centralized business logic, securely manage token-based authentication (SimpleJWT), and expose robust RESTful endpoints.
- **PostgreSQL (SQL):** An advanced relational database ideal for ensuring data integrity and storing core platform state, such as user profiles, editorial articles, and subscription ledgers.
- **PayMongo:** A unified payment gateway integrated to securely process localized e-wallet transactions and credit card subscriptions through a streamlined API.
- **Resend:** A developer-focused email service utilized to reliably construct and dispatch transactional communications, including system notifications and digital receipts.
- **Anthropic Claude API:** An advanced artificial intelligence engine integrated to evaluate editorial submissions and generate deterministic narrative briefs utilizing structured outputs.

Our tech stack choices prioritize solutions that enforce a secure, decoupled architecture, optimize the editorial workflow, and provide the essential scalability, performance, and maintainability required for a modern digital publication.

Project Feasibility

Operational Feasibility

The operational feasibility of the Novus platform is highly promising, as it directly resolves the critical pain points identified in recent stakeholder surveys. To address disjointed workflows and unclear timelines, the system introduces centralized editorial pipelines with automated draft status transitions, ensuring complete transparency for writers, editors, and managers. Furthermore, communication gaps and fragmented ideation are mitigated by embedding contextual editorial feedback and deterministic AI-generated scoring directly into the review process before human editors intervene. Finally, the shift to a cloud-native, browser-based workspace eliminates physical and operational friction, allowing the editorial team to collaborate seamlessly regardless of their location, operating system, or hardware limitations.

Economic Feasibility

From a financial perspective, the project demonstrates exceptional viability through its strategic reliance on cost-effective, modern infrastructure. The initial development and staging phases require zero upfront capital expenditure, leveraging the generous free tiers provided by Vercel, Cloudflare R2, Render, and Resend. As the platform transitions into a live production environment, the baseline operational expenditure remains highly predictable and remarkably low, starting at approximately PHP800 per month for managed compute and database hosting. Additionally, external integrations like PayMongo and the Anthropic Claude API utilize strict pay-as-you-go models, ensuring that infrastructure costs scale strictly in proportion to user adoption and subscription revenue generation, thereby securing a strong return on investment.

Technical Feasibility

The technical foundation of Novus is built upon stable, highly supported frameworks, ensuring a high degree of technical feasibility. By utilizing a decoupled architecture consisting of a Vue.js frontend and a Django REST Framework backend, the system guarantees a strict separation of concerns, which enhances both long-term maintainability and independent scaling. Data integrity is secured through PostgreSQL, which handles complex relational data—such as subscription ledgers and user profiles—while utilizing native JSON capabilities to store dynamic AI-generated briefs with ACID-compliant reliability. Furthermore, delegating specialized functions to established enterprise APIs drastically reduces custom development overhead and mitigates the risks associated with infrastructure maintenance.

Schedule Feasibility

The development of Novus is strategically phased across three academic terms under the Project-Based Learning curriculum, ensuring a modular and low-risk delivery:

- **Phase 1 (MNTSDEV):** Focuses on system design, requirement elicitation, and the development of the initial high-fidelity prototypes for the editorial dashboard.
- **Phase 2 (MSYSADD):** Dedicated to technical modeling, including UML diagrams and the architecture of the AI evaluation service and unified database schema.
- **Phase 3 (MCSPROJ):** Encompasses the final coding phase, unified API integration, rigorous system testing, and deployment to the production environment.

Requirements Analysis

Product Vision

The vision for Novus is to democratize professional-grade editorial infrastructure for independent publishers worldwide, transforming fragmented content production into a streamlined, high-output operation. By centralizing the editorial pipeline — from initial assignment to final approval — Novus aims to eliminate the administrative friction that typically stifles small-to-medium media houses. Through the integration of AI-assisted review systems, the platform empowers editorial teams to maintain rigorous standards without the need for massive human headcount, ensuring that even the smallest publications can produce polished, content at the speed of modern digital discourse.

Beyond operational efficiency, Novus envisions a future where creators reclaim control over their financial destiny through owned platforms and direct reader engagement. The mission is to move publishing away from total reliance on social media algorithms and toward a sustainable 'freemium' ecosystem that converts passive scrollers into invested subscribers. By providing a built-in, secure localized payment gateway and an owned, searchable reader portal, Novus equips publishers with the professional digital presence and monetization strategies required to turn a labor of love into a profitable international brand, ensuring the long-term survival and independence of diverse voices in the global media landscape.

Project Lean Canvas

The Lean Canvas summarizes the key strategic elements of Novus, including the identified problem, proposed solution, unique value proposition, unfair advantage, customer segments, key metrics, channels, cost structure, and revenue streams.

Problem - Editor manually reviews 10+ articles per cycle with no automation – single point of failure - No centralized system for submission, revision tracking, or deadline management - BOSS Magazine PH has no website and no revenue stream from its published content	Solution - AI-assisted first-pass review for grammar and readability - Centralized editorial dashboard for submissions, revisions, approvals, and deadline tracking - Public magazine website with subscription and digital issue sales	Unique Value Proposition - The only all-in-one editorial and publishing platform built for independent Filipino magazines - Manage writers, enforce deadlines, publish professionally, and earn from content without juggling chat threads	Unfair Advantage - Built on real validated client data – not hypothetical - First AI editorial system deployed in local independent PH magazine publishing - Direct client access enables fast iteration	Customer Segments - BOSS Magazine PH editor and publisher – primary users - Writers submitting content to the magazine - Filipino fashion and culture readers – subscribers and digital issue buyers
Existing Alternatives - Messenger, Viber, email for article submission and revision coordination - Google Docs for drafting with no submission or review system - The only platforms that highlight BOSS magazine's presence are Facebook, Instagram, and a non-operational WebToolz website. - No subscription or payment system – all content free	Key Metrics - Publishing cycle time reduced from 2+ weeks - Editor hours saved per week from 31-40 hours - Messages reduced from 21-49 - Number of active reader subscriptions - AI review accuracy - Revenue generated per publishing cycle	High Level Concept - Novus is Notion + Substack + AI Editor for independent Filipino magazine publishers - One platform from brief to publish to revenue	Channels - Direct deployment at BOSS Magazine PH as first live client - BOSS Magazine PH social media driving readers to the platform - Word of mouth in the local independent publishing community	Early Adopters - BOSS Magazine PH editor and publisher - Current BOSS Magazine PH writer roster - Existing BOSS Magazine PH social media followers who become first subscribers
Cost Structure - AI API Fees: Variable usage costs for LLM-powered editorial reviews - Cloud & Storage: Monthly costs for app hosting, databases, and digital media storage - Payment Processing: Transaction-based fees for subscriptions and digital sales. - Domain & Security: Fixed annual costs for the domain, SSL, and professional email.		Revenue Streams - Monthly reader subscriptions – recurring access to premium magazine content - Annual reader subscription plan – yearly rate - Single digital issue purchases – one-time buy per magazine issue - Future: platform licensing to other small publishers who adopt		

Figure 1 Novus Project Lean Canvas

Product Backlog / User Stories

TABLE VI.

Table IV Product Backlog

Persona	Story Title	User Story	Acceptance Criteria
Writer	Login and Role-Based Redirect	As a Writer, I want to log in and be automatically directed to my Writer Dashboard, so that I can access my	1. Writer opens Novus and lands on the Login / Role Landing screen.

		workspace without manual navigation.	2. Writer enters registered email and password, then taps “Log In.” 3. If invalid, system shows a pop-up “Incorrect email or password. Please try again.” Writer remains on the Login screen with the password field cleared. 4. If valid, system shows “Login Successful” and navigates to the Dashboard Redirect screen, confirming the role “Writer.” 5. System automatically navigates the Writer to the Writer Dashboard within 2 seconds — no further action is needed.
Writer	Viewing the Writer Dashboard	As a Writer, I want to see my active, in-revision, and approved articles as soon as I log in, so that I know what needs my attention.	1. On arrival, system displays three counters: Active, In Revision, Approved, and a task list of assigned/drafted articles with Title, Status Badge, and Deadline. 2. Writer taps “+ New Submission” → system navigates to a blank Article Submission Form. 3. Writer taps an existing “Drafting” article → system navigates to the Article Submission Form pre-filled with the saved draft. 4. Writer taps an article marked “Needs Revision” → system navigates to the Revision Notes Inbox. 5. No confirmation pop-up is required here since these are navigation-only actions.
Writer	Viewing the Pinned Assignment Brief	As a Writer, I want my assignment brief pinned to my writing screen, so that I always know the topic, word count, tone, and deadline without leaving the editor.	1. When the Article Submission Form opens for an assigned topic, system displays a collapsible “Assignment Brief” panel. 2. Panel shows Topic, Word Count Target, Tone, and Deadline as read-only fields. 3. Writer taps the collapse icon to show/hide the panel; it remains visible while scrolling the article body. 4. Writer cannot edit any field in the panel — tapping into it produces no cursor and saves no changes. 5. This panel persists through drafting, auto-save, and submission.

Writer	Drafting an Article with Cloud Auto-Save	As a Writer, I want my draft to save automatically in the cloud as I type, so that I never lose progress.	1. Writer enters Article Title, selects Category, enters Word Count, and types the body. 2. System auto-saves every 30 seconds of activity, or immediately after 3 seconds of typing pause. 3. On successful save, system shows “Saved to Cloud” near the top of the editor, fading after 2 seconds; Writer remains on the same screen. 4. If the network drops, system shows a persistent banner “Offline — changes will sync once reconnected” and keeps saving locally. 5. Once reconnected, system auto-syncs, briefly shows “Synced to Cloud” for 2 seconds; Writer remains on the same screen. 6. Writer may tap “Save Draft” manually at any time — same confirmation applies, no navigation change.
Writer	Submitting an Article for AI Pre-Screening	As a Writer, I want to submit my finished draft for automated review, so that I get an initial quality score before the Editor sees it.	1. Once drafting is complete, Writer taps “Submit for Review.” 2. System shows a confirmation pop-up “Submit this article for AI review? This will lock further edits until scoring is complete.” with Cancel / Confirm. 3. Writer taps “Cancel” → pop-up closes, Writer remains on the Article Submission Form, no status change. 4. Writer taps “Confirm” → system changes status to “In Review,” shows “Article submitted successfully,” and navigates to the AI Pre-Screen Result screen.
Writer	Viewing the AI Pre-Screen Result	As a Writer, I want to see my grammar and readability score right after submitting, so that I know whether my article advances or needs revision.	1. System displays the composite score out of 100 (Grammar + Readability breakdown) within 60 seconds of submission. 2. If score is 70 or higher, system shows “Your article passed pre-screening and has been sent to the Editor.” Writer taps “Continue” → returns to the Writer Dashboard. 3. If score is below 70%, system shows “Your article did not meet the minimum score and has been returned for revision.” Writer taps “Revise Now” → navigates to the Article Submission Form with

			flagged sentences highlighted, or taps “Later” → returns to the Writer Dashboard.
Writer	Tracking Submission Status	As a Writer, I want to track my article’s progress through the pipeline, so that I always know its stage without asking the Editor directly.	1. Writer taps “View Status” on a submitted article → opens the Submission Status View. 2. Screen shows a vertical tracker: Submitted → AI Review → Editor Review → Revision Requested → Approved → Published, with the current stage highlighted. 3. Writer taps “Back” → returns to the Writer Dashboard; no confirmation pop-up needed. 4. When the stage changes, system adds a notification badge; tapping it navigates directly to the relevant screen.
Writer	Responding to Revision Notes and Resubmitting	As a Writer, I want the Editor’s revision notes attached directly to my draft, so that feedback is clear and I can resubmit once I’ve made changes.	1. Writer opens the Revision Notes Inbox; system displays each comment beside its corresponding paragraph, with Editor name and timestamp. 2. Writer taps “View Full Article” → opens the Article Submission Form with the comments shown inline. 3. Writer edits the flagged sections; auto-save applies throughout. 4. Writer taps “Mark Resolved” on an addressed comment → system shows “Marked as Resolved” and collapses that comment; Writer remains on the same screen. 5. Once all comments are resolved, Writer taps “Submit Revised Draft.” 6. System shows a confirmation pop-up “Resubmit this article to the Editor?” with Cancel / Confirm. 7. Writer taps “Confirm” → system shows “Revised article resubmitted successfully,” changes status to “In Review,” and navigates the Writer back to the Writer Dashboard.
Writer	Receiving Deadline Reminders	As a Writer, I want visual and automated reminders for my due dates, so that I don’t miss the deadline.	1. On the Writer Dashboard, each active article shows a color-coded deadline badge: Yellow (< 48 hrs), Red (< 24 hrs). 2. System automatically sends an email reminder 24 hours before the deadline.

			3. System sends an in-app notification 1 hour before the deadline; tapping it navigates directly to the Article Submission Form. 4. If the deadline passes without submission, system flags the article “Overdue” and moves it to the top of the task list.
Editor	AI Pre-Screening Engine	As an editor, I want the system to check articles before I read them, so I only spend time on good submissions.	1. Writer sends in an article. 2. The system scores grammar and readability from 0 to 100. 3. Verdict: pass (70+), revise (40–69), or rewrite (below 40). 4. Articles below 70 are returned to the writer. 5. The editor gets a simple report of what was flagged. 6. The editor can accept a returned article with a short explanation.
Editor	Sprint Board	As an editor, I want to see all articles and where they are in the process, so I always know what is going on.	1. Editor opens the board. 2. The board shows all articles sorted by their current stage. 3. Each article shows the title, the writer, and the due date. 4. Editor can move articles from one stage to another. 5. Editor can search or filter articles by writer or due date.
Editor	Approvals and Rejections	As an editor, I want to approve or send back an article with one click, so the process is quick and easy to track.	1. Editor finishes reading an article. 2. Editor clicks Approve to mark it as done and lock it from further changes. 3. Or, editor clicks Send Back to return it to the writer for changes. 4. Editor writes a short note explaining what needs to be fixed. 5. The system saves a record of what was decided and when.
Editor	Reassignment and Priority	As an editor, I want to move an article to a different writer or mark it as urgent, so nothing gets stuck or delayed.	1. Editor opens an article. 2. Editor picks a different writer from the list to take over. 3. The new writer gets notified right away.

			4. Editor can mark the article as urgent so it stands out. 5. The change is saved so everyone can see what happened.
Graphic designer	File Submission	As a graphic designer, I want a place to upload my work, so it is always connected to the right article.	1. Graphic designer opens the upload page. 2. Graphic designer chooses which article the files belong to. 3. Graphic designer uploads the image or layout files. 4. The system confirms the upload was successful. 5. The editor gets notified that new files are ready to review.
Graphic designer	File Versions	As a graphic designer, I want to upload updated versions of my work without losing the old ones, so the editor can see the changes clearly.	1. Graphic designer uploads a new version of a file. 2. The system keeps the old version and saves the new one separately. 3. Both the graphic designer and editor can view all versions. 4. The editor can leave feedback on any version. 5. The Graphic designer marks one version as the final one when ready.
Publisher	Publishing and Deadlines	As a publisher, I want to set issue deadlines and batch-publish approved articles, so that I can automatically release complete issues without manual publishing errors.	1. Publisher opens 'Issue Management', selects a target release date via datetime picker, and clicks 'Save', which updates the global issue release date and syncs the countdown timer across all user dashboards. 2. Clicking 'Publish Issue' triggers an automated system check on all linked articles; if any article is in 'Draft' or 'Review' status, the system aborts execution and displays an error modal listing the unapproved articles. 3. When all articles are in 'Approved' status, confirming the prompt executes a database batch update setting all article statuses to 'Published' and populating published timestamps simultaneously. 4. Upon completion, the system updates the issue status to 'Live', displays a success modal with the total count of published articles, and provides a direct public preview link.

Publisher	Financial and Subscription Dashboard	As a publisher, I want a real-time revenue and subscription dashboard, so that I can track Monthly Recurrent Revenue (MRR) and platform monetization trends.	1. Upon loading the financial dashboard, the system queries backend APIs to display core summary metrics: Total Active Subscribers, Current MRR, and Recent Transactions. 2. Selecting a date filter ('Weekly', 'Monthly', or 'Yearly') issues an API query that dynamically re-renders the 12-month subscriber growth chart and transaction table without a full page reload. 3. When payment channel triggers a 'charge.paid' or 'subscription.created' webhook, the server validates the payload signature, records the transaction, and pushes dynamic updates to the dashboard view within 3 seconds. 4. System automatically aggregates daily subscription totals every 24 hours at 00:00 UTC to maintain rapid query speeds for historical data.
Publisher	Editorial Workflow and Assignment Tracking	As a publisher, I want to assign articles to specific editors and set section deadlines, so that I can manage staff workload and ensure balanced issue coverage.	1. Publisher opens 'Editorial Workspace', views unassigned draft submissions and selects an assigned Editor from a dropdown menu. 2. System updates the article assignment record, changes status to 'In Review', and sends an automated notification to the designated Editor's dashboard. 3. Publisher sets individual section deadlines (e.g., Feature, News, Opinion), which automatically calculates and displays workload progress bars across all active team members. 4. Publisher receives a real-time status alert if any assigned article remains untouched within 24 hours of its section deadline.
Admin	Role-Based Access Control (RBAC)	As an Admin, I will first log in using my administrator account. After successfully logging in, I will click User Management from the dashboard menu to open the user management screen. I will select a user from the list, then click Edit Role to change the user's role (Writer, Editor, or Publisher etc.) or click Suspend Account if necessary. After clicking Save or Confirm, I want the system to immediately update the user's permissions so they can only	1. Admin can log in and access the User Management page. 2. Admin can create, edit, suspend, and reactivate user accounts. 3. Admin can assign or change a user's role (Writer, Editor, Publisher). 4. Users immediately receive their updated permissions without logging out. 5. A confirmation dialog appears before suspending an account.

		access the menus and features assigned to their role.	6. After saving changes, a success notification (e.g., User role updated successfully.) appears. 7. The system remains on the User Management page and automatically refreshes the user list. 8. Every account creation, role update, suspension, or reactivation is recorded in the audit log.
Admin	API Token Monitoring	As an Admin, I will log in and open the Admin Dashboard. I will click the API Monitoring menu to view the API usage screen. I want to see the total API requests, monthly usage, estimated cost, and remaining quota for each AI service. If usage approaches the monthly limit, I want the system to notify me so I can take action before exceeding the budget.	1. Admin can open the API Monitoring page from the dashboard. 2. Dashboard displays total API requests, remaining quota, monthly usage percentage, and estimated cost. 3. API usage is grouped by service (e.g., article scoring, editor's brief). 4. When usage reaches 90% of the monthly limit, a warning notification is displayed. 5. When usage exceeds the limit, an alert notification appears on the dashboard. 6. Dashboard data refreshes automatically or through a Refresh button. 7. If the page refreshes successfully, a notification confirms that the latest data has been loaded.
Admin	Global Settings and Maintenance	As an Admin, I will log in and open the Settings menu from the dashboard. I will select Global Settings to access the maintenance settings screen. I will enable or disable Maintenance Mode using a toggle switch or schedule a start and end time for automatic activation. After saving the changes, I want the website to immediately display the maintenance page to public visitors while allowing internal users to continue accessing the system.	1. Admin can open the Global Settings page. 2. Admin can enable or disable Maintenance Mode using a toggle switch. 3. Admin can schedule automatic start and end times for maintenance. 4. A confirmation dialog appears before enabling Maintenance Mode. 5. After clicking Save, a success notification (e.g., Maintenance settings saved successfully.) is displayed. 6. Public users immediately see the "Coming Soon" page when maintenance is active. 7. Internal users (Writers, Editors, Publishers) can still log in and access the backend.

			8. After saving, the system stays on the Global Settings page and displays the updated configuration.
Reader	Landing on the Public Homepage	As a Reader, I want a clear homepage when I visit the site, so that I can immediately see featured and recent content.	1. Reader navigates to the BOSS Magazine PH website URL. 2. System loads the Homepage: hero article banner, category navigation bar, and a grid of latest articles. 3. No login is required to view this screen. 4. Reader taps a category → proceeds to the Article Listing Page, or taps an article → proceeds to the Single Article View, or taps the language toggle.
Reader	Browsing Articles by Category	As a Reader, I want to filter articles by category, so that I can find content relevant to my interests.	1. Reader taps a category tag from the Homepage or nav bar. 2. System navigates to the Article Listing Page, showing only articles tagged with that category. 3. Reader may type a keyword into the search bar; system filters results in real time and shows “No results found” if nothing matches. 4. Reader taps an article card → proceeds to the Single Article View, or taps “Back” → returns to the Homepage with scroll position preserved.
Reader	Reading an Article and Encountering the Paywall	As a Reader, I want to read an article’s content and be clearly told if it requires a subscription, so that I understand what’s free versus paid.	1. Reader taps an article card from the Homepage or Article Listing Page. 2. System navigates to the Single Article View and renders the headline, byline, and body. 3. If the article is free-tier, the full body renders with no interruption. 4. If the article is premium, system renders the headline and first two paragraphs, blurs the rest, and shows a “Subscribe to read the full article” button. 5. Reader taps “Subscribe” → proceeds to the Subscription Page, or taps “Back” → returns to the previous screen with no state change.
Reader	Registering for an Account	As a Reader, I want to create an account, so that I can proceed to subscribe for premium access.	1. Reader taps “Register” from the nav bar or the paywall prompt.

			2. System navigates to the Registration screen with fields for Full Name, Email, and Password. 3. Reader fills in the fields and taps “Create Account.” 4. If any field is invalid, system shows inline error text and keeps the Reader on the Registration screen. 5. If valid, system shows “Account created successfully! Welcome to BOSS Magazine PH” and automatically logs the Reader in. 6. System navigates the Reader to the Subscription Page, since registration alone does not unlock paywalled content.
Reader	Selecting a Subscription Plan	As a Reader, I want to choose between a monthly or annual plan, so that I can pick what fits my budget.	1. Subscription Page shows two plan cards: “Monthly” and “Annual” (Annual marked “Best Value”). 2. Reader taps a plan card → system highlights it and updates the “Continue” button to reflect the chosen plan and price. 3. Reader taps “Continue” → system navigates to the Payment Checkout screen, carrying over the selected plan. 4. Reader may instead tap “Back” → returns to the previous screen with no charge and no confirmation pop-up.
Reader	Completing Payment and Becoming a Subscriber	As a Reader, I want to check out securely with my preferred payment method, so that I can unlock premium content immediately.	1. Reader selects a payment method (GCash, Maya, Credit Card, or Debit Card) and enters the required details. 2. Reader taps “Pay and Subscribe”; system processes the transaction through the payment gateway. 3. If payment fails, system shows “Payment unsuccessful. Please try again or use a different method.” Reader remains on the Checkout screen; sensitive card fields are cleared. 4. If payment succeeds, system shows “Payment successful! Your subscription is now active” and upgrades the account from Reader to Subscriber. 5. System sends a payment confirmation email with receipt and subscription details.

			6. System automatically navigates the newly upgraded Subscriber back to the article originally attempted, now fully unlocked.
Reader	Subscribing to the Email Newsletter	As a Reader, I want to opt into the email newsletter, so that I receive updates without checking the site manually.	1. Reader scrolls to the newsletter signup section on the Homepage or footer. 2. Reader enters an email address and taps "Subscribe to Newsletter." 3. If the email format is invalid, system shows inline error text and keeps the Reader on the same screen. 4. If valid, system shows "You're subscribed! Check your inbox for a welcome email." and the field updates to show "Subscribed."
Subscriber	Paywall Bypass	As a subscriber, I want my active account to instantly unlock premium articles, so that I can read paid content seamlessly across all my devices.	1. When a user visits a premium article in URL, backend middleware verifies their session token and checks their account subscription status. 2. If subscription status is 'Active', the server renders the complete article body with no blur overlays or truncation. 3. Authenticating with active subscriber credentials on any secondary device or browser immediately grants full reading permissions via session validation. 4. If status is 'Expired' or 'Unauthenticated', the system truncates text at 200 words, applies a blur overlay to the remaining text, and presents a 'Renew/Subscribe to Continue Reading' CTA.
Subscriber	Bookmark and Save	As a subscriber, I want to save specific articles into custom folders, so that I can organize and re-read them later from my profile.	1. Clicking the 'Save' icon on an article opens a selection popover displaying existing user folders and a '+ New Folder' input field. 2. Selecting a folder or creating a new one sends a POST request to save the record to the database, changing the save icon to an active filled state and showing a success toast. 3. Saved items populate under their corresponding folder categories in the 'My Library' tab on the profile page. 4. Clicking 'Remove' on a saved article issues a DELETE request, animating the item out of the DOM and immediately removing it from the user's library list.

Subscriber	Subscription Management	As a subscriber, I want to view my billing details and manage auto-renewal, so that I retain full control over my recurring payments.	1. Navigating to 'Account Settings > Billing' displays the current subscription tier, status ('Active', 'Past Due', or 'Canceled'), and next renewal date. 2. Clicking 'Cancel Subscription' opens a retention prompt offering alternative choices ('Keep My Subscription' or 'Proceed to Cancel'). 3. Selecting 'Proceed to Cancel' and confirming sends an API request to payment channel to disable recurring auto-renewal for the account. 4. Account status updates to 'Pending Cancellation', premium access remains active until the end of the current billing cycle, and an automated confirmation email is sent with the exact access expiration date.
All Roles (System-Wide)	Forgot Password Recovery	As a registered user (any internal role or Subscriber), I want to reset my password if I forget it, so that I can regain access without contacting support.	1. On the Login screen, the user taps "Forgot Password?" 2. System navigates to the Password Reset Request screen with an Email field. 3. User enters their registered email and taps "Send Reset Link." 4. If the email is not found, system shows a pop-up "No account found with this email." User remains on the same screen. 5. If found, system shows a confirmation pop-up "Reset link sent. Please check your email." and navigates the user back to the Login screen. 6. User opens the emailed link, which opens the New Password screen. 7. User enters and confirms a new password, then taps "Reset Password." 8. If the password is too weak, system shows inline error text and keeps the user on the same screen. 9. If valid, system shows a success pop-up "Password reset successfully. Please log in with your new password." and navigates the user to the Login screen.
All Roles (System-Wide)	Logging Out	As a logged-in user, I want to log out of my account, so that my session ends securely on shared or public devices.	1. From any authenticated screen, the user taps their profile icon, then taps "Log Out."

			2. System shows a confirmation pop-up “Are you sure you want to log out?” with Cancel / Confirm. 3. User taps “Cancel” → pop-up closes, user remains on the current screen. 4. User taps “Confirm” → system ends the session, shows “Logged out successfully,” and navigates the user to the Login / Role Landing screen.
All Roles (System-Wide)	Session Timeout Handling	As a logged-in user, I want to be notified if my session expires from inactivity, so that I understand why I was signed out and can log back in easily.	1. System monitors inactivity; after 30 minutes with no interaction, the session is flagged expired. 2. On the user’s next action, system intercepts it and shows a pop-up “Your session has expired. Please log in again.” 3. User taps “OK” → system navigates the user to the Login screen. 4. Once logged back in, system returns the user to the last screen they were viewing before the timeout, where possible.
All Roles (System-Wide)	Viewing In-App Notifications	As a logged-in user, I want a central place to see all my notifications, so that I don’t miss important status changes.	1. User taps the notification bell icon from any authenticated screen. 2. System opens a Notification Panel listing recent events, newest first, with unread items visually marked. 3. User taps a notification → system marks it read and navigates directly to the relevant screen. 4. User taps “Mark all as read” → system clears unread markers with a confirmation label; user remains on the panel. 5. User taps outside the panel or the close icon → panel closes; user remains on their current screen.

Use Case Diagrams and Fully Dressed Use Case

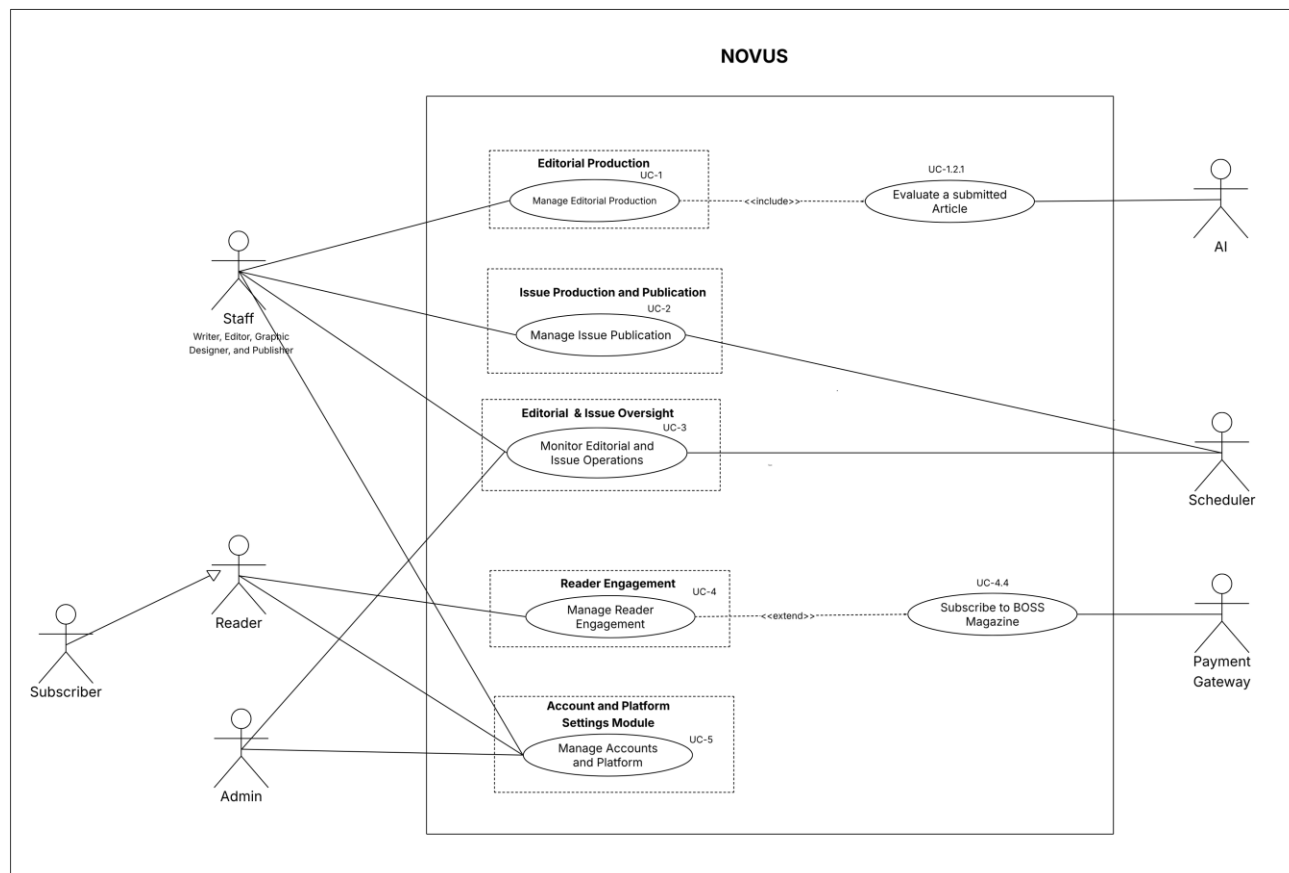


Figure 2 Use case diagram

A. Fully Dressed Use Cases

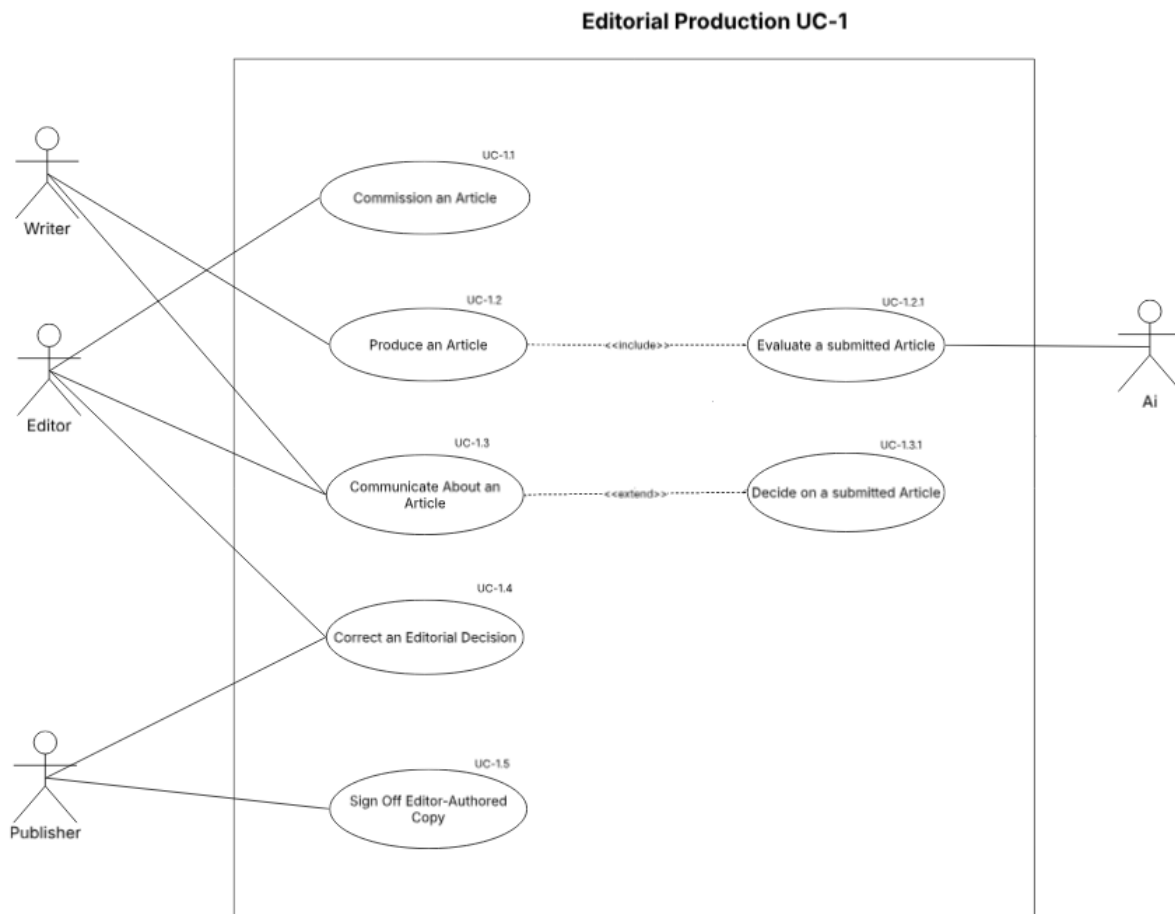


Figure 3 Editorial Production

Use Case Name: Commission an Article

Use Case ID:	UC-1.1
Author	Rei Khalil S. Galido
Purpose	Enable an Editor to place a piece of work with a Writer, giving them the topic, the angle and the date it is wanted.
Related Use Case	«include» UC-3.3 (Consult the Published Archive)
Priority	High
Preconditions	- The system is available - The Editor is authenticated in the Editor role - At least one active Writer exists in the directory

Postconditions	• An article record exists in the assigned state • The brief, the deadline and the commissioning Editor are recorded • The Writer is notified and the work appears on their dashboard • Any deadline is reflected in deadline tracking and on the calendar																																							
Actors	• Editor • Writer (recipient) • AI																																							
Flow of Actions	BASIC FLOW <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>Editor selects Assign article from the dashboard.</td><td>1.1 System displays the assignment form.</td></tr><tr><td></td><td></td><td>1.2 System lists the active Writers available to receive the work.</td></tr><tr><td>2</td><td>Editor enters the working title, category, editorial brief and deadline, and selects the Writer.</td><td></td></tr><tr><td>3</td><td>Editor confirms the commission.</td><td>3.1 System validates that a title and a Writer have been supplied.</td></tr><tr><td></td><td></td><td>3.2 System creates the article in the Assigned state with its brief, deadline and commissioning Editor.</td></tr><tr><td></td><td></td><td>3.3 System notifies the Writer.</td></tr><tr><td>4</td><td>Writer opens their dashboard.</td><td>4.1 System lists the commission under To write, with its brief and deadline.</td></tr></table> ALTERNATE FLOW A1 (No Deadline Set) Branches from step 2 of the basic flow. <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>Editor leaves the deadline empty and confirms the commission.</td><td>1.1 System accepts the commission, as a deadline is optional.</td></tr><tr><td></td><td></td><td>1.2 System creates the article in the Assigned state without a deadline.</td></tr><tr><td></td><td></td><td>1.3 System states that the work will not appear in deadline tracking or on the calendar until a deadline is added.</td></tr><tr><td></td><td></td><td>1.4 System notifies the Writer.</td></tr></table> Resumes at step 4 of the basic flow. ALTERNATE FLOW A2 (Assignment Transferred) Branches after step 3 of the basic flow, before the Writer has submitted.	Step	Actor Action	System Response	1	Editor selects Assign article from the dashboard.	1.1 System displays the assignment form.			1.2 System lists the active Writers available to receive the work.	2	Editor enters the working title, category, editorial brief and deadline, and selects the Writer.		3	Editor confirms the commission.	3.1 System validates that a title and a Writer have been supplied.			3.2 System creates the article in the Assigned state with its brief, deadline and commissioning Editor.			3.3 System notifies the Writer.	4	Writer opens their dashboard.	4.1 System lists the commission under To write, with its brief and deadline.	Step	Actor Action	System Response	1	Editor leaves the deadline empty and confirms the commission.	1.1 System accepts the commission, as a deadline is optional.			1.2 System creates the article in the Assigned state without a deadline.			1.3 System states that the work will not appear in deadline tracking or on the calendar until a deadline is added.			1.4 System notifies the Writer.
Step	Actor Action	System Response																																						
1	Editor selects Assign article from the dashboard.	1.1 System displays the assignment form.																																						
		1.2 System lists the active Writers available to receive the work.																																						
2	Editor enters the working title, category, editorial brief and deadline, and selects the Writer.																																							
3	Editor confirms the commission.	3.1 System validates that a title and a Writer have been supplied.																																						
		3.2 System creates the article in the Assigned state with its brief, deadline and commissioning Editor.																																						
		3.3 System notifies the Writer.																																						
4	Writer opens their dashboard.	4.1 System lists the commission under To write, with its brief and deadline.																																						
Step	Actor Action	System Response																																						
1	Editor leaves the deadline empty and confirms the commission.	1.1 System accepts the commission, as a deadline is optional.																																						
		1.2 System creates the article in the Assigned state without a deadline.																																						
		1.3 System states that the work will not appear in deadline tracking or on the calendar until a deadline is added.																																						
		1.4 System notifies the Writer.																																						

Step	Actor Action	System Response
1	Editor opens an unstarted assignment and selects a different Writer.	1.1 System validates that the work has not been submitted.
2	Editor confirms the transfer.	2.1 System transfers the assignment, keeping the brief and the deadline.
		2.2 System notifies both the previous and the new Writer.
3	New Writer opens their dashboard.	3.1 System lists the commission under To write, with its brief and deadline.

The use case ends.

ALTERNATE FLOW

A-VALID (Incomplete or Invalid Input)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	Editor confirms the commission with a required item missing or malformed.	1.1 System validates every required item before saving anything.
		1.2 System marks each problem beside the field it concerns, with guidance.
		1.3 System saves nothing, and the workflow state is unchanged.
2	Editor corrects the items and submits again.	2.1 System validates the input again.

Resumes at step 3.2 of the basic flow.

EXCEPTION FLOW

E1 (Work Already Submitted)

Branches from step 1 of alternate flow A2.

Step	Actor Action	System Response
1	Editor attempts to transfer or cancel an assignment that has already been submitted.	1.1 System detects that the copy has entered review.
		1.2 System refuses the change and leaves the assignment as it was.
		1.3 System explains that submitted copy belongs to its author and must be decided on under UC-1.3.1 rather than reassigned.

The use case ends.

EXCEPTION FLOW

E-SAVE (Persistence Failure)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	Editor confirms the commission.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
2	Editor retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.
Resumes at step 3 of the basic flow.		
EXCEPTION FLOW		
E-AUTH (Authorization or Session Failure)		
Branches from step 3 of the basic flow.		
Step	Actor Action	System Response
1	Editor confirms the commission after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
		1.2 System blocks the change without partly saving it.
		1.3 System redirects to sign-in where the session has expired, or states that access is denied.
2	Editor signs in again.	2.1 System restores the session and returns to the page.
Where the session had expired, resumes at step 3 of the basic flow; where the role or ownership was missing, the use case ends.		

Use Case Name: Produce an Article

Use Case ID:	UC-1.2
Author	Roman Rico Albania
Purpose	Enable a Writer to take a commission from an empty page to a submission the Editor can act on, including any revision the Editor asks for.
Related Use Case	«include» UC-1.2.1 (Evaluate a Submitted Article)
Priority	High
Preconditions	- The system is available - The Writer is authenticated in the Writer role - The article is commissioned to this Writer under UC-1.1, or was started by them

Postconditions	• The copy, standfirst and photography are stored • On submission, an immutable version snapshot is recorded • On submission, an assessment is attached and the article routed by it • The article is no longer directly editable once submitted																																							
Actors	• Writer • AI																																							
Flow of Actions	BASIC FLOW <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>Writer opens the commissioned work.</td><td>1.1 System displays the composer with the brief and the deadline in view.</td></tr><tr><td>2</td><td>Writer enters the headline, standfirst and body copy.</td><td>2.1 System saves the work at intervals and reports the save state.</td></tr><tr><td></td><td></td><td>2.2 System reports the word count and reading time as they change.</td></tr><tr><td>3</td><td>Writer attaches the hero image and any photography within the body.</td><td>3.1 System stores each image against the article.</td></tr><tr><td>4</td><td>Writer submits the article for review.</td><td>4.1 System checks that the draft is ready to be assessed.</td></tr><tr><td></td><td></td><td>4.2 System records an immutable version snapshot of the submitted copy.</td></tr><tr><td></td><td></td><td>4.3 System invokes UC-1.2.1 to assess the article and routes it by the verdict.</td></tr></table> ALTERNATE FLOW A1 (Article Returned for Revision) <i>Branches after step 4 of the basic flow, when the Editor or pre-screening returns the article.</i> <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>Writer opens the returned article from their dashboard.</td><td>1.1 System displays the revision notes beside the copy, so each point can be addressed where it applies.</td></tr><tr><td></td><td></td><td>1.2 System keeps every earlier version, so the resubmission will be recorded as a further version.</td></tr><tr><td>2</td><td>Writer revises the copy.</td><td>2.1 System saves the work at intervals and reports the save state.</td></tr></table> <i>Resumes at step 4 of the basic flow.</i> ALTERNATE FLOW A2 (No Photography Supplied) <i>Branches from step 3 of the basic flow.</i> <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr></table>	Step	Actor Action	System Response	1	Writer opens the commissioned work.	1.1 System displays the composer with the brief and the deadline in view.	2	Writer enters the headline, standfirst and body copy.	2.1 System saves the work at intervals and reports the save state.			2.2 System reports the word count and reading time as they change.	3	Writer attaches the hero image and any photography within the body.	3.1 System stores each image against the article.	4	Writer submits the article for review.	4.1 System checks that the draft is ready to be assessed.			4.2 System records an immutable version snapshot of the submitted copy.			4.3 System invokes UC-1.2.1 to assess the article and routes it by the verdict.	Step	Actor Action	System Response	1	Writer opens the returned article from their dashboard.	1.1 System displays the revision notes beside the copy, so each point can be addressed where it applies.			1.2 System keeps every earlier version, so the resubmission will be recorded as a further version.	2	Writer revises the copy.	2.1 System saves the work at intervals and reports the save state.	Step	Actor Action	System Response
Step	Actor Action	System Response																																						
1	Writer opens the commissioned work.	1.1 System displays the composer with the brief and the deadline in view.																																						
2	Writer enters the headline, standfirst and body copy.	2.1 System saves the work at intervals and reports the save state.																																						
		2.2 System reports the word count and reading time as they change.																																						
3	Writer attaches the hero image and any photography within the body.	3.1 System stores each image against the article.																																						
4	Writer submits the article for review.	4.1 System checks that the draft is ready to be assessed.																																						
		4.2 System records an immutable version snapshot of the submitted copy.																																						
		4.3 System invokes UC-1.2.1 to assess the article and routes it by the verdict.																																						
Step	Actor Action	System Response																																						
1	Writer opens the returned article from their dashboard.	1.1 System displays the revision notes beside the copy, so each point can be addressed where it applies.																																						
		1.2 System keeps every earlier version, so the resubmission will be recorded as a further version.																																						
2	Writer revises the copy.	2.1 System saves the work at intervals and reports the save state.																																						
Step	Actor Action	System Response																																						

1	Writer skips attaching photography and submits the article.	1.1 System accepts the submission, as photography is optional.
		1.2 System states that the article will run as text only unless the Editor supplies photography during review.

Resumes at step 4.1 of the basic flow.

ALTERNATE FLOW

A3 (Session Lapses While Writing)

Branches from step 2 of the basic flow.

Step	Actor Action	System Response
1	Writer continues writing after the session has expired.	1.1 System exchanges the renewal credential for a fresh session.
		1.2 System retries the pending save.
		1.3 System preserves the work, so the Writer continues uninterrupted.
		1.4 Where renewal fails, System warns the Writer before any work is lost.

Resumes at step 2 of the basic flow.

ALTERNATE FLOW

A4 (Article Written by an Editor)

Branches from step 4.3 of the basic flow.

Step	Actor Action	System Response
1	Editor, as the author, submits their own article.	1.1 System obtains the assessment under UC-1.2.1 as normal.
		1.2 On a passing verdict, System routes the article to the Publisher rather than to the Editor's own review queue.
2	Publisher opens the sign-off queue.	2.1 System lists the article for decision under UC-1.5.

The use case ends.

ALTERNATE FLOW

A5 (Quick Fixes Applied)

Branches from step 1 of alternate flow A1, when the article was returned in the Revise band.

Step	Actor Action	System Response
1	Writer opens the returned article.	1.1 System shows each quick fix as the original passage struck through beside its replacement, with a one-line reason.
		1.2 System shows how many fixes have been resolved.

2	Writer selects to apply a fix.	2.1 System saves any unsaved edits and locks the editor for the request.
		2.2 System applies the fix to the saved copy, inside text only, and records the decision.
		2.3 System reloads the corrected copy into the editor and updates the count; the score is unchanged until the article is assessed again.
		2.4 Where the Writer has since changed that passage, System marks the fix as no longer applicable rather than forcing it in.
3	Writer dismisses any fix they do not want.	3.1 System records the dismissal and leaves the copy unchanged.

Resumes at step 4 of the basic flow.

EXCEPTION FLOW

E1 (Draft Not Ready for Assessment)

Branches from step 4.1 of the basic flow.

Step	Actor Action	System Response
1	Writer submits a draft that is unfinished in some mechanical way.	1.1 System checks the word count against the configured minimum, unresolved placeholder markers, a missing headline or standfirst, and whether the copy has changed since its last assessment.
		1.2 System refuses the submission and lists every reason together.
		1.3 System records no version and requests no assessment.
2	Writer corrects the draft.	2.1 System saves the changes.

Resumes at step 4 of the basic flow.

EXCEPTION FLOW

E2 (Assessment Rate Limited)

Branches from step 4.1 of the basic flow.

Step	Actor Action	System Response
1	Writer resubmits within the cooldown after a previous assessment.	1.1 System checks the cooldown, which is waived once every quick fix from the previous assessment has been resolved.
		1.2 System refuses the submission and states how long remains.

		1.3 Where the daily limit has been reached, System says so and notes that an Editor may review the article without an assessment.
		1.4 System requests no assessment, and the article stays with the Writer.
2	Writer waits for the cooldown to pass.	2.1 System accepts a submission once the cooldown has passed.

Resumes at step 4 of the basic flow.

EXCEPTION FLOW

E-SAVE (Persistence Failure)

Branches from step 4 of the basic flow.

Step	Actor Action	System Response
1	Writer submits the article for review.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
2	Writer retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.

Resumes at step 4 of the basic flow.

EXCEPTION FLOW

E-AUTH (Authorization or Session Failure)

Branches from step 4 of the basic flow.

Step	Actor Action	System Response
1	Writer submits the article for review after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
		1.2 System blocks the change without partly saving it.
		1.3 System redirects to sign-in where the session has expired, or states that access is denied.
2	Writer signs in again.	2.1 System restores the session and returns to the page.

Where the session had expired, resumes at step 4 of the basic flow; where the role or ownership was missing, the use case ends.

Use Case Name: Evaluate a Submitted Article

Use Case ID:	UC-1.2.1																								
Author	Roman Rico Albania																								
Purpose	Assess a submitted article for grammar, readability and structure, and route it according to whether it meets the publication's minimum standard.																								
Related Use Case	Included by UC-1.2 (Produce an Article)																								
Priority	High																								
Preconditions	• The system is available • An article has been submitted for review under UC-1.2 • The evaluation service is reachable and holds a valid credential																								
Postconditions	1. An assessment is attached to the article with its scores and summary 2. Suggestions are recorded in a form the revision composer can consume 3. The article is routed by the verdict against the passing mark 4. The producing model and the usage are recorded against the assessment 5. Where the verdict is revise, quick fixes are recorded against the assessment; otherwise none are kept																								
Actors	• AI • Writer (recipient) • Editor (recipient)																								
Flow of Actions	BASIC FLOW <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>Writer submits an article under UC-1.2.</td><td>1.1 System assembles the headline, standfirst and body into an assessment request.</td></tr><tr><td></td><td></td><td>1.2 System sends the request to the AI Evaluation Service.</td></tr><tr><td>2</td><td>AI Evaluation Service returns scores, a summary and suggestions.</td><td>2.1 System validates the response structure before accepting it.</td></tr><tr><td></td><td></td><td>2.2 System stores the assessment with the producing model and the usage recorded.</td></tr><tr><td></td><td></td><td>2.3 System compares the overall score against the passing mark and finds it met.</td></tr><tr><td></td><td></td><td>2.4 System places the article in the Editor's review queue and notifies the Editor.</td></tr><tr><td>3</td><td>Editor opens the review queue.</td><td>3.1 System shows the article with its assessment attached.</td></tr></table> ALTERNATE FLOW <i>A1 (Revise: Between the Rewrite Mark and the Passing Mark)</i>	Step	Actor Action	System Response	1	Writer submits an article under UC-1.2.	1.1 System assembles the headline, standfirst and body into an assessment request.			1.2 System sends the request to the AI Evaluation Service.	2	AI Evaluation Service returns scores, a summary and suggestions.	2.1 System validates the response structure before accepting it.			2.2 System stores the assessment with the producing model and the usage recorded.			2.3 System compares the overall score against the passing mark and finds it met.			2.4 System places the article in the Editor's review queue and notifies the Editor.	3	Editor opens the review queue.	3.1 System shows the article with its assessment attached.
Step	Actor Action	System Response																							
1	Writer submits an article under UC-1.2.	1.1 System assembles the headline, standfirst and body into an assessment request.																							
		1.2 System sends the request to the AI Evaluation Service.																							
2	AI Evaluation Service returns scores, a summary and suggestions.	2.1 System validates the response structure before accepting it.																							
		2.2 System stores the assessment with the producing model and the usage recorded.																							
		2.3 System compares the overall score against the passing mark and finds it met.																							
		2.4 System places the article in the Editor's review queue and notifies the Editor.																							
3	Editor opens the review queue.	3.1 System shows the article with its assessment attached.																							

Branches from step 2.3 of the basic flow.

Step	Actor Action	System Response
1	AI Evaluation Service returns an overall score below the passing mark but at or above the rewrite mark.	1.1 System records each suggestion as a revision note with no authoring Editor.
		1.2 System keeps the quick fixes whose passages appear exactly once in the copy.
		1.3 System returns the article to its Writer and notifies them, stating how many quick fixes were suggested.
2	Writer opens the returned article.	2.1 System shows the revision notes and the quick fixes beside the copy.

The Writer revises under UC-1.2, alternate flows A1 and A5. The use case ends.

ALTERNATE FLOW

A2 (Rewrite: Below the Rewrite Mark)

Branches from step 2.3 of the basic flow.

Step	Actor Action	System Response
1	AI Evaluation Service returns an overall score below the rewrite mark.	1.1 System records the suggestions as revision notes and keeps no quick fixes.
		1.2 System returns the article to its Writer and notifies them that it needs rework rather than line edits.
2	Writer opens the returned article.	2.1 System shows the revision notes beside the copy.

The Writer reworks the draft under UC-1.2. The use case ends.

ALTERNATE FLOW

A3 (Malformed Assessment)

Branches from step 2.1 of the basic flow.

Step	Actor Action	System Response
1	AI Evaluation Service returns a response that does not match the expected structure.	1.1 System attempts to read the scored fields, removing any formatting around them.
		1.2 Where the fields cannot be read, System treats the assessment as not obtained and records no score.
		1.3 System logs the response for diagnosis.
		1.4 System routes the article to the Editor's review queue, or to the

		Publisher's sign-off queue where an Editor wrote it.
2	Editor opens the review queue.	2.1 System shows the article with the absence of an assessment stated.

The use case ends.

ALTERNATE FLOW

A4 (Editor-Authored Article Passes)

Branches from step 2.4 of the basic flow.

Step	Actor Action	System Response
1	AI Evaluation Service returns a passing score for an article written by an Editor.	1.1 System routes the article to the Publisher's sign-off queue rather than to the Editor's review queue.
		1.2 System notifies the Publisher that copy awaits sign-off.
2	Publisher opens the sign-off queue.	2.1 System lists the article with its assessment, for decision under UC-1.5.

The use case ends.

EXCEPTION FLOW

E1 (Service Unavailable or Credential Refused)

Branches from step 1.2 of the basic flow.

Step	Actor Action	System Response
1	Writer submits an article while the AI Evaluation Service is unavailable, refuses the credential, or has no credential configured.	1.1 System does not retry indefinitely and does not block the Writer.
		1.2 System records no score, rather than a score of zero that would fail every draft.
		1.3 System routes the article to the Editor's review queue, or to the Publisher's sign-off queue where an Editor wrote it.
		1.4 System notifies the reviewer that the article awaits review without an assessment.
2	Editor opens the review queue.	2.1 System shows the article with the absence of an assessment stated, so it is never mistaken for a pass.
3	Administrator opens platform settings.	3.1 System shows the evaluation service as unavailable or unconfigured, under UC-5.4.

The use case ends.

Use Case Name: Communicate About an Article

Use Case ID:	UC-1.3																												
Author	Joshmar Clavero																												
Purpose	Enable staff to hold the conversation an article needs in the place the article lives, rather than across separate messaging applications.																												
Related Use Case	«extend» UC-1.3.1 (Decide on a Submitted Article)																												
Priority	Medium																												
Preconditions	• The system is available • The actor is authenticated and the article falls within their scope																												
Postconditions	• The message is recorded against the article • The other participants are notified, subject to their preferences • The article's state is unchanged																												
Actors	• Writer • Editor • Publisher • Graphic Designer																												
Flow of Actions	BASIC FLOW <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Writer or Editor opens the correspondence on an article.</td><td>1.1 System displays the messages in order with their authors and times.</td></tr> <tr> <td>2</td><td>Writer or Editor writes a message and sends it.</td><td>2.1 System records the message against the article.</td></tr> <tr> <td></td><td></td><td>2.2 System notifies the other party, subject to their preferences.</td></tr> <tr> <td>3</td><td>The other party opens the article.</td><td>3.1 System shows the new message in the thread.</td></tr> </table> ALTERNATE FLOW <i>A1 (Recipient Has Muted This Notice)</i> <i>Branches from step 2.2 of the basic flow.</i> <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Writer or Editor sends a message to a participant who has muted correspondence notices.</td><td>1.1 System records the message as normal.</td></tr> <tr> <td></td><td></td><td>1.2 System does not notify that participant.</td></tr> <tr> <td>2</td><td>The participant later opens the article.</td><td>2.1 System shows the message in the thread, so nothing is lost by muting.</td></tr> </table>		Step	Actor Action	System Response	1	Writer or Editor opens the correspondence on an article.	1.1 System displays the messages in order with their authors and times.	2	Writer or Editor writes a message and sends it.	2.1 System records the message against the article.			2.2 System notifies the other party, subject to their preferences.	3	The other party opens the article.	3.1 System shows the new message in the thread.	Step	Actor Action	System Response	1	Writer or Editor sends a message to a participant who has muted correspondence notices.	1.1 System records the message as normal.			1.2 System does not notify that participant.	2	The participant later opens the article.	2.1 System shows the message in the thread, so nothing is lost by muting.
Step	Actor Action	System Response																											
1	Writer or Editor opens the correspondence on an article.	1.1 System displays the messages in order with their authors and times.																											
2	Writer or Editor writes a message and sends it.	2.1 System records the message against the article.																											
		2.2 System notifies the other party, subject to their preferences.																											
3	The other party opens the article.	3.1 System shows the new message in the thread.																											
Step	Actor Action	System Response																											
1	Writer or Editor sends a message to a participant who has muted correspondence notices.	1.1 System records the message as normal.																											
		1.2 System does not notify that participant.																											
2	The participant later opens the article.	2.1 System shows the message in the thread, so nothing is lost by muting.																											

	<i>The use case ends.</i>	
	EXCEPTION FLOW	
	E-SAVE (Persistence Failure)	
	<i>Branches from step 2 of the basic flow.</i>	
	Step	Actor Action
	1	Writer or Editor sends a message.
		1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
	2	Writer or Editor retries once the service has recovered.
		2.1 System keeps the input on screen, so nothing needs to be entered again.
	<i>Resumes at step 2 of the basic flow.</i>	
	EXCEPTION FLOW	
	E-AUTH (Authorization or Session Failure)	
	<i>Branches from step 2 of the basic flow.</i>	
	Step	Actor Action
	1	Writer or Editor sends a message after the session has expired, or without the role or ownership the action requires.
		1.1 System checks the session, and the role or ownership the action requires, before changing anything.
		1.2 System blocks the change without partly saving it.
		1.3 System redirects to sign-in where the session has expired, or states that access is denied.
	2	Writer or Editor signs in again.
		2.1 System restores the session and returns to the page.
	<i>Where the session had expired, resumes at step 2 of the basic flow; where the role or ownership was missing, the use case ends.</i>	

Use Case Name: Decide on a Submitted Article

Use Case ID:	UC-1.3.1
Author	Joshmar Clavero
Purpose	Enable an Editor to reach and record a decision on submitted copy: to accept it, to return it for revision, to accept it against the assessment, or to remove it from the pipeline.
Related Use Case	Extended by UC-1.3 (Communicate About an Article)

Priority	High																																	
Preconditions	• The system is available • The Editor is authenticated in the Editor role • The article has been submitted and assessed under UC-1.2.1, or is recorded as having no assessment																																	
Postconditions	• The article is approved, returned for revision, or withdrawn • The deciding Editor and the time of the decision are recorded • Where the assessment was overridden, the justification is recorded against it • The Writer is notified of the outcome • An approved article becomes eligible for scheduling into an issue																																	
Actors	• Editor • Writer (notified)																																	
Flow of Actions	BASIC FLOW <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>Editor opens an article from the review queue.</td><td>1.1 System displays the copy with its assessment and version history.</td></tr><tr><td></td><td></td><td>1.2 System displays the status timeline and the correspondence beside it.</td></tr><tr><td></td><td></td><td>1.3 System lists any AI-suggested fixes the Writer accepted, showing the original and the replacement.</td></tr><tr><td>2</td><td>Editor reads the copy and the assessment.</td><td></td></tr><tr><td>3</td><td>Editor replaces or captions the photography where the article needs it.</td><td>3.1 System stores the change and records the uploading user.</td></tr><tr><td>4</td><td>Editor selects to approve the article.</td><td>4.1 System validates that the article is under review.</td></tr><tr><td></td><td></td><td>4.2 System validates that no assessment stands against it unoverridden.</td></tr><tr><td></td><td></td><td>4.3 System records the approval and the deciding Editor.</td></tr><tr><td></td><td></td><td>4.4 System notifies the Writer and makes the article eligible for scheduling.</td></tr><tr><td>5</td><td>Writer opens their dashboard.</td><td>5.1 System shows the article as approved.</td></tr></table> ALTERNATE FLOW A1 (Revision Requested) Branches from step 4 of the basic flow.	Step	Actor Action	System Response	1	Editor opens an article from the review queue.	1.1 System displays the copy with its assessment and version history.			1.2 System displays the status timeline and the correspondence beside it.			1.3 System lists any AI-suggested fixes the Writer accepted, showing the original and the replacement.	2	Editor reads the copy and the assessment.		3	Editor replaces or captions the photography where the article needs it.	3.1 System stores the change and records the uploading user.	4	Editor selects to approve the article.	4.1 System validates that the article is under review.			4.2 System validates that no assessment stands against it unoverridden.			4.3 System records the approval and the deciding Editor.			4.4 System notifies the Writer and makes the article eligible for scheduling.	5	Writer opens their dashboard.	5.1 System shows the article as approved.
Step	Actor Action	System Response																																
1	Editor opens an article from the review queue.	1.1 System displays the copy with its assessment and version history.																																
		1.2 System displays the status timeline and the correspondence beside it.																																
		1.3 System lists any AI-suggested fixes the Writer accepted, showing the original and the replacement.																																
2	Editor reads the copy and the assessment.																																	
3	Editor replaces or captions the photography where the article needs it.	3.1 System stores the change and records the uploading user.																																
4	Editor selects to approve the article.	4.1 System validates that the article is under review.																																
		4.2 System validates that no assessment stands against it unoverridden.																																
		4.3 System records the approval and the deciding Editor.																																
		4.4 System notifies the Writer and makes the article eligible for scheduling.																																
5	Writer opens their dashboard.	5.1 System shows the article as approved.																																

Step	Actor Action	System Response
1	Editor selects to return the article.	1.1 System presents the note composer, prefilled from the assessment suggestions where these exist.
2	Editor amends, removes or adds notes, each with a section, type and priority, and confirms.	2.1 System validates that at least one note carries an instruction.
		2.2 System records the notes, returns the article to its Writer and notifies them.
3	Writer opens their dashboard.	3.1 System shows the article returned, with the notes attached.

The Writer revises under UC-1.2. The use case ends.

ALTERNATE FLOW

A2 (Assessment Overridden)

Branches from step 4 of the basic flow.

Step	Actor Action	System Response
1	Editor opens an article the assessment returned and judges it publishable.	1.1 System shows the verdict and offers an override.
2	Editor selects to override and enters an editorial justification.	2.1 System validates that the justification is of sufficient length.
		2.2 System marks the assessment overridden and records the justification, the Editor and the time.
		2.3 System approves the article and shows the justification beside the assessment for anyone who reads it later.

Resumes at step 5 of the basic flow.

ALTERNATE FLOW

A3 (Article Withdrawn)

Branches from step 4 of the basic flow.

Step	Actor Action	System Response
1	Editor selects to withdraw the article.	1.1 System asks for a reason.
2	Editor enters the reason and confirms.	2.1 System removes the article from any issue and corrects that issue's readiness.
		2.2 System records the withdrawal and its reason.
		2.3 Where the issue had been closed, System notifies the Publisher, since the

		layout was planned around those contents.
--	--	---

The use case ends.

ALTERNATE FLOW

A4 (No Assessment Obtained)

Branches from step 1 of the basic flow.

Step	Actor Action	System Response
1	Editor opens an article that carries no assessment.	1.1 System states the absence plainly on the review screen.
2	Editor reads the copy and decides on editorial judgement alone.	2.1 System requires no override, because no verdict stands against the article.

Resumes at step 4 of the basic flow.

EXCEPTION FLOW

E1 (Article Not Under Review)

Branches from step 4.1 of the basic flow.

Step	Actor Action	System Response
1	Editor attempts to decide on an article that is not under review.	1.1 System checks the article's state before acting.
		1.2 System refuses the decision and names the article's actual state.
		1.3 System changes nothing, so the assessment cannot be bypassed by acting on a draft.

The use case ends.

EXCEPTION FLOW

E2 (Justification Absent or Insufficient)

Branches from step 2 of alternate flow A2.

Step	Actor Action	System Response
1	Editor confirms the override without an adequate justification.	1.1 System checks the justification before recording anything.
		1.2 System refuses the override and leaves the verdict unchanged.
		1.3 System explains that a written justification is required for audit.
2	Editor writes a justification and confirms again.	2.1 System validates the justification.

Resumes at step 2 of alternate flow A2.

EXCEPTION FLOW

	E-SAVE (Persistence Failure)		
	<i>Branches from step 4 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Editor approves the article.	1.1 System validates the request.
			1.2 System attempts to save the change, and the save fails or times out.
			1.3 System keeps the last confirmed state and does not report success.
			1.4 System records the failure and displays a message inviting a retry.
	2	Editor retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.
	<i>Resumes at step 4 of the basic flow.</i>		
	EXCEPTION FLOW		
	E-AUTH (Authorization or Session Failure)		
	<i>Branches from step 4 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Editor approves the article after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
			1.2 System blocks the change without partly saving it.
			1.3 System redirects to sign-in where the session has expired, or states that access is denied.
	2	Editor signs in again.	2.1 System restores the session and returns to the page.
	<i>Where the session had expired, resumes at step 4 of the basic flow; where the role or ownership was missing, the use case ends.</i>		

Use Case Name: Correct an Editorial Decision

Use Case ID:	UC-1.4
Author	Joshua Concepcion
Purpose	Enable an Editor or Publisher to return an approved article to review, so that an early or mistaken approval can be corrected without destroying the work.
Related Use Case	None
Priority	Medium
Preconditions	The system is available

	• The actor is authenticated in the Editor or Publisher role • The article was approved under UC-1.3.1 and has not been published • Where the article is scheduled into an issue, the actor holds the Publisher role																																				
Postconditions	• The article returns to the review queue • The reason is recorded as a revision note against the article • Any issue the article belonged to has its readiness corrected • The Writer is notified																																				
Actors	• Editor • Publisher • Writer (notified)																																				
Flow of Actions	BASIC FLOW <table><thead><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr></thead><tbody><tr><td>1</td><td>Editor or Publisher opens an approved article and selects to return it to review.</td><td>1.1 System presents a reason field.</td></tr><tr><td></td><td></td><td>1.2 System states that the article will leave any issue it belongs to.</td></tr><tr><td>2</td><td>Editor or Publisher enters the reason and confirms.</td><td>2.1 System validates the actor's authority against the article's scheduling.</td></tr><tr><td></td><td></td><td>2.2 System removes the article from any issue and corrects that issue's readiness.</td></tr><tr><td></td><td></td><td>2.3 System returns the article to review and records the reason as a revision note.</td></tr><tr><td></td><td></td><td>2.4 System notifies the Writer, and the Publisher where an issue was affected.</td></tr><tr><td>3</td><td>Writer opens their dashboard.</td><td>3.1 System shows the article back with the editor, with the reason attached as a revision note.</td></tr></tbody></table> ALTERNATE FLOW A1 (Editor Acts on a Scheduled Article) <i>Branches from step 2.1 of the basic flow.</i><table><thead><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr></thead><tbody><tr><td>1</td><td>Editor confirms returning an article that is already placed in an issue.</td><td>1.1 System detects the scheduling.</td></tr><tr><td></td><td></td><td>1.2 System refuses and leaves the issue untouched.</td></tr><tr><td></td><td></td><td>1.3 System explains that the Publisher is planning around those contents and must authorise the removal.</td></tr></tbody></table>	Step	Actor Action	System Response	1	Editor or Publisher opens an approved article and selects to return it to review.	1.1 System presents a reason field.			1.2 System states that the article will leave any issue it belongs to.	2	Editor or Publisher enters the reason and confirms.	2.1 System validates the actor's authority against the article's scheduling.			2.2 System removes the article from any issue and corrects that issue's readiness.			2.3 System returns the article to review and records the reason as a revision note.			2.4 System notifies the Writer, and the Publisher where an issue was affected.	3	Writer opens their dashboard.	3.1 System shows the article back with the editor, with the reason attached as a revision note.	Step	Actor Action	System Response	1	Editor confirms returning an article that is already placed in an issue.	1.1 System detects the scheduling.			1.2 System refuses and leaves the issue untouched.			1.3 System explains that the Publisher is planning around those contents and must authorise the removal.
Step	Actor Action	System Response																																			
1	Editor or Publisher opens an approved article and selects to return it to review.	1.1 System presents a reason field.																																			
		1.2 System states that the article will leave any issue it belongs to.																																			
2	Editor or Publisher enters the reason and confirms.	2.1 System validates the actor's authority against the article's scheduling.																																			
		2.2 System removes the article from any issue and corrects that issue's readiness.																																			
		2.3 System returns the article to review and records the reason as a revision note.																																			
		2.4 System notifies the Writer, and the Publisher where an issue was affected.																																			
3	Writer opens their dashboard.	3.1 System shows the article back with the editor, with the reason attached as a revision note.																																			
Step	Actor Action	System Response																																			
1	Editor confirms returning an article that is already placed in an issue.	1.1 System detects the scheduling.																																			
		1.2 System refuses and leaves the issue untouched.																																			
		1.3 System explains that the Publisher is planning around those contents and must authorise the removal.																																			

The Editor raises the matter with the Publisher, who may perform the correction under the basic flow. The use case ends.

EXCEPTION FLOW

E1 (Reason Absent)

Branches from step 2 of the basic flow.

Step	Actor Action	System Response
1	Editor or Publisher confirms without an adequate reason.	1.1 System checks the reason before committing anything.
		1.2 System refuses the correction and leaves the approval standing.
2	Editor or Publisher enters a reason and confirms again.	2.1 System validates the reason.

Resumes at step 2.1 of the basic flow.

EXCEPTION FLOW

E-SAVE (Persistence Failure)

Branches from step 2 of the basic flow.

Step	Actor Action	System Response
1	Editor or Publisher confirms the return to review.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
2	Editor or Publisher retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.

Resumes at step 2 of the basic flow.

EXCEPTION FLOW

E-AUTH (Authorization or Session Failure)

Branches from step 2 of the basic flow.

Step	Actor Action	System Response
1	Editor or Publisher confirms the return to review after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
		1.2 System blocks the change without partly saving it.
		1.3 System redirects to sign-in where the session has expired, or states that access is denied.

	2	Editor or Publisher signs in again.	2.1 System restores the session and returns to the page.
	<i>Where the session had expired, resumes at step 2 of the basic flow; where the role or ownership was missing, the use case ends.</i>		

Use Case Name: Sign Off Editor-Authored Copy

Use Case ID:	UC-1.5																					
Author	Joshua Concepcion																					
Purpose	Enable a Publisher to provide the second reading that editorial practice requires on articles written by an Editor.																					
Related Use Case	None																					
Priority	Medium																					
Preconditions	• The system is available • The Publisher is authenticated in the Publisher role • The article was written by an Editor and has passed assessment																					
Postconditions	• The article is approved and the signing Publisher recorded as its editor of record • The author is notified • The article becomes eligible for scheduling into an issue																					
Actors	• Publisher • Editor (as author)																					
Flow of Actions	BASIC FLOW <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>Publisher opens the sign-off queue.</td><td>1.1 System lists the articles awaiting sign-off with their assessment scores.</td></tr><tr><td>2</td><td>Publisher reads an article in the preview panel.</td><td></td></tr><tr><td>3</td><td>Publisher confirms the sign-off.</td><td>3.1 System validates that the article awaits sign-off.</td></tr><tr><td></td><td></td><td>3.2 System approves the article and records the Publisher as its editor of record.</td></tr><tr><td></td><td></td><td>3.3 System notifies the author and makes the article eligible for scheduling.</td></tr><tr><td>4</td><td>Editor opens their own article.</td><td>4.1 System shows it approved, with the Publisher recorded as editor of record.</td></tr></table> ALTERNATE FLOW A1 (Returned to the Author) Branches from step 3 of the basic flow.	Step	Actor Action	System Response	1	Publisher opens the sign-off queue.	1.1 System lists the articles awaiting sign-off with their assessment scores.	2	Publisher reads an article in the preview panel.		3	Publisher confirms the sign-off.	3.1 System validates that the article awaits sign-off.			3.2 System approves the article and records the Publisher as its editor of record.			3.3 System notifies the author and makes the article eligible for scheduling.	4	Editor opens their own article.	4.1 System shows it approved, with the Publisher recorded as editor of record.
Step	Actor Action	System Response																				
1	Publisher opens the sign-off queue.	1.1 System lists the articles awaiting sign-off with their assessment scores.																				
2	Publisher reads an article in the preview panel.																					
3	Publisher confirms the sign-off.	3.1 System validates that the article awaits sign-off.																				
		3.2 System approves the article and records the Publisher as its editor of record.																				
		3.3 System notifies the author and makes the article eligible for scheduling.																				
4	Editor opens their own article.	4.1 System shows it approved, with the Publisher recorded as editor of record.																				

Step	Actor Action	System Response
1	Publisher selects to return the article to its author.	1.1 System presents the note composer.
2	Publisher enters notes and confirms.	2.1 System validates that at least one note carries an instruction.
		2.2 System records the notes, returns the article to its author and notifies them.
3	Editor, as the author, opens the article.	3.1 System shows it returned, with the Publisher's notes attached.

The author revises and resubmits under UC-1.2. The use case ends.

EXCEPTION FLOW

E-SAVE (Persistence Failure)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	Publisher confirms the sign-off.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
2	Publisher retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.

Resumes at step 3 of the basic flow.

EXCEPTION FLOW

E-AUTH (Authorization or Session Failure)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	Publisher confirms the sign-off after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
		1.2 System blocks the change without partly saving it.
		1.3 System redirects to sign-in where the session has expired, or states that access is denied.
2	Publisher signs in again.	2.1 System restores the session and returns to the page.

	<i>Where the session had expired, resumes at step 3 of the basic flow; where the role or ownership was missing, the use case ends.</i>
--	--

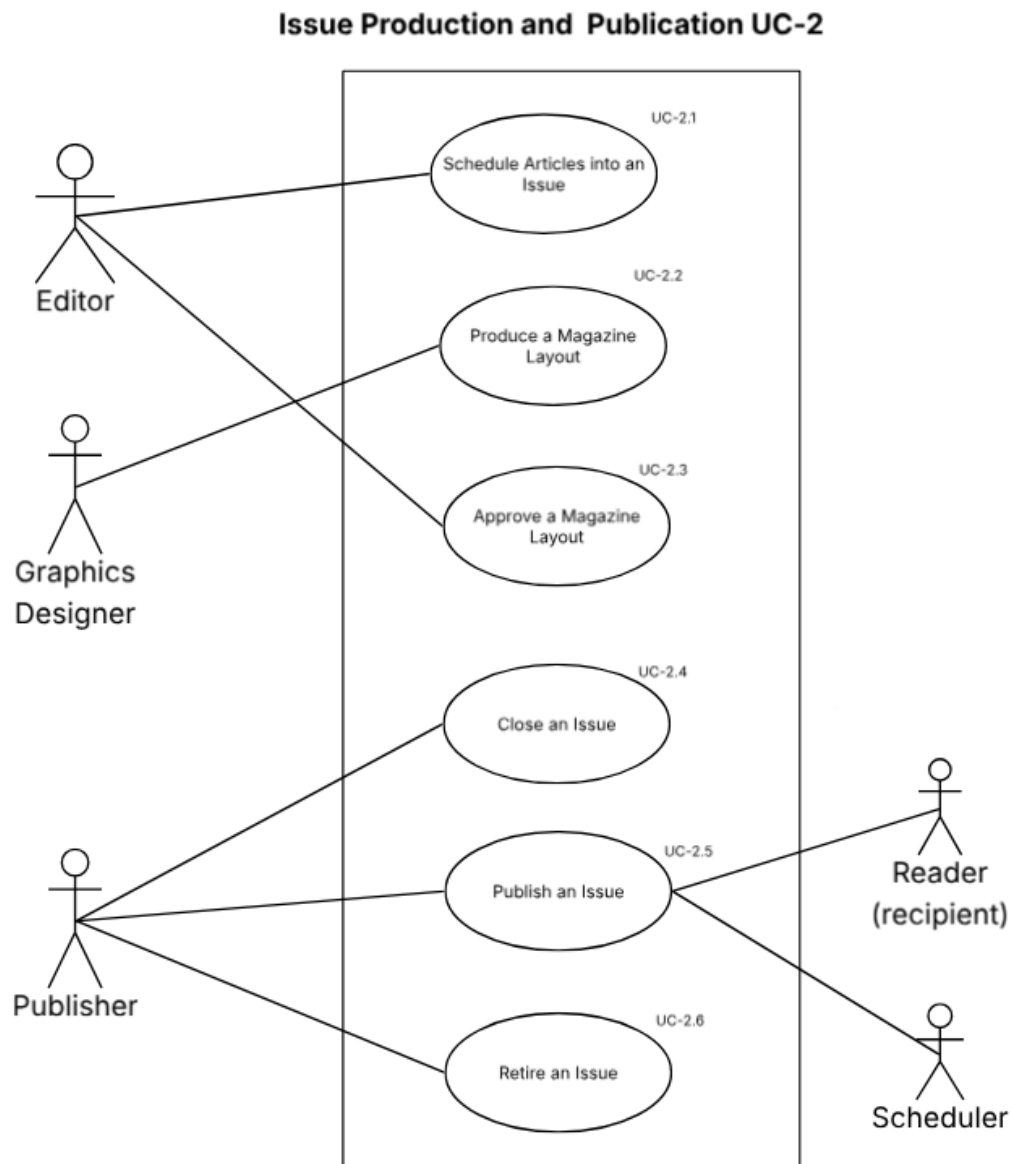


Figure 4 Issue Production and Publication

Use Case Name: Schedule Articles into an Issue

Use Case ID:	UC-2.1
Author	Rei Khalil S. Galido
Purpose	Enable an Editor to determine which approved articles form a given issue, and on what terms readers may see each of them.
Related Use Case	None
Priority	High

Preconditions	• The system is available • The Editor is authenticated in the Editor role • The article is approved • The target issue exists and remains open • The article has been approved under UC-1.3.1																														
Postconditions	• The article belongs to the issue and carries a position within it • The article's reader access is recorded • The issue's readiness reflects the change • The article becomes visible to the Graphic Designer																														
Actors	• Editor • Graphic Designer (recipient)																														
Flow of Actions	BASIC FLOW <table><thead><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr></thead><tbody><tr><td>1</td><td>Editor opens the publishing panel on an approved article.</td><td>1.1 System lists the issues open for scheduling with their current contents and planned counts.</td></tr><tr><td>2</td><td>Editor selects an issue.</td><td>2.1 System places the article in the issue and marks it subscriber-only.</td></tr><tr><td>3</td><td>Editor sets the reader access, overriding the default where the piece is to run free.</td><td>3.1 System records the access level and applies it to the reader portal.</td></tr><tr><td></td><td></td><td>3.2 System updates the issue's readiness and makes the article visible for layout.</td></tr><tr><td>4</td><td>Graphic Designer opens the workspace.</td><td>4.1 System lists the article under its issue in Ready for layout.</td></tr></tbody></table> ALTERNATE FLOW A1 (Left as a Standalone Article) Branches from step 2 of the basic flow.<table><thead><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr></thead><tbody><tr><td>1</td><td>Editor chooses not to place the article in an issue.</td><td>1.1 System clears any existing issue and marks the article free to read, since it no longer forms part of the paid product.</td></tr><tr><td></td><td></td><td>1.2 System makes the article eligible for standalone release by the Publisher.</td></tr><tr><td></td><td></td><td>1.3 System leaves the article out of the Graphic Designer's queue, since a web-only piece has no print layout.</td></tr></tbody></table> The use case ends. ALTERNATE FLOW	Step	Actor Action	System Response	1	Editor opens the publishing panel on an approved article.	1.1 System lists the issues open for scheduling with their current contents and planned counts.	2	Editor selects an issue.	2.1 System places the article in the issue and marks it subscriber-only.	3	Editor sets the reader access, overriding the default where the piece is to run free.	3.1 System records the access level and applies it to the reader portal.			3.2 System updates the issue's readiness and makes the article visible for layout.	4	Graphic Designer opens the workspace.	4.1 System lists the article under its issue in Ready for layout.	Step	Actor Action	System Response	1	Editor chooses not to place the article in an issue.	1.1 System clears any existing issue and marks the article free to read, since it no longer forms part of the paid product.			1.2 System makes the article eligible for standalone release by the Publisher.			1.3 System leaves the article out of the Graphic Designer's queue, since a web-only piece has no print layout.
Step	Actor Action	System Response																													
1	Editor opens the publishing panel on an approved article.	1.1 System lists the issues open for scheduling with their current contents and planned counts.																													
2	Editor selects an issue.	2.1 System places the article in the issue and marks it subscriber-only.																													
3	Editor sets the reader access, overriding the default where the piece is to run free.	3.1 System records the access level and applies it to the reader portal.																													
		3.2 System updates the issue's readiness and makes the article visible for layout.																													
4	Graphic Designer opens the workspace.	4.1 System lists the article under its issue in Ready for layout.																													
Step	Actor Action	System Response																													
1	Editor chooses not to place the article in an issue.	1.1 System clears any existing issue and marks the article free to read, since it no longer forms part of the paid product.																													
		1.2 System makes the article eligible for standalone release by the Publisher.																													
		1.3 System leaves the article out of the Graphic Designer's queue, since a web-only piece has no print layout.																													

A2 (Issue Below Its Planned Count)*Branches from step 2.1 of the basic flow.*

Step	Actor Action	System Response
1	Editor places an article in an issue that holds fewer articles than planned.	1.1 System states the shortfall beside the issue.
		1.2 System proceeds with the scheduling, as the planned count is an intention rather than a limit.
		1.3 System continues to report the issue as not yet ready.

*Resumes at step 3 of the basic flow.***EXCEPTION FLOW****E1 (Issue Closed)***Branches from step 2 of the basic flow.*

Step	Actor Action	System Response
1	Editor selects an issue whose contents have been closed.	1.1 System detects that the table of contents is fixed.
		1.2 System refuses the change and leaves the issue untouched.
		1.3 System explains that the Publisher may reopen the issue.
2	Editor selects a different issue, or asks the Publisher to reopen this one.	2.1 System lists the issues open for scheduling.

*Resumes at step 2 of the basic flow.***EXCEPTION FLOW****E-SAVE (Persistence Failure)***Branches from step 2 of the basic flow.*

Step	Actor Action	System Response
1	Editor places the article in an issue.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
2	Editor retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.

*Resumes at step 2 of the basic flow.***EXCEPTION FLOW**

	E-AUTH (Authorization or Session Failure)		
	<i>Branches from step 2 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Editor places the article in an issue after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
			1.2 System blocks the change without partly saving it.
			1.3 System redirects to sign-in where the session has expired, or states that access is denied.
	2	Editor signs in again.	2.1 System restores the session and returns to the page.
<i>Where the session had expired, resumes at step 2 of the basic flow; where the role or ownership was missing, the use case ends.</i>			

Use Case Name: Produce a Magazine Layout -

Use Case ID:	UC-2.2		
Author	Rei Khalil S. Galido		
Purpose	Enable a Graphic Designer to lay out an issue and submit the edition readers will receive.		
Related Use Case	None		
Priority	High		
Preconditions	• The system is available • The Graphic Designer is authenticated in the Graphic Designer role • The issue holds at least one approved article		
Postconditions	• A versioned edition is recorded against the issue • The edition is held in protected storage and is not publicly addressable • The cover image is recorded for the public archive • The Editor is notified that an edition awaits review		
Actors	• Graphic Designer • Editor (notified)		
Flow of Actions	BASIC FLOW		
	Step	Actor Action	System Response
	1	Graphic Designer opens the workspace.	1.1 System lists each issue awaiting layout with the articles scheduled into it.

2	Graphic Designer reads the approved copy and produces the layout outside the system.	
3	Graphic Designer selects the issue, enters the version label, and attaches the edition PDF, the cover image and any notes for the Editor.	3.1 System previews the cover image.
4	Graphic Designer submits the layout for editor review.	4.1 System confirms the issue is open, that no version is already under review, and that the version label is unused.
		4.2 System issues a signed instruction permitting one transfer to storage, valid briefly.
		4.3 System transfers the edition from the browser directly to storage and reports the progress.
		4.4 System verifies with storage that the edition arrived and matches what was declared.
		4.5 System records the layout as a new version awaiting review and notifies the Editor.
5	Editor opens their dashboard.	5.1 System shows the layout awaiting review.

ALTERNATE FLOW**A1 (Corrected Version Submitted)**

Branches after UC-2.3 alternate flow A1, when the Editor has returned the edition with notes.

Step	Actor Action	System Response
1	Graphic Designer opens the returned version.	1.1 System shows the Editor's notes beside the version.
2	Graphic Designer attaches a corrected edition under a new version label and submits it.	2.1 System validates that the label is unique within the issue.
		2.2 System records the new version and keeps the previous one.
		2.3 System notifies the Editor.

Resumes at step 5 of the basic flow.

ALTERNATE FLOW**A-VALID (Incomplete or Invalid Input)**

Branches from step 4 of the basic flow.

Step	Actor Action	System Response
------	--------------	-----------------

1	Graphic Designer submits the layout with a required item missing or malformed.	1.1 System validates every required item before saving anything.
		1.2 System marks each problem beside the field it concerns, with guidance.
		1.3 System saves nothing, and the workflow state is unchanged.
2	Graphic Designer corrects the items and submits again.	2.1 System validates the input again.

Resumes at step 4.2 of the basic flow.

EXCEPTION FLOW

E1 (Version Already Under Review)

Branches from step 4.1 of the basic flow.

Step	Actor Action	System Response
1	Graphic Designer submits while a version for this issue awaits the Editor.	1.1 System detects the pending version before permitting any transfer.
		1.2 System refuses the submission and names the version already with the Editor.
		1.3 System explains that the Editor's response comes first.
		1.4 System transfers nothing and creates no record.

The use case ends.

EXCEPTION FLOW

E2 (Transfer Incomplete)

Branches from step 4.3 of the basic flow.

Step	Actor Action	System Response
1	Graphic Designer's transfer to storage is interrupted or abandoned.	1.1 System asks storage whether the edition arrived, rather than trusting the browser's report.
		1.2 System finds nothing in storage and creates no record, so nothing claims a file that does not exist.
		1.3 System leaves any partial object to be removed by the storage retention sweep.
2	Graphic Designer submits the layout again.	2.1 System issues a fresh transfer instruction.

Resumes at step 4.3 of the basic flow.

EXCEPTION FLOW

E3 (Working File Submitted)*Branches from step 4.1 of the basic flow.*

Step	Actor Action	System Response
1	Graphic Designer attaches a working file rather than an exported PDF.	1.1 System validates the file type before storing anything.
		1.2 System refuses the submission.
		1.3 System explains that the file becomes the edition subscribers read, and must therefore be exported as a PDF.
2	Graphic Designer exports the layout as a PDF and attaches it.	2.1 System validates the file type.

*Resumes at step 4 of the basic flow.***EXCEPTION FLOW****E4 (Issue Already Published)***Branches from step 4.1 of the basic flow.*

Step	Actor Action	System Response
1	Graphic Designer submits a layout for an issue that has already been published.	1.1 System detects that the issue is published.
		1.2 System refuses the submission and states that a published issue's edition is final.
		1.3 System transfers nothing and creates no record.

*The use case ends.***EXCEPTION FLOW****E-SAVE (Persistence Failure)***Branches from step 4 of the basic flow.*

Step	Actor Action	System Response
1	Graphic Designer submits the layout.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
2	Graphic Designer retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.

Resumes at step 4 of the basic flow.

	EXCEPTION FLOW E-AUTH (Authorization or Session Failure) <i>Branches from step 4 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Graphic Designer submits the layout after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
			1.2 System blocks the change without partly saving it.
			1.3 System redirects to sign-in where the session has expired, or states that access is denied.
	2	Graphic Designer signs in again.	2.1 System restores the session and returns to the page.
<i>Where the session had expired, resumes at step 4 of the basic flow; where the role or ownership was missing, the use case ends.</i>			

Use Case Name: Approve a Magazine Layout

Use Case ID:	UC-2.3					
Author	Roman Rico Albania					
Purpose	Enable an Editor to accept a laid-out edition, or to return it with the changes required.					
Related Use Case	None					
Priority	High					
Preconditions	• The system is available • The Editor is authenticated in the Editor role • An edition version awaits review • An edition version submitted under UC-2.2 awaits review					
Postconditions	• The edition is accepted or returned with notes • On acceptance, the reviewing Editor and the time are recorded • On acceptance, every other version for the issue is retired • The version becomes the issue's edition, released to subscribers when the issue is published					
Actors	• Editor • Graphic Designer (notified)					
Flow of Actions	BASIC FLOW <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr></table>			Step	Actor Action	System Response
Step	Actor Action	System Response				

1	Editor opens the layout awaiting review.	1.1 System displays the version, the Graphic Designer's notes and a link to open the file.
2	Editor opens and examines the edition.	
3	Editor approves it.	3.1 System records the approval with the Editor and the time.
		3.2 System retires every other version for the issue, whether approved earlier, awaiting review, or returned.
		3.3 System notifies the Graphic Designer.
4	Graphic Designer opens the workspace.	4.1 System shows the version as approved and lists the retired versions under Earlier versions.

ALTERNATE FLOW

A1 (Returned for Revision)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	Editor selects to return the edition.	1.1 System presents a notes field.
2	Editor records the changes required and confirms.	2.1 System validates that notes have been supplied.
		2.2 System returns the version to the Graphic Designer and notifies them.
3	Graphic Designer opens the workspace.	3.1 System shows the version returned, with the Editor's notes.

The Graphic Designer submits a corrected version under UC-2.2. The use case ends.

EXCEPTION FLOW

E-SAVE (Persistence Failure)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	Editor approves the layout.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
2	Editor retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.

	<i>Resumes at step 3 of the basic flow.</i>	
	EXCEPTION FLOW	
	E-AUTH (Authorization or Session Failure)	
	<i>Branches from step 3 of the basic flow.</i>	
	Step	Actor Action
	1	Editor approves the layout after the session has expired, or without the role or ownership the action requires.
		1.1 System checks the session, and the role or ownership the action requires, before changing anything.
		1.2 System blocks the change without partly saving it.
		1.3 System redirects to sign-in where the session has expired, or states that access is denied.
	2	Editor signs in again.
		2.1 System restores the session and returns to the page.
	<i>Where the session had expired, resumes at step 3 of the basic flow; where the role or ownership was missing, the use case ends.</i>	

Use Case Name: Close an Issue

Use Case ID:	UC-2.4		
Author	Joshmar Clavero		
Purpose	Enable a Publisher to fix the table of contents of an issue, so that its layout can be finalised without the contents shifting beneath it.		
Related Use Case	None		
Priority	High		
Preconditions	• The system is available • The Publisher is authenticated in the Publisher role • The issue holds at least one scheduled article		
Postconditions	• The issue's contents are fixed and further scheduling is refused • The closing Publisher and the time are recorded • Editors and Graphic designers are notified that the contents are settled • The issue becomes eligible for publication		
Actors	• Publisher • Editor (notified) • Graphic Designer (notified)		
Flow of Actions	BASIC FLOW		
	Step	Actor Action	System Response

1	Publisher opens an issue in preparation.	1.1 System displays the issue's contents, readiness and anything blocking it.
2	Publisher selects to close the issue.	2.1 System presents a confirmation naming the article count and the consequences.
3	Publisher confirms.	3.1 System validates that the issue holds at least one article.
		3.2 System fixes the contents and records the closing Publisher and the time.
		3.3 System notifies Editors and Designers that the contents are settled.
4	Editor or Graphic Designer opens the issues overview.	4.1 System shows the issue as closed, with its contents fixed.

ALTERNATE FLOW**A1 (Issue Reopened)**

Branches after step 3 of the basic flow, when a late story warrants admission to the closed issue.

Step	Actor Action	System Response
1	Publisher opens the closed issue and selects to reopen it.	1.1 System asks for a reason.
2	Publisher enters the reason and confirms.	2.1 System validates that a reason of sufficient length is supplied.
		2.2 System reopens the issue and records the reason against it.
		2.3 System allows Editors to schedule into the issue once more.

The use case ends.

ALTERNATE FLOW**A2 (Issue Below Its Planned Count)**

Branches from step 2.1 of the basic flow.

Step	Actor Action	System Response
1	Publisher selects to close an issue holding fewer articles than planned.	1.1 System states the shortfall in the confirmation.
2	Publisher confirms regardless.	2.1 System closes the issue, as the planned count is an intention rather than a limit.
		2.2 System continues to report the issue as not yet ready until the count is met or the plan revised.

Resumes at step 4 of the basic flow.

EXCEPTION FLOW

E1 (Empty Issue)

Branches from step 3.1 of the basic flow.

Step	Actor Action	System Response
1	Publisher confirms closing an issue that holds no articles.	1.1 System validates the article count before committing anything.
		1.2 System refuses the closure.
		1.3 System explains that an empty issue has no contents to fix.

The use case ends.

EXCEPTION FLOW

E2 (Reason Absent on Reopening)

Branches from step 2 of alternate flow A1.

Step	Actor Action	System Response
1	Publisher confirms the reopening without an adequate reason.	1.1 System checks the reason before committing anything.
		1.2 System refuses and leaves the issue closed.
2	Publisher enters a reason and confirms again.	2.1 System validates the reason.

Resumes at step 2 of alternate flow A1.

EXCEPTION FLOW

E-SAVE (Persistence Failure)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	Publisher confirms the closure.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
2	Publisher retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.

Resumes at step 3 of the basic flow.

EXCEPTION FLOW

E-AUTH (Authorization or Session Failure)

Branches from step 3 of the basic flow.

	Step	Actor Action	System Response
	1	Publisher confirms the closure after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
			1.2 System blocks the change without partly saving it.
			1.3 System redirects to sign-in where the session has expired, or states that access is denied.
	2	Publisher signs in again.	2.1 System restores the session and returns to the page.
<i>Where the session had expired, resumes at step 3 of the basic flow; where the role or ownership was missing, the use case ends.</i>			

Use Case Name: Publish an Issue

Use Case ID:	UC-2.5							
Author	Joshmar Clavero							
Purpose	Release an issue and every article within it to readers, as a single indivisible act.							
Related Use Case	None							
Priority	High							
Preconditions	• The system is available • The Publisher is authenticated in the Publisher role • The issue is closed • Every scheduled article is approved • The issue meets its planned article count • An edition has been accepted for the issue • The issue has been closed under UC-2.4							
Postconditions	• The issue and every article within it are released to readers • The release time and the releasing Publisher are recorded • Subscribers are notified that a new issue is available • The digital edition is available where an accepted edition exists							
Actors	• Publisher • Reader (recipient) • Scheduler							
Flow of Actions	BASIC FLOW <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Publisher opens a closed issue and selects to publish.</td><td>1.1 System confirms the issue is closed, holds an approved edition, and that every article in it is approved.</td></tr> </table>		Step	Actor Action	System Response	1	Publisher opens a closed issue and selects to publish.	1.1 System confirms the issue is closed, holds an approved edition, and that every article in it is approved.
Step	Actor Action	System Response						
1	Publisher opens a closed issue and selects to publish.	1.1 System confirms the issue is closed, holds an approved edition, and that every article in it is approved.						

		1.2 System presents a confirmation naming every article that will be released.
2	Publisher confirms.	2.1 System releases the issue and all its articles in a single transaction.
		2.2 System records the release time and the releasing Publisher.
		2.3 System removes superseded layout versions from storage, keeping their records.
		2.4 System notifies subscribers that a new issue is available.
3	Reader opens the reader portal.	3.1 System presents the new issue and its articles.

ALTERNATE FLOW**A1 (Standalone Article Released)***Branches from step 1 of the basic flow.*

Step	Actor Action	System Response
1	Publisher selects to publish an approved article that belongs to no issue.	1.1 System confirms that the article publishes on its own.
2	Publisher confirms.	2.1 System releases the single article without reference to any issue.
		2.2 System presents the article on the reader portal as a web piece with no edition.

*The use case ends.***ALTERNATE FLOW****A2 (Release Scheduled)***Branches from step 2 of the basic flow.*

Step	Actor Action	System Response
1	Publisher sets a future release time rather than publishing at once.	1.1 System validates that the time lies ahead and that the issue is ready.
		1.2 System records the scheduled release.
2	Scheduler reaches the appointed time.	2.1 System releases the issue as in step 2 of the basic flow.
		2.2 System notifies the Publisher that the release has occurred.

*Resumes at step 3 of the basic flow.***EXCEPTION FLOW****E1 (No Accepted Edition)**

Branches from step 1.1 of the basic flow.

Step	Actor Action	System Response
1	Publisher selects to publish an issue with no approved edition.	1.1 System detects that no layout has been approved for the issue.
		1.2 System refuses publication and names the requirement.
		1.3 System explains that the Graphic Designer must submit the edition and an Editor approve it.
		1.4 System publishes nothing.

The use case ends.

EXCEPTION FLOW

E2 (Issue Not Closed)

Branches from step 1.1 of the basic flow.

Step	Actor Action	System Response
1	Publisher selects to publish an issue still open for scheduling.	1.1 System detects that the contents are not fixed.
		1.2 System refuses publication and names the closure requirement.
2	Publisher closes the issue under UC-2.4.	2.1 System fixes the contents.

Resumes at step 1 of the basic flow.

EXCEPTION FLOW

E3 (Unapproved Article Present)

Branches from step 1.1 of the basic flow.

Step	Actor Action	System Response
1	Publisher selects to publish an issue containing an article that has not been approved.	1.1 System evaluates readiness before beginning the release.
		1.2 System refuses publication and names each blocking article.
		1.3 System releases no article, since a partially published issue is not an issue.

The use case ends.

EXCEPTION FLOW

E4 (Scheduled Release Cannot Proceed)

Branches from step 2 of alternate flow A2.

Step	Actor Action	System Response
------	--------------	-----------------

1	Scheduler reaches the appointed time after an article in the issue has been withdrawn.	1.1 System finds the issue no longer ready.
		1.2 System cancels the release rather than publishing an incomplete issue.
		1.3 System notifies the Publisher that intervention is required.

The use case ends.

EXCEPTION FLOW

E-SAVE (Persistence Failure)

Branches from step 2 of the basic flow.

Step	Actor Action	System Response
1	Publisher confirms the publication.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
2	Publisher retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.

Resumes at step 2 of the basic flow.

EXCEPTION FLOW

E-AUTH (Authorization or Session Failure)

Branches from step 2 of the basic flow.

Step	Actor Action	System Response
1	Publisher confirms the publication after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
		1.2 System blocks the change without partly saving it.
		1.3 System redirects to sign-in where the session has expired, or states that access is denied.
2	Publisher signs in again.	2.1 System restores the session and returns to the page.

Where the session had expired, resumes at step 2 of the basic flow; where the role or ownership was missing, the use case ends.

Use Case Name: Retire an Issue

Use Case ID:	UC-2.6																															
Author	Joshua Concepcion																															
Purpose	Enable a Publisher to move a completed issue out of the active publishing list while readers keep access to it.																															
Related Use Case	None																															
Priority	Low																															
Preconditions	• The system is available • The Publisher is authenticated in the Publisher role • The issue has been published • The issue has been published under UC-2.5																															
Postconditions	• The issue no longer appears among issues in preparation • The issue and its articles remain readable on the reader portal • The issue remains searchable in the published archive																															
Actors	• Publisher																															
Flow of Actions	BASIC FLOW <table border="1"> <thead> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> </thead> <tbody> <tr> <td>1</td><td>Publisher opens a published issue and selects to retire it.</td><td>1.1 System presents a confirmation stating that readers keep access.</td></tr> <tr> <td>2</td><td>Publisher confirms.</td><td>2.1 System retires the issue from the active publishing list.</td></tr> <tr> <td></td><td></td><td>2.2 System leaves the issue and its articles available to readers and searchable in the archive.</td></tr> </tbody> </table> EXCEPTION FLOW E-SAVE (Persistence Failure) <i>Branches from step 2 of the basic flow.</i> <table border="1"> <thead> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> </thead> <tbody> <tr> <td>1</td><td>Publisher confirms the retirement.</td><td>1.1 System validates the request.</td></tr> <tr> <td></td><td></td><td>1.2 System attempts to save the change, and the save fails or times out.</td></tr> <tr> <td></td><td></td><td>1.3 System keeps the last confirmed state and does not report success.</td></tr> <tr> <td></td><td></td><td>1.4 System records the failure and displays a message inviting a retry.</td></tr> <tr> <td>2</td><td>Publisher retries once the service has recovered.</td><td>2.1 System keeps the input on screen, so nothing needs to be entered again.</td></tr> </tbody> </table> <i>Resumes at step 2 of the basic flow.</i>		Step	Actor Action	System Response	1	Publisher opens a published issue and selects to retire it.	1.1 System presents a confirmation stating that readers keep access.	2	Publisher confirms.	2.1 System retires the issue from the active publishing list.			2.2 System leaves the issue and its articles available to readers and searchable in the archive.	Step	Actor Action	System Response	1	Publisher confirms the retirement.	1.1 System validates the request.			1.2 System attempts to save the change, and the save fails or times out.			1.3 System keeps the last confirmed state and does not report success.			1.4 System records the failure and displays a message inviting a retry.	2	Publisher retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.
Step	Actor Action	System Response																														
1	Publisher opens a published issue and selects to retire it.	1.1 System presents a confirmation stating that readers keep access.																														
2	Publisher confirms.	2.1 System retires the issue from the active publishing list.																														
		2.2 System leaves the issue and its articles available to readers and searchable in the archive.																														
Step	Actor Action	System Response																														
1	Publisher confirms the retirement.	1.1 System validates the request.																														
		1.2 System attempts to save the change, and the save fails or times out.																														
		1.3 System keeps the last confirmed state and does not report success.																														
		1.4 System records the failure and displays a message inviting a retry.																														
2	Publisher retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.																														

EXCEPTION FLOW**E-AUTH (Authorization or Session Failure)**

Branches from step 2 of the basic flow.

Step	Actor Action	System Response
1	Publisher confirms the retirement after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
		1.2 System blocks the change without partly saving it.
		1.3 System redirects to sign-in where the session has expired, or states that access is denied.
2	Publisher signs in again.	2.1 System restores the session and returns to the page.

Where the session had expired, resumes at step 2 of the basic flow; where the role or ownership was missing, the use case ends.

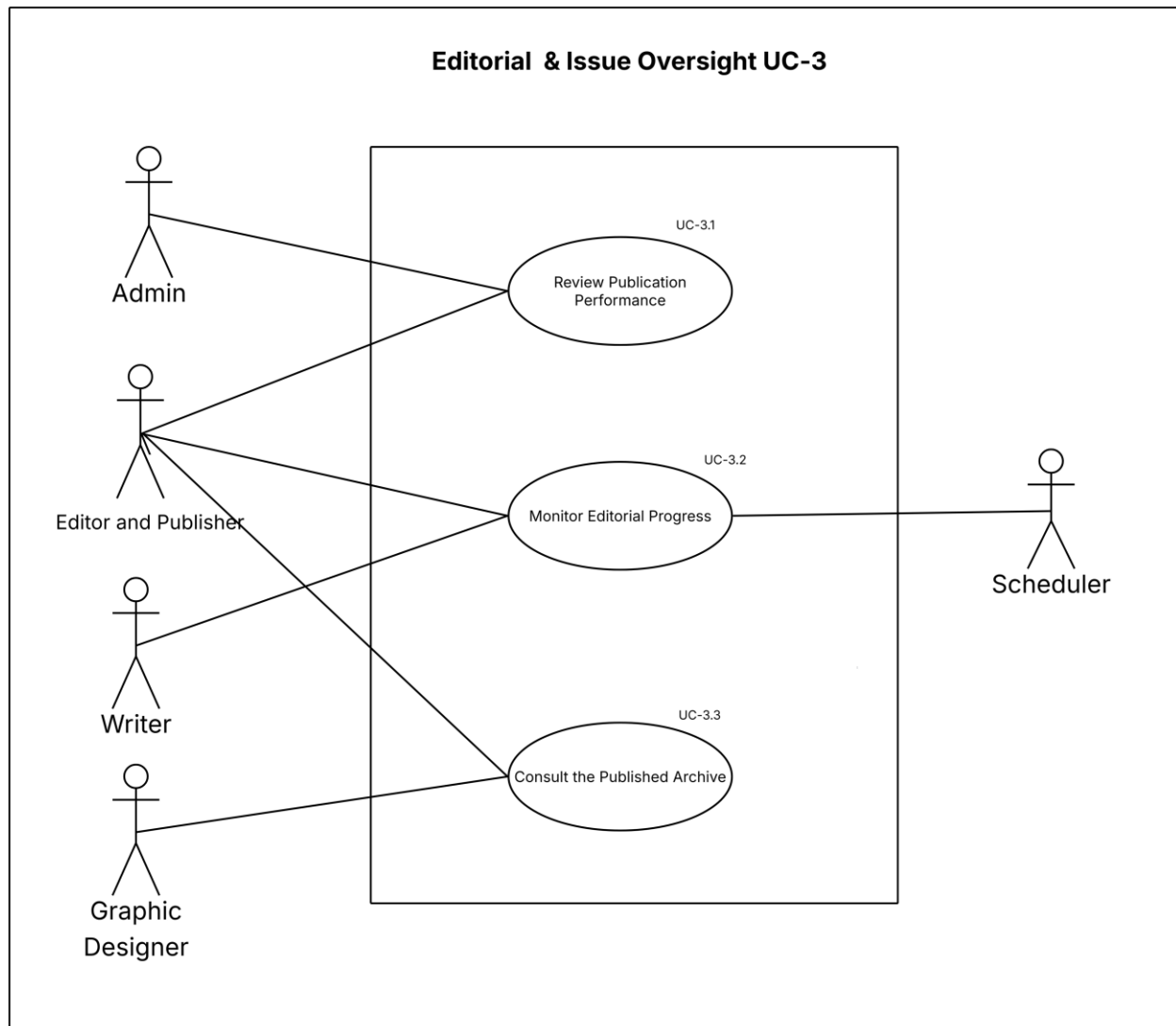


Figure 5 Editorial & Issue Oversight

Use Case Name: Review Publication Performance

Use Case ID:	UC-3.1																									
Author	Roman Rico Albania																									
Purpose	Enable an Editor, Publisher or Administrator to understand how the publication is performing: what is moving, how long it takes, how the assessment is behaving, and what the subscriber base is worth.																									
Related Use Case	None																									
Priority	Medium																									
Preconditions	• The system is available • The actor is authenticated in the Editor, Publisher or Administrator role • Revenue reporting additionally requires the Publisher or Administrator role																									
Postconditions	• The requested report is presented for the chosen period • The basis of any derived figure is stated • No state change occurs																									
Actors	• Editor • Publisher • Administrator																									
Flow of Actions	BASIC FLOW <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Editor, Publisher or Administrator opens the reports area and selects a report.</td><td></td></tr> <tr> <td>2</td><td>Editor, Publisher or Administrator sets the period to report on.</td><td>2.1 System aggregates the underlying records for that period.</td></tr> <tr> <td></td><td></td><td>2.2 System presents the report with its figures and their basis.</td></tr> <tr> <td></td><td></td><td>2.3 System accompanies each chart with a textual description, so the figures are available to anyone who cannot read the chart.</td></tr> <tr> <td>3</td><td>Editor, Publisher or Administrator changes the period or moves to another report.</td><td></td></tr> </table> ALTERNATE FLOW A1 (No Data in Period) Branches from step 2.1 of the basic flow. <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Editor, Publisher or Administrator sets a period that holds no records of the kind reported.</td><td>1.1 System states that no data exists for the period.</td></tr> </table>		Step	Actor Action	System Response	1	Editor, Publisher or Administrator opens the reports area and selects a report.		2	Editor, Publisher or Administrator sets the period to report on.	2.1 System aggregates the underlying records for that period.			2.2 System presents the report with its figures and their basis.			2.3 System accompanies each chart with a textual description, so the figures are available to anyone who cannot read the chart.	3	Editor, Publisher or Administrator changes the period or moves to another report.		Step	Actor Action	System Response	1	Editor, Publisher or Administrator sets a period that holds no records of the kind reported.	1.1 System states that no data exists for the period.
Step	Actor Action	System Response																								
1	Editor, Publisher or Administrator opens the reports area and selects a report.																									
2	Editor, Publisher or Administrator sets the period to report on.	2.1 System aggregates the underlying records for that period.																								
		2.2 System presents the report with its figures and their basis.																								
		2.3 System accompanies each chart with a textual description, so the figures are available to anyone who cannot read the chart.																								
3	Editor, Publisher or Administrator changes the period or moves to another report.																									
Step	Actor Action	System Response																								
1	Editor, Publisher or Administrator sets a period that holds no records of the kind reported.	1.1 System states that no data exists for the period.																								

			1.2 System offers a wider period.
			1.3 System draws no empty chart, since an empty chart reads as zero rather than absent.
	<i>Resumes at step 3 of the basic flow.</i>		
	ALTERNATE FLOW		
	A2 (Demonstration Data Present)		
	<i>Branches from step 2.2 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Editor, Publisher or Administrator opens a report whose period includes assessments produced for demonstration rather than by the evaluation service.	1.1 System identifies the demonstration assessments in the report.
			1.2 System presents the figures with that qualification rather than suppressing them.
	<i>Resumes at step 3 of the basic flow.</i>		
	EXCEPTION FLOW		
	E-AUTH (Authorization or Session Failure)		
	<i>Branches from step 1 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Editor, Publisher or Administrator opens a report after the session has expired, or without a role permitted to see it.	1.1 System checks the session and the role before showing anything.
			1.2 System shows nothing the role does not permit.
			1.3 System redirects to sign-in where the session has expired, or returns the user to the dashboard for their own role.
	2	Editor, Publisher or Administrator signs in again.	2.1 System restores the session and returns to the page.
	<i>Where the session had expired, resumes at step 1 of the basic flow; where the role was not permitted, the use case ends.</i>		

Use Case Name: Monitor Editorial Progress

Use Case ID:	UC-3.2
Author	Joshmar Clavero
Purpose	Enable staff to see what needs their attention, what is sitting with somebody else, and what is at risk of being late.

Related Use Case	None		
Priority	High		
Preconditions	• The system is available • The actor is authenticated in a staff role		
Postconditions	• Work is presented grouped by whether the actor can act on it • Overdue work is identified • Deadline warnings have been delivered to those responsible • No state change occurs		
Actors	• Writer • Editor • Publisher • Graphic Designer • Scheduler		
Flow of Actions	BASIC FLOW		
	Step	Actor Action	System Response
	1	Staff member signs in and arrives at their dashboard.	1.1 System determines the actor's role and assembles the queues relevant to it.
			1.2 System groups the work by whether the actor can act on it now or is waiting on another party.
			1.3 System identifies overdue work and offers a filtered view of it.
	2	Staff member opens the calendar to see deadlines and release targets across the month.	2.1 System displays the month, distinguishing overdue work, work in hand and settled work.
	3	Staff member selects an item and acts on it.	
	ALTERNATE FLOW		
	A1 (Deadline Sweep)		
	Runs independently of the basic flow, at the scheduled time each day.		
	Step	Actor Action	System Response
	1	Scheduler calls the deadline sweep with its credential.	1.1 System authenticates the request against the scheduler credential.
			1.2 System identifies open work whose deadline is approaching or has passed.
			1.3 System notifies each Writer, and the commissioning Editor for overdue work.
			1.4 System records the outcome of the sweep.

2	Staff member signs in and arrives at their dashboard.	2.1 System shows the warned work among the overdue items.
---	---	---

Resumes at step 2 of the basic flow.

ALTERNATE FLOW

A2 (Nothing Outstanding)

Branches from step 1.2 of the basic flow.

Step	Actor Action	System Response
1	Staff member opens a queue or a calendar month that holds no items.	1.1 System explains what would appear there and how it arrives.
		1.2 System directs the staff member to the action that would populate it.
		1.3 System draws no empty chart or blank panel, since an empty display reads as zero rather than absent.

The use case ends.

EXCEPTION FLOW

E1 (Unauthenticated Sweep Request)

Branches from step 1 of alternate flow A1.

Step	Actor Action	System Response
1	Scheduler, or any other caller, requests the deadline sweep without a valid credential.	1.1 System refuses the request and sends no notifications.
		1.2 System records the refusal, since an unauthenticated call to a scheduled endpoint may indicate probing.

The use case ends.

EXCEPTION FLOW

E-AUTH (Authorization or Session Failure)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	Staff member opens an item belonging to another role's workspace after the session has expired, or without a role permitted to see it.	1.1 System checks the session and the role before showing anything.
		1.2 System shows nothing the role does not permit.
		1.3 System redirects to sign-in where the session has expired, or returns the user to the dashboard for their own role.
2	Staff member signs in again.	2.1 System restores the session and returns to the page.

	Where the session had expired, resumes at step 3 of the basic flow; where the role was not permitted, the use case ends.
--	--

Use Case Name: Consult the Published Archive

Use Case ID:	UC-3.3																												
Author	Joshua Concepcion																												
Purpose	Enable staff to find anything the publication has run, so that past work informs present decisions.																												
Related Use Case	None																												
Priority	Medium																												
Preconditions	• The system is available • The actor is authenticated in a staff role																												
Postconditions	• The matching published work is presented • No state change occurs																												
Actors	• Editor • Writer • Publisher • Graphic Designer																												
Flow of Actions	BASIC FLOW <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Staff member opens the archive.</td><td>1.1 System presents everything published, grouped by the month it ran.</td></tr> <tr> <td>2</td><td>Staff member searches a term, or filters by issue, category or year.</td><td>2.1 System presents the matching work with its issue and its reader access.</td></tr> <tr> <td>3</td><td>Staff member opens an article.</td><td>3.1 System opens a read-only preview with the full text, the photography and the article's publication details.</td></tr> <tr> <td></td><td></td><td>3.2 System offers a link to the article's page on the public site.</td></tr> </table> ALTERNATE FLOW A1 (No Matches) <i>Branches from step 2.1 of the basic flow.</i> <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Staff member searches a term or applies filters that match nothing.</td><td>1.1 System states that nothing matched.</td></tr> <tr> <td></td><td></td><td>1.2 System offers to clear the filters.</td></tr> <tr> <td></td><td></td><td>1.3 System keeps the total published count visible, so the absence reads as a</td></tr> </table>		Step	Actor Action	System Response	1	Staff member opens the archive.	1.1 System presents everything published, grouped by the month it ran.	2	Staff member searches a term, or filters by issue, category or year.	2.1 System presents the matching work with its issue and its reader access.	3	Staff member opens an article.	3.1 System opens a read-only preview with the full text, the photography and the article's publication details.			3.2 System offers a link to the article's page on the public site.	Step	Actor Action	System Response	1	Staff member searches a term or applies filters that match nothing.	1.1 System states that nothing matched.			1.2 System offers to clear the filters.			1.3 System keeps the total published count visible, so the absence reads as a
Step	Actor Action	System Response																											
1	Staff member opens the archive.	1.1 System presents everything published, grouped by the month it ran.																											
2	Staff member searches a term, or filters by issue, category or year.	2.1 System presents the matching work with its issue and its reader access.																											
3	Staff member opens an article.	3.1 System opens a read-only preview with the full text, the photography and the article's publication details.																											
		3.2 System offers a link to the article's page on the public site.																											
Step	Actor Action	System Response																											
1	Staff member searches a term or applies filters that match nothing.	1.1 System states that nothing matched.																											
		1.2 System offers to clear the filters.																											
		1.3 System keeps the total published count visible, so the absence reads as a																											

			narrow search rather than an empty archive.
	2	Staff member clears the filters.	2.1 System presents everything published, grouped by month.
	<i>Resumes at step 2 of the basic flow.</i>		
	EXCEPTION FLOW		
	E-AUTH (Authorization or Session Failure)		
	<i>Branches from step 1 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Staff member opens the archive after the session has expired, or without a role permitted to see it.	1.1 System checks the session and the role before showing anything.
			1.2 System shows nothing the role does not permit.
			1.3 System redirects to sign-in where the session has expired, or returns the user to the dashboard for their own role.
	2	Staff member signs in again.	2.1 System restores the session and returns to the page.
	<i>Where the session had expired, resumes at step 1 of the basic flow; where the role was not permitted, the use case ends.</i>		

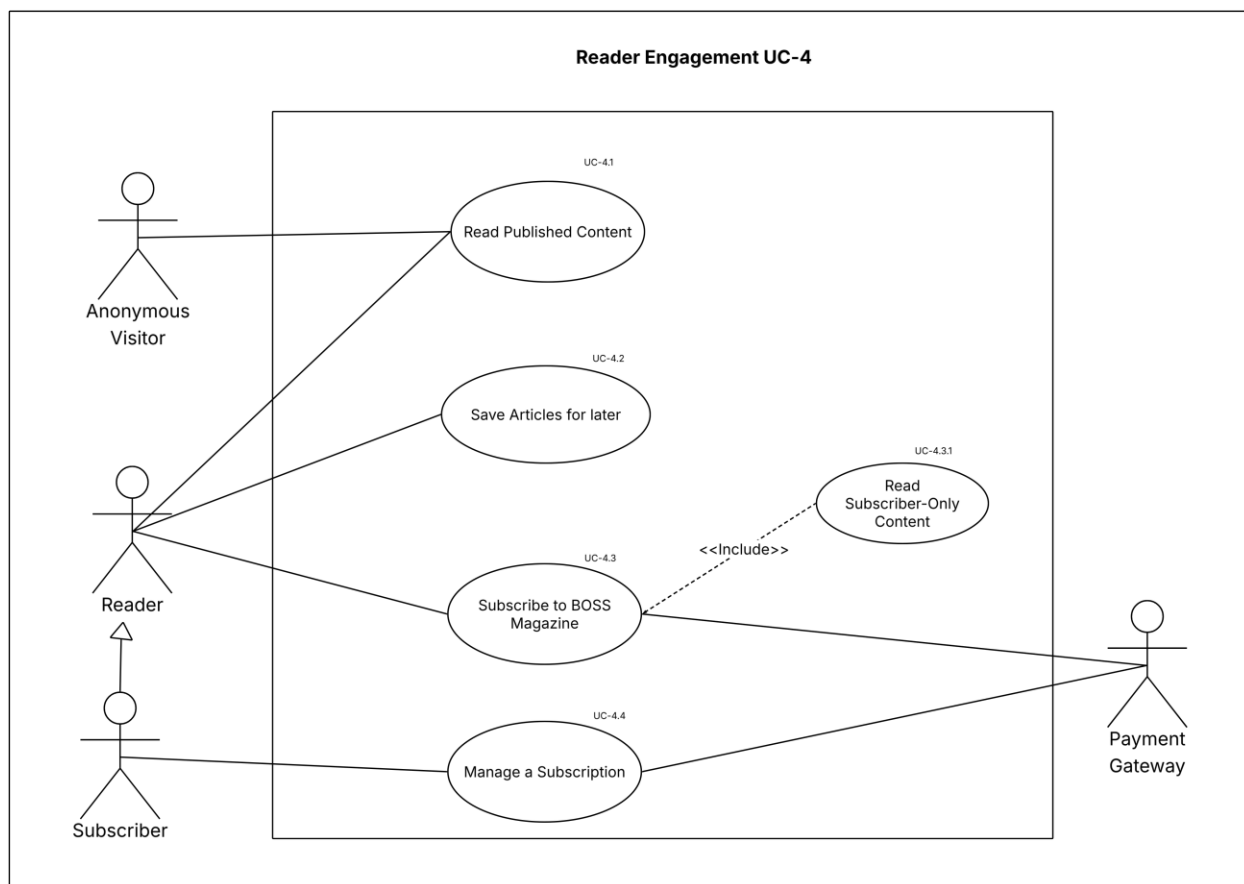


Figure 6 Reader Engagement

Use Case Name: Read Published Content

Use Case ID:	UC-4.1		
Author	Rei Khalil S. Galido		
Purpose	Enable anyone to read the publication, and to understand what a subscription would give them.		
Related Use Case	None		
Priority	High		
Preconditions	• The system is available • At least one article has been published		
Postconditions	• The requested content is presented within the reader's entitlement • No state change occurs		
Actors	• Reader • Anonymous Visitor		
Flow of Actions	BASIC FLOW		
	Step	Actor Action	System Response

	1	Visitor opens the reader portal.	1.1 System presents the lead article and the most recent published work.
	2	Visitor filters by category or opens an article.	2.1 System determines the visitor's entitlement.
			2.2 System presents the article in full where it is freely available.
	3	Visitor passes the article to somebody else.	3.1 System offers the device's sharing options, or copies the address where these are unavailable.
	ALTERNATE FLOW A1 (Subscriber-only Article) <i>Branches from step 2.1 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Visitor opens an article reserved for subscribers without a subscription.	1.1 System presents the standfirst and the opening passage.
			1.2 System presents the subscription notice beneath the extract.
			1.3 System withholds the remainder.
	2	Subscriber or staff member opens the same article.	2.1 System presents the article in full without interruption.
	<i>Resumes at step 3 of the basic flow.</i>		
	EXCEPTION FLOW E1 (Platform Under Maintenance) <i>Branches from step 1 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Visitor opens the reader portal while an Administrator has enabled maintenance.	1.1 System returns the maintenance notice.
			1.2 System serves no article content.
	2	Staff member signs in to the workspace.	2.1 System continues to serve staff normally, since a maintenance mode that locked out the people performing the maintenance would defeat itself.
	<i>The use case ends.</i>		

Use Case Name: Save Articles for Later

Use Case ID:	UC-4.2
Author	Roman Rico Albania
Purpose	Enable a reader to keep articles and return to them.

Related Use Case	«include» UC-4.3.1 (Read Subscriber-Only Content)																																					
Priority	Medium																																					
Preconditions	• The system is available • The reader is authenticated • The article has been published																																					
Postconditions	• The article appears in the reader's saved list • Saving the same article twice produces no duplicate																																					
Actors	• Reader • Payment Gateway																																					
Flow of Actions	BASIC FLOW <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Reader selects the save control on an article.</td><td>1.1 System records the article against the reader and confirms.</td></tr> <tr> <td>2</td><td>Reader opens their saved list.</td><td>2.1 System presents the saved articles, most recently saved first.</td></tr> <tr> <td>3</td><td>Reader removes an article from the list.</td><td>3.1 System removes the record and the article leaves the list.</td></tr> </table> ALTERNATE FLOW A1 (Visitor Not Signed In) <i>Branches from step 1 of the basic flow.</i> <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Anonymous visitor selects the save control on an article.</td><td>1.1 System records nothing.</td></tr> <tr> <td></td><td></td><td>1.2 System explains that saving requires an account and offers to sign in.</td></tr> <tr> <td></td><td></td><td>1.3 System keeps the visitor on the article, so their place is not lost.</td></tr> <tr> <td>2</td><td>Visitor signs in.</td><td>2.1 System returns them to the article.</td></tr> </table> <i>Resumes at step 1 of the basic flow.</i> EXCEPTION FLOW E-SAVE (Persistence Failure) <i>Branches from step 1 of the basic flow.</i> <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Reader selects the save control on an article.</td><td>1.1 System validates the request.</td></tr> <tr> <td></td><td></td><td>1.2 System attempts to save the change, and the save fails or times out.</td></tr> </table>		Step	Actor Action	System Response	1	Reader selects the save control on an article.	1.1 System records the article against the reader and confirms.	2	Reader opens their saved list.	2.1 System presents the saved articles, most recently saved first.	3	Reader removes an article from the list.	3.1 System removes the record and the article leaves the list.	Step	Actor Action	System Response	1	Anonymous visitor selects the save control on an article.	1.1 System records nothing.			1.2 System explains that saving requires an account and offers to sign in.			1.3 System keeps the visitor on the article, so their place is not lost.	2	Visitor signs in.	2.1 System returns them to the article.	Step	Actor Action	System Response	1	Reader selects the save control on an article.	1.1 System validates the request.			1.2 System attempts to save the change, and the save fails or times out.
Step	Actor Action	System Response																																				
1	Reader selects the save control on an article.	1.1 System records the article against the reader and confirms.																																				
2	Reader opens their saved list.	2.1 System presents the saved articles, most recently saved first.																																				
3	Reader removes an article from the list.	3.1 System removes the record and the article leaves the list.																																				
Step	Actor Action	System Response																																				
1	Anonymous visitor selects the save control on an article.	1.1 System records nothing.																																				
		1.2 System explains that saving requires an account and offers to sign in.																																				
		1.3 System keeps the visitor on the article, so their place is not lost.																																				
2	Visitor signs in.	2.1 System returns them to the article.																																				
Step	Actor Action	System Response																																				
1	Reader selects the save control on an article.	1.1 System validates the request.																																				
		1.2 System attempts to save the change, and the save fails or times out.																																				

			1.3 System keeps the last confirmed state and does not report success.
			1.4 System records the failure and displays a message inviting a retry.
	2	Reader retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.
	<i>Resumes at step 1 of the basic flow.</i>		
	EXCEPTION FLOW		
	E-AUTH (Authorization or Session Failure)		
	<i>Branches from step 1 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Reader selects the save control on an article after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
			1.2 System blocks the change without partly saving it.
			1.3 System redirects to sign-in where the session has expired, or states that access is denied.
	2	Reader signs in again.	2.1 System restores the session and returns to the page.
	<i>Where the session had expired, resumes at step 1 of the basic flow; where the role or ownership was missing, the use case ends.</i>		

Use Case Name: Subscribe to BOSS Magazine

Use Case ID:	UC-4.3
Author	Joshmar Clavero
Purpose	Enable a reader to purchase access to the publication's paid content.
Related Use Case	«include» UC-4.3.1 (Read Subscriber-Only Content)
Priority	High
Preconditions	• The system is available • The reader holds an account • The payment provider is reachable
Postconditions	• A subscription exists and the reader holds the subscriber tier • The transaction is recorded with its amount, currency and outcome • The provider's customer reference is recorded • No card detail is held by this system
Actors	• Reader

	• Payment Gateway		
Flow of Actions	BASIC FLOW		
	Step	Actor Action	System Response
	1	Reader selects a subscription plan.	1.1 System presents the terms and what the plan grants.
			1.2 System creates a checkout session with the Payment Gateway and directs the Reader to it.
	2	Reader supplies payment details to the Payment Gateway and confirms.	
	3	Payment Gateway processes the payment and returns the Reader to the platform.	3.1 System confirms the outcome with the Payment Gateway before granting anything.
			3.2 System records the transaction and grants the subscriber tier.
			3.3 System makes subscriber-only content available to the Reader immediately.
	ALTERNATE FLOW		
	A1 (Checkout Abandoned)		
	<i>Branches from step 2 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Reader closes the checkout without completing payment.	1.1 System receives no confirmation from the Payment Gateway.
			1.2 System grants no entitlement and records no transaction.
			1.3 System leaves the Reader's account unchanged.
	2	Reader selects a plan again.	2.1 System creates a fresh checkout session.
	<i>Resumes at step 2 of the basic flow.</i>		
	ALTERNATE FLOW		
	A2 (Duplicate Confirmation Received)		
	<i>Branches from step 3.1 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Payment Gateway sends a confirmation the system has already processed.	1.1 System recognises the transaction reference.
			1.2 System neither records the transaction twice nor grants the entitlement again.

		1.3 System acknowledges receipt so the Payment Gateway stops retrying.
<i>The use case ends.</i>		
EXCEPTION FLOW		
E1 (Payment Declined)		
<i>Branches from step 3 of the basic flow.</i>		
Step	Actor Action	System Response
1	Payment Gateway declines the payment and returns the Reader.	1.1 System records the outcome and grants no entitlement.
		1.2 System states that payment was not completed, without disclosing the Payment Gateway's reason.
2	Reader retries with different payment details.	2.1 System creates a fresh checkout session.
<i>Resumes at step 2 of the basic flow.</i>		
EXCEPTION FLOW		
E2 (Confirmation Not Received)		
<i>Branches from step 3.1 of the basic flow.</i>		
Step	Actor Action	System Response
1	Payment Gateway completes the payment, but no confirmation reaches the system.	1.1 System does not grant entitlement on the Reader's return alone.
		1.2 System queries the Payment Gateway for the definitive outcome.
		1.3 Where the payment is confirmed, System records the transaction and grants the subscriber tier.
		1.4 Where it cannot be confirmed, System records the discrepancy for administrator review rather than guessing.
<i>Where the payment was confirmed, resumes at step 3.3 of the basic flow; otherwise the use case ends.</i>		
EXCEPTION FLOW		
E3 (Confirmation Fails Verification)		
<i>Branches from step 3.1 of the basic flow.</i>		
Step	Actor Action	System Response
1	A confirmation arrives whose signature does not verify.	1.1 System does not process it and grants nothing.
		1.2 System records the rejected confirmation.

		1.3 System alerts an Administrator, since a failed signature may indicate an attempted forgery.
	<i>The use case ends.</i>	
	EXCEPTION FLOW	
	E-SAVE (Persistence Failure)	
	<i>Branches from step 3 of the basic flow.</i>	
	Step	Actor Action
	1	Payment Gateway returns the Reader after a successful payment.
		1.1 System confirms the outcome with the Payment Gateway.
		1.2 System attempts to record the transaction, and the save fails.
		1.3 System grants nothing it has not recorded.
		1.4 System records the failure for administrator review.
	2	Payment Gateway sends its payment notification again.
		2.1 System records the transaction and grants the subscriber tier.
	<i>Resumes at step 3.3 of the basic flow.</i>	

Use Case Name: Read Subscriber-Only Content

Use Case ID:	UC-4.3.1		
Author	Joshmar Clavero		
Purpose	Enable a subscriber to read the laid-out magazine, in the browser or as a file they keep.		
Related Use Case	Included by UC-4.3 (Subscribe to BOSS Magazine)		
Priority	High		
Preconditions	• The system is available • The reader is authenticated and holds a subscription • The issue has been published and carries an accepted edition • The issue has been published and carries an edition accepted under UC-2.3		
Postconditions	• A time-limited address for the edition is issued to the reader • The address expires shortly after issue • The reader's page-turn preference is retained		
Actors	• Reader (subscriber)		
Flow of Actions	BASIC FLOW		
	Step	Actor Action	System Response

1	Subscriber opens a subscriber-only article.	1.1 System verifies the subscription.
		1.2 System presents the article in full.
2	Subscriber opens a published issue and selects to read it.	2.1 System confirms that an approved edition exists.
		2.2 System issues a time-limited address for the edition.
		2.3 System renders the first page and reports the page count.
3	Subscriber turns pages by control, keyboard or slider.	3.1 System renders each page at the size the display allows.
4	Subscriber downloads the edition, or adjusts the page-turn treatment.	4.1 System retains the page-turn preference for subsequent readings.

ALTERNATE FLOW**A1 (Reduced Motion Requested)***Branches from step 3 of the basic flow.*

Step	Actor Action	System Response
1	Subscriber turns pages on a device that requests reduced motion.	1.1 System sets the page-turn treatment to none without being asked.
		1.2 System changes pages without animation.
2	Subscriber chooses an animated treatment explicitly.	2.1 System applies it and keeps the preference.

*Resumes at step 3 of the basic flow.***ALTERNATE FLOW****A2 (No Edition for the Issue)***Branches from step 2.1 of the basic flow.*

Step	Actor Action	System Response
1	Subscriber selects to read a published issue that carries no approved edition.	1.1 System states that no digital edition is available.
		1.2 System keeps the issue's articles readable on the reader portal.
		1.3 System makes an edition approved later available without republication.

*The use case ends.***EXCEPTION FLOW****E1 (Reader Not a Subscriber)***Branches from step 2.1 of the basic flow.*

Step	Actor Action	System Response
------	--------------	-----------------

	1	Free or anonymous reader selects to read the edition.	1.1 System checks the entitlement and finds none.
			1.2 System refuses the request and issues no address.
			1.3 System explains that the edition is available to subscribers.
			1.4 System keeps the file unreachable, since it is not publicly addressable.
	<i>The use case ends.</i>		
	EXCEPTION FLOW		
	E2 (Edition Cannot Be Rendered)		
	<i>Branches from step 2.3 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Subscriber opens an edition the browser cannot render.	1.1 System states that the edition could not be opened.
			1.2 System offers the download instead.
			1.3 System records the failure for diagnosis.
	<i>The use case ends.</i>		
	EXCEPTION FLOW		
	E-AUTH (Authorization or Session Failure)		
	<i>Branches from step 2 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Subscriber selects to read an edition after the session has expired, or without a role permitted to see it.	1.1 System checks the session and the role before showing anything.
			1.2 System shows nothing the role does not permit.
			1.3 System redirects to sign-in where the session has expired, or returns the user to the dashboard for their own role.
	2	Subscriber signs in again.	2.1 System restores the session and returns to the page.
	<i>Where the session had expired, resumes at step 2 of the basic flow; where the role was not permitted, the use case ends.</i>		

Use Case Name: Manage a Subscription

Use Case ID:	UC-4.4
Author	Joshua Concepcion

Purpose	Enable a subscriber to see what they are paying for, change how they pay, or stop paying.																															
Related Use Case	None																															
Priority	Medium																															
Preconditions	• The system is available • The reader is authenticated and holds a subscription • A subscription exists, obtained under UC-4.3																															
Postconditions	• The subscription reflects the reader's instruction • Access continues to the end of any period already paid for • No card detail is held by this system																															
Actors	• Subscriber • Payment Gateway																															
Flow of Actions	BASIC FLOW <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Subscriber opens their subscription details.</td><td>1.1 System presents the plan, the renewal date and the payment method as the Payment Gateway describes it.</td></tr> <tr> <td>2</td><td>Subscriber selects to cancel.</td><td>2.1 System states the date to which access will continue.</td></tr> <tr> <td>3</td><td>Subscriber confirms.</td><td>3.1 System sends the cancellation to the Payment Gateway.</td></tr> <tr> <td>4</td><td>Payment Gateway confirms the cancellation takes effect at the end of the paid period.</td><td>4.1 System records the date access ends.</td></tr> <tr> <td></td><td></td><td>4.2 System returns the Reader to the free tier at that date.</td></tr> </table> ALTERNATE FLOW A1 (Payment Method Updated) <i>Branches from step 2 of the basic flow.</i> <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Subscriber selects to update their payment method.</td><td>1.1 System directs them to the Payment Gateway's interface.</td></tr> <tr> <td>2</td><td>Subscriber updates the method with the Payment Gateway and returns.</td><td>2.1 System shows the updated method as the Payment Gateway describes it.</td></tr> </table> <i>The use case ends.</i> ALTERNATE FLOW A2 (Cancellation Reversed) <i>Branches after step 4 of the basic flow, before the paid period ends.</i> <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> </table>		Step	Actor Action	System Response	1	Subscriber opens their subscription details.	1.1 System presents the plan, the renewal date and the payment method as the Payment Gateway describes it.	2	Subscriber selects to cancel.	2.1 System states the date to which access will continue.	3	Subscriber confirms.	3.1 System sends the cancellation to the Payment Gateway.	4	Payment Gateway confirms the cancellation takes effect at the end of the paid period.	4.1 System records the date access ends.			4.2 System returns the Reader to the free tier at that date.	Step	Actor Action	System Response	1	Subscriber selects to update their payment method.	1.1 System directs them to the Payment Gateway's interface.	2	Subscriber updates the method with the Payment Gateway and returns.	2.1 System shows the updated method as the Payment Gateway describes it.	Step	Actor Action	System Response
Step	Actor Action	System Response																														
1	Subscriber opens their subscription details.	1.1 System presents the plan, the renewal date and the payment method as the Payment Gateway describes it.																														
2	Subscriber selects to cancel.	2.1 System states the date to which access will continue.																														
3	Subscriber confirms.	3.1 System sends the cancellation to the Payment Gateway.																														
4	Payment Gateway confirms the cancellation takes effect at the end of the paid period.	4.1 System records the date access ends.																														
		4.2 System returns the Reader to the free tier at that date.																														
Step	Actor Action	System Response																														
1	Subscriber selects to update their payment method.	1.1 System directs them to the Payment Gateway's interface.																														
2	Subscriber updates the method with the Payment Gateway and returns.	2.1 System shows the updated method as the Payment Gateway describes it.																														
Step	Actor Action	System Response																														

1	Subscriber selects to resume the subscription.	1.1 System reinstates the subscription with the Payment Gateway.
		1.2 System records the reinstatement, so no gap in access occurs.

The use case ends.

EXCEPTION FLOW

E-SAVE (Persistence Failure)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	Subscriber confirms the cancellation.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
2	Subscriber retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.

Resumes at step 3 of the basic flow.

EXCEPTION FLOW

E-AUTH (Authorization or Session Failure)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	Subscriber confirms the cancellation after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
		1.2 System blocks the change without partly saving it.
		1.3 System redirects to sign-in where the session has expired, or states that access is denied.
2	Subscriber signs in again.	2.1 System restores the session and returns to the page.

Where the session had expired, resumes at step 3 of the basic flow; where the role or ownership was missing, the use case ends.

Accounts and Platform Administration UC-5

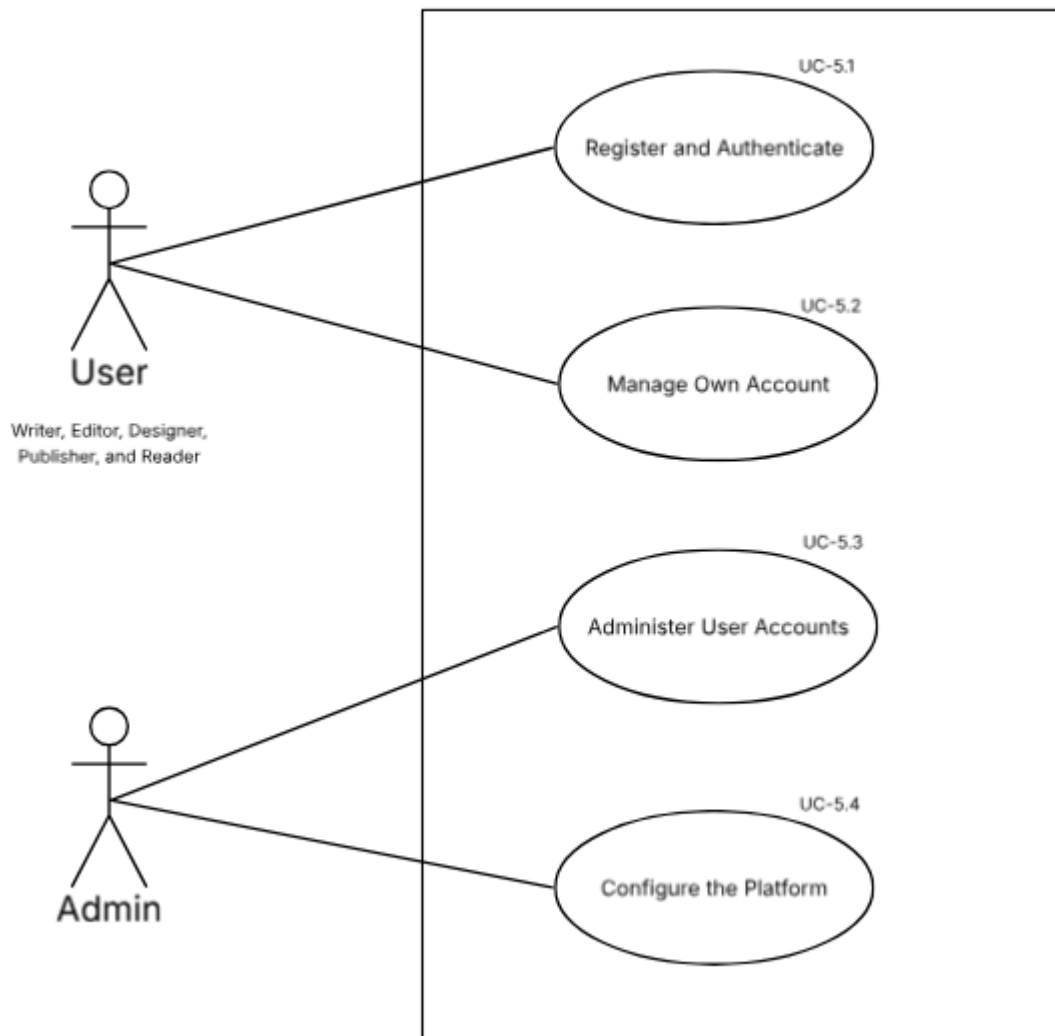


Figure 7 Accounts Administration

Use Case Name: Register and Authenticate

Use Case ID:	UC-5.1
Author	Rei Khalil S. Galido
Purpose	Enable a person to obtain an account and to prove who they are on each return.
Related Use Case	None
Priority	High
Preconditions	The system is available
Postconditions	An account exists, or a session is established and scoped to the account's role

	• The actor is directed to the interface appropriate to their role • Consent to the terms is recorded at registration																																							
Actors	• User (Reader, Writer, Editor, Publisher or Graphic Designer)																																							
Flow of Actions	BASIC FLOW <table><thead><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr></thead><tbody><tr><td>1</td><td>User opens the registration page and enters their name, email address and a password.</td><td>1.1 System indicates the strength of the password as it is entered.</td></tr><tr><td>2</td><td>User consents to the terms and submits.</td><td>2.1 System validates the email format, the password strength and the uniqueness of the address.</td></tr><tr><td></td><td></td><td>2.2 System creates the account, records the consent and establishes a session.</td></tr><tr><td>3</td><td>On a later visit, the User enters their email address and password.</td><td>3.1 System validates the credentials and confirms the account is active.</td></tr><tr><td></td><td></td><td>3.2 System issues an access credential and a renewal credential, and directs the User to the interface for their role.</td></tr></tbody></table> ALTERNATE FLOW A1 (Session Renewed Transparently) Branches at any point after step 3 of the basic flow, while the User is working. <table><thead><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr></thead><tbody><tr><td>1</td><td>User continues working after the access credential has expired.</td><td>1.1 System exchanges the renewal credential for a fresh access credential.</td></tr><tr><td></td><td></td><td>1.2 System retries the interrupted request, so no work is lost.</td></tr><tr><td></td><td></td><td>1.3 Where the renewal credential has also expired, System ends the session and says so plainly.</td></tr></tbody></table> The use case ends. ALTERNATE FLOW A2 (Staff Account at the Reader Sign-in) Branches from step 3.1 of the basic flow. <table><thead><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr></thead><tbody><tr><td>1</td><td>Staff member signs in through the reader portal.</td><td>1.1 System authenticates the credentials.</td></tr><tr><td></td><td></td><td>1.2 System ends the session and directs the staff member to the staff portal, since the two portals serve different audiences.</td></tr></tbody></table> The use case ends.	Step	Actor Action	System Response	1	User opens the registration page and enters their name, email address and a password.	1.1 System indicates the strength of the password as it is entered.	2	User consents to the terms and submits.	2.1 System validates the email format, the password strength and the uniqueness of the address.			2.2 System creates the account, records the consent and establishes a session.	3	On a later visit, the User enters their email address and password.	3.1 System validates the credentials and confirms the account is active.			3.2 System issues an access credential and a renewal credential, and directs the User to the interface for their role.	Step	Actor Action	System Response	1	User continues working after the access credential has expired.	1.1 System exchanges the renewal credential for a fresh access credential.			1.2 System retries the interrupted request, so no work is lost.			1.3 Where the renewal credential has also expired, System ends the session and says so plainly.	Step	Actor Action	System Response	1	Staff member signs in through the reader portal.	1.1 System authenticates the credentials.			1.2 System ends the session and directs the staff member to the staff portal, since the two portals serve different audiences.
Step	Actor Action	System Response																																						
1	User opens the registration page and enters their name, email address and a password.	1.1 System indicates the strength of the password as it is entered.																																						
2	User consents to the terms and submits.	2.1 System validates the email format, the password strength and the uniqueness of the address.																																						
		2.2 System creates the account, records the consent and establishes a session.																																						
3	On a later visit, the User enters their email address and password.	3.1 System validates the credentials and confirms the account is active.																																						
		3.2 System issues an access credential and a renewal credential, and directs the User to the interface for their role.																																						
Step	Actor Action	System Response																																						
1	User continues working after the access credential has expired.	1.1 System exchanges the renewal credential for a fresh access credential.																																						
		1.2 System retries the interrupted request, so no work is lost.																																						
		1.3 Where the renewal credential has also expired, System ends the session and says so plainly.																																						
Step	Actor Action	System Response																																						
1	Staff member signs in through the reader portal.	1.1 System authenticates the credentials.																																						
		1.2 System ends the session and directs the staff member to the staff portal, since the two portals serve different audiences.																																						

ALTERNATE FLOW**A3 (Address Already Registered)**

Branches from step 2.1 of the basic flow.

Step	Actor Action	System Response
1	User registers with an address that already belongs to an account.	1.1 System refuses the registration.
		1.2 System directs the User to sign in instead.
		1.3 System discloses nothing beyond what the User already supplied.

The use case ends.

ALTERNATE FLOW**A4 (Reader Account at the Staff Sign-in)**

Branches from step 3.1 of the basic flow.

Step	Actor Action	System Response
1	Reader signs in through the staff portal.	1.1 System authenticates the credentials.
		1.2 System ends the session and states that readers sign in on the public site.

The use case ends.

EXCEPTION FLOW**E1 (Credentials Not Recognised)**

Branches from step 3.1 of the basic flow.

Step	Actor Action	System Response
1	User enters an incorrect address or password.	1.1 System finds no match and refuses the sign-in.
		1.2 System does not state which field was wrong, since doing so would confirm which addresses hold accounts.
2	User enters the credentials again.	2.1 System validates the credentials.

Resumes at step 3.1 of the basic flow.

EXCEPTION FLOW**E2 (Account Suspended)**

Branches from step 3.1 of the basic flow.

Step	Actor Action	System Response
1	User signs in with correct credentials to an account that has been suspended.	1.1 System refuses the sign-in and establishes no session.
		1.2 System states that the account is not currently active and directs the User to an administrator.

	<i>The use case ends.</i>	
	EXCEPTION FLOW	
	E3 (Too Many Attempts)	
	<i>Branches from step 3.1 of the basic flow.</i>	
	Step	Actor Action
	1	User, or anyone else, makes a sixth sign-in attempt on the same account within a minute.
		1.1 System refuses the attempt without checking the password, even if it is correct.
		1.2 System states that there have been too many attempts and asks the User to wait.
		1.3 System leaves other accounts unaffected.
	2	User waits a minute and signs in again.
		2.1 System validates the credentials.
	<i>Resumes at step 3.1 of the basic flow.</i>	
	EXCEPTION FLOW	
	E-SAVE (Persistence Failure)	
	<i>Branches from step 2 of the basic flow.</i>	
	Step	Actor Action
	1	User submits the registration.
		1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.
	2	User retries once the service has recovered.
		2.1 System keeps the input on screen, so nothing needs to be entered again.
	<i>Resumes at step 2 of the basic flow.</i>	

Use Case Name: Manage Own Account

Use Case ID:	UC-5.2
Author	Roman Rico Albania
Purpose	Enable a person to keep their own details current, control their password, and choose what the system tells them about.
Related Use Case	None
Priority	Medium
Preconditions	The system is available

	The actor is authenticated, except where recovering a forgotten password																																				
Postconditions	- The account reflects the actor's changes - A changed password is stored as a hash and other sessions are ended - Notification preferences govern subsequent notifications - Security and account notices continue to be delivered regardless of preference																																				
Actors	User (Reader, Writer, Editor, Publisher or Graphic Designer)																																				
Flow of Actions	BASIC FLOW <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>User opens their account settings.</td><td>1.1 System presents the current name and sign-in address.</td></tr><tr><td>2</td><td>User amends their details.</td><td>2.1 Where the sign-in address has changed, System asks for the current password.</td></tr><tr><td>3</td><td>User supplies the password and saves.</td><td>3.1 System verifies the password and the uniqueness of the new address.</td></tr><tr><td></td><td></td><td>3.2 System stores the changes and confirms them.</td></tr></table> ALTERNATE FLOW A1 (Name Change Only) <i>Branches from step 2 of the basic flow.</i> <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>User amends their name without changing the sign-in address, and saves.</td><td>1.1 System requires no password.</td></tr><tr><td></td><td></td><td>1.2 System stores the change and confirms it.</td></tr></table> <i>The use case ends.</i> ALTERNATE FLOW A2 (Password Changed While Signed In) <i>Branches from step 2 of the basic flow.</i> <table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>User selects to change their password and enters the current and the new password.</td><td>1.1 System verifies the current password and validates the strength of the new one.</td></tr><tr><td></td><td></td><td>1.2 System stores the new password and keeps the current session.</td></tr><tr><td></td><td></td><td>1.3 System ends the account's other sessions.</td></tr></table> <i>The use case ends.</i>	Step	Actor Action	System Response	1	User opens their account settings.	1.1 System presents the current name and sign-in address.	2	User amends their details.	2.1 Where the sign-in address has changed, System asks for the current password.	3	User supplies the password and saves.	3.1 System verifies the password and the uniqueness of the new address.			3.2 System stores the changes and confirms them.	Step	Actor Action	System Response	1	User amends their name without changing the sign-in address, and saves.	1.1 System requires no password.			1.2 System stores the change and confirms it.	Step	Actor Action	System Response	1	User selects to change their password and enters the current and the new password.	1.1 System verifies the current password and validates the strength of the new one.			1.2 System stores the new password and keeps the current session.			1.3 System ends the account's other sessions.
Step	Actor Action	System Response																																			
1	User opens their account settings.	1.1 System presents the current name and sign-in address.																																			
2	User amends their details.	2.1 Where the sign-in address has changed, System asks for the current password.																																			
3	User supplies the password and saves.	3.1 System verifies the password and the uniqueness of the new address.																																			
		3.2 System stores the changes and confirms them.																																			
Step	Actor Action	System Response																																			
1	User amends their name without changing the sign-in address, and saves.	1.1 System requires no password.																																			
		1.2 System stores the change and confirms it.																																			
Step	Actor Action	System Response																																			
1	User selects to change their password and enters the current and the new password.	1.1 System verifies the current password and validates the strength of the new one.																																			
		1.2 System stores the new password and keeps the current session.																																			
		1.3 System ends the account's other sessions.																																			

ALTERNATE FLOW**A3 (Password Forgotten)**

Branches from the sign-in page, before the basic flow begins.

Step	Actor Action	System Response
1	User requests password recovery for their address.	1.1 System sends a single-use, time-limited recovery link to the address.
2	User follows the link and enters a new password.	2.1 System validates the link's authenticity and expiry, then the new password's strength.
		2.2 System stores the new password and ends existing sessions.

The use case ends.

ALTERNATE FLOW**A4 (Recovery for an Unregistered Address)**

Branches from step 1 of alternate flow A3.

Step	Actor Action	System Response
1	User requests recovery for an address that belongs to no account.	1.1 System displays the same confirmation as for a registered address.
		1.2 System sends no recovery message, so the response does not reveal whether an account exists.

The use case ends.

ALTERNATE FLOW**A5 (Notifications Adjusted)**

Branches from step 2 of the basic flow.

Step	Actor Action	System Response
1	User mutes a kind of notification they no longer want, and saves.	1.1 System records the preference and applies it to later notifications.
		1.2 System keeps the underlying events visible on the dashboard, so nothing is lost by muting.
		1.3 System continues to deliver security and account notices regardless.

The use case ends.

EXCEPTION FLOW**E1 (Password Not Correct)**

Branches from step 3.1 of the basic flow.

Step	Actor Action	System Response
1	User supplies an incorrect current password for a protected change.	1.1 System verifies the password before committing anything.

		1.2 System refuses the change and leaves the account unaltered.
2	User enters the correct password and saves again.	2.1 System verifies the password.

Resumes at step 3.1 of the basic flow.

EXCEPTION FLOW

E2 (Address Already in Use)

Branches from step 3.1 of the basic flow.

Step	Actor Action	System Response
1	User saves a new sign-in address that already belongs to another account.	1.1 System detects the conflict before committing anything.
		1.2 System refuses the change and states that the address is unavailable, without disclosing whose account holds it.
2	User enters a different address and saves again.	2.1 System verifies the new address.

Resumes at step 3.1 of the basic flow.

EXCEPTION FLOW

E3 (Recovery Link Expired or Used)

Branches from step 2 of alternate flow A3.

Step	Actor Action	System Response
1	User follows a recovery link after it has expired or been used.	1.1 System finds the link invalid and refuses the change.
		1.2 System keeps the existing password in force.
		1.3 System offers to send a fresh link.

Resumes at step 1 of alternate flow A3.

EXCEPTION FLOW

E-SAVE (Persistence Failure)

Branches from step 3 of the basic flow.

Step	Actor Action	System Response
1	User saves the changes.	1.1 System validates the request.
		1.2 System attempts to save the change, and the save fails or times out.
		1.3 System keeps the last confirmed state and does not report success.
		1.4 System records the failure and displays a message inviting a retry.

	2	User retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.														
	Resumes at step 3 of the basic flow.																
	EXCEPTION FLOW																
	E-AUTH (Authorization or Session Failure)																
	Branches from step 3 of the basic flow.																
	<table><tr><th>Step</th><th>Actor Action</th><th>System Response</th></tr><tr><td>1</td><td>User saves the changes after the session has expired, or without the role or ownership the action requires.</td><td>1.1 System checks the session, and the role or ownership the action requires, before changing anything.</td></tr><tr><td></td><td></td><td>1.2 System blocks the change without partly saving it.</td></tr><tr><td></td><td></td><td>1.3 System redirects to sign-in where the session has expired, or states that access is denied.</td></tr><tr><td>2</td><td>User signs in again.</td><td>2.1 System restores the session and returns to the page.</td></tr></table>			Step	Actor Action	System Response	1	User saves the changes after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.			1.2 System blocks the change without partly saving it.			1.3 System redirects to sign-in where the session has expired, or states that access is denied.	2	User signs in again.
Step	Actor Action	System Response															
1	User saves the changes after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.															
		1.2 System blocks the change without partly saving it.															
		1.3 System redirects to sign-in where the session has expired, or states that access is denied.															
2	User signs in again.	2.1 System restores the session and returns to the page.															
Where the session had expired, resumes at step 3 of the basic flow; where the role or ownership was missing, the use case ends.																	

Use Case Name: Administer User Accounts

Use Case ID:	UC-5.3							
Author	Joshmar Clavero							
Purpose	Enable an Administrator to control who has access to the system and what each person may do.							
Related Use Case	None							
Priority	High							
Preconditions	- The system is available - The Administrator is authenticated with administration access							
Postconditions	- The account reflects the change - A role change takes effect on the account's next request - A suspended account is refused regardless of the role it holds - The account's articles, versions and history are retained in every case							
Actors	Administrator							
Flow of Actions	BASIC FLOW <table> <tr> <th>Step</th><th>Actor Action</th><th>System Response</th></tr> <tr> <td>1</td><td>Administrator opens the user directory.</td><td>1.1 System lists the accounts with their role, status and last sign-in.</td></tr> </table>		Step	Actor Action	System Response	1	Administrator opens the user directory.	1.1 System lists the accounts with their role, status and last sign-in.
Step	Actor Action	System Response						
1	Administrator opens the user directory.	1.1 System lists the accounts with their role, status and last sign-in.						

2	Administrator searches or filters to find an account.	
3	Administrator changes the account's role, suspends it, or grants a subscription tier.	3.1 System validates the change against the roles and tiers it recognises.
		3.2 System stores the change and applies it on the account's next request.
		3.3 System retains the account's articles, versions and history in every case.

ALTERNATE FLOW**A1 (Account Holds Outstanding Work)***Branches from step 3 of the basic flow.*

Step	Actor Action	System Response
1	Administrator changes the role of an account that holds articles commissioned under its previous role.	1.1 System stores the role change.
		1.2 System does not reassign or withdraw that work.
2	Administrator reassigns any outstanding work the change makes unreachable to its holder.	2.1 System transfers each assignment and notifies the Writers involved.

*The use case ends.***ALTERNATE FLOW****A2 (Subscription Granted Directly)***Branches from step 3 of the basic flow.*

Step	Actor Action	System Response
1	Administrator sets a reader's tier to subscriber without payment.	1.1 System records the tier and the start date.
		1.2 System grants access on exactly the same terms as a purchased subscription, since the entitlement checks do not distinguish between the two routes.

*The use case ends.***EXCEPTION FLOW****E-SAVE (Persistence Failure)***Branches from step 3 of the basic flow.*

Step	Actor Action	System Response
1	Administrator applies the change to the account.	1.1 System validates the request.

			1.2 System attempts to save the change, and the save fails or times out.
			1.3 System keeps the last confirmed state and does not report success.
			1.4 System records the failure and displays a message inviting a retry.
	2	Administrator retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.
	<i>Resumes at step 3 of the basic flow.</i>		
	EXCEPTION FLOW		
	<i>E-AUTH (Authorization or Session Failure)</i>		
	<i>Branches from step 3 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Administrator applies the change to the account after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
			1.2 System blocks the change without partly saving it.
			1.3 System redirects to sign-in where the session has expired, or states that access is denied.
	2	Administrator signs in again.	2.1 System restores the session and returns to the page.
<i>Where the session had expired, resumes at step 3 of the basic flow; where the role or ownership was missing, the use case ends.</i>			

Use Case Name: Configure the Platform

Use Case ID:	UC-5.4
Author	Joshua Concepcion
Purpose	Enable an Administrator to set how the publication presents itself, to see whether its external services are working, and to hold public traffic during an update.
Related Use Case	None
Priority	Medium
Preconditions	- The system is available - The Administrator is authenticated in the Administrator role
Postconditions	- The configuration is stored and takes effect without redeployment - The updating Administrator is recorded - Where maintenance is enabled, public traffic receives the notice and staff continue working

Actors	• Administrator		
Flow of Actions	BASIC FLOW		
	Step	Actor Action	System Response
	1	Administrator opens platform settings.	1.1 System presents the current configuration and the status of each external service.
	2	Administrator amends the publication name, tagline, accent colour or page size.	2.1 System validates the values, including the colour format.
			2.2 System stores the configuration, records the Administrator, and applies it without redeployment.
	3	Administrator enables maintenance and sets the notice before an update.	3.1 System returns the notice to unauthenticated public requests while continuing to serve staff.
	4	Administrator disables maintenance once the update is complete.	
	ALTERNATE FLOW		
	A1 (Service Not Configured)		
	<i>Branches from step 1.1 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Administrator opens platform settings while an external service holds no credential.	1.1 System reports the service as unconfigured.
			1.2 System states what the platform does instead in its absence.
			1.3 System displays no credential, since the interface does not hold them.
	<i>Resumes at step 2 of the basic flow.</i>		
	ALTERNATE FLOW		
	A-VALID (Incomplete or Invalid Input)		
	<i>Branches from step 2 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Administrator saves the configuration with a required item missing or malformed.	1.1 System validates every required item before saving anything.
			1.2 System marks each problem beside the field it concerns, with guidance.
			1.3 System saves nothing, and the workflow state is unchanged.

	2	Administrator corrects the items and submits again.	2.1 System validates the input again.
	<i>Resumes at step 2.2 of the basic flow.</i>		
	EXCEPTION FLOW		
	E-SAVE (Persistence Failure)		
	<i>Branches from step 2 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Administrator saves the configuration.	1.1 System validates the request.
			1.2 System attempts to save the change, and the save fails or times out.
			1.3 System keeps the last confirmed state and does not report success.
			1.4 System records the failure and displays a message inviting a retry.
	2	Administrator retries once the service has recovered.	2.1 System keeps the input on screen, so nothing needs to be entered again.
	<i>Resumes at step 2 of the basic flow.</i>		
	EXCEPTION FLOW		
	E-AUTH (Authorization or Session Failure)		
	<i>Branches from step 2 of the basic flow.</i>		
	Step	Actor Action	System Response
	1	Administrator saves the configuration after the session has expired, or without the role or ownership the action requires.	1.1 System checks the session, and the role or ownership the action requires, before changing anything.
			1.2 System blocks the change without partly saving it.
			1.3 System redirects to sign-in where the session has expired, or states that access is denied.
	2	Administrator signs in again.	2.1 System restores the session and returns to the page.
	<i>Where the session had expired, resumes at step 2 of the basic flow; where the role or ownership was missing, the use case ends.</i>		

B. Activity Diagrams with Swim-lanes

EDITORIAL PRODUCTION

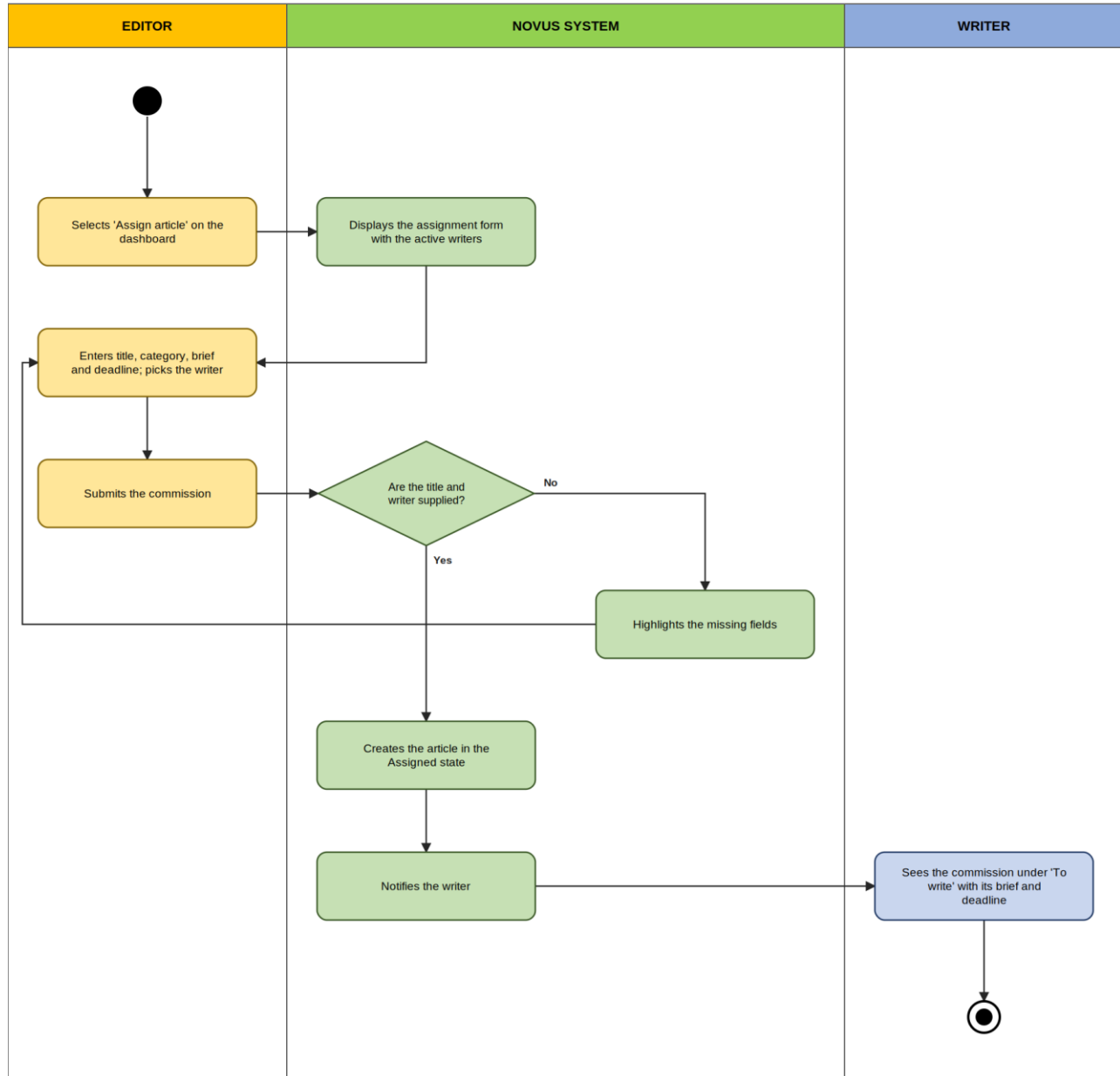


Figure 8 UC-1.1: Commission an article

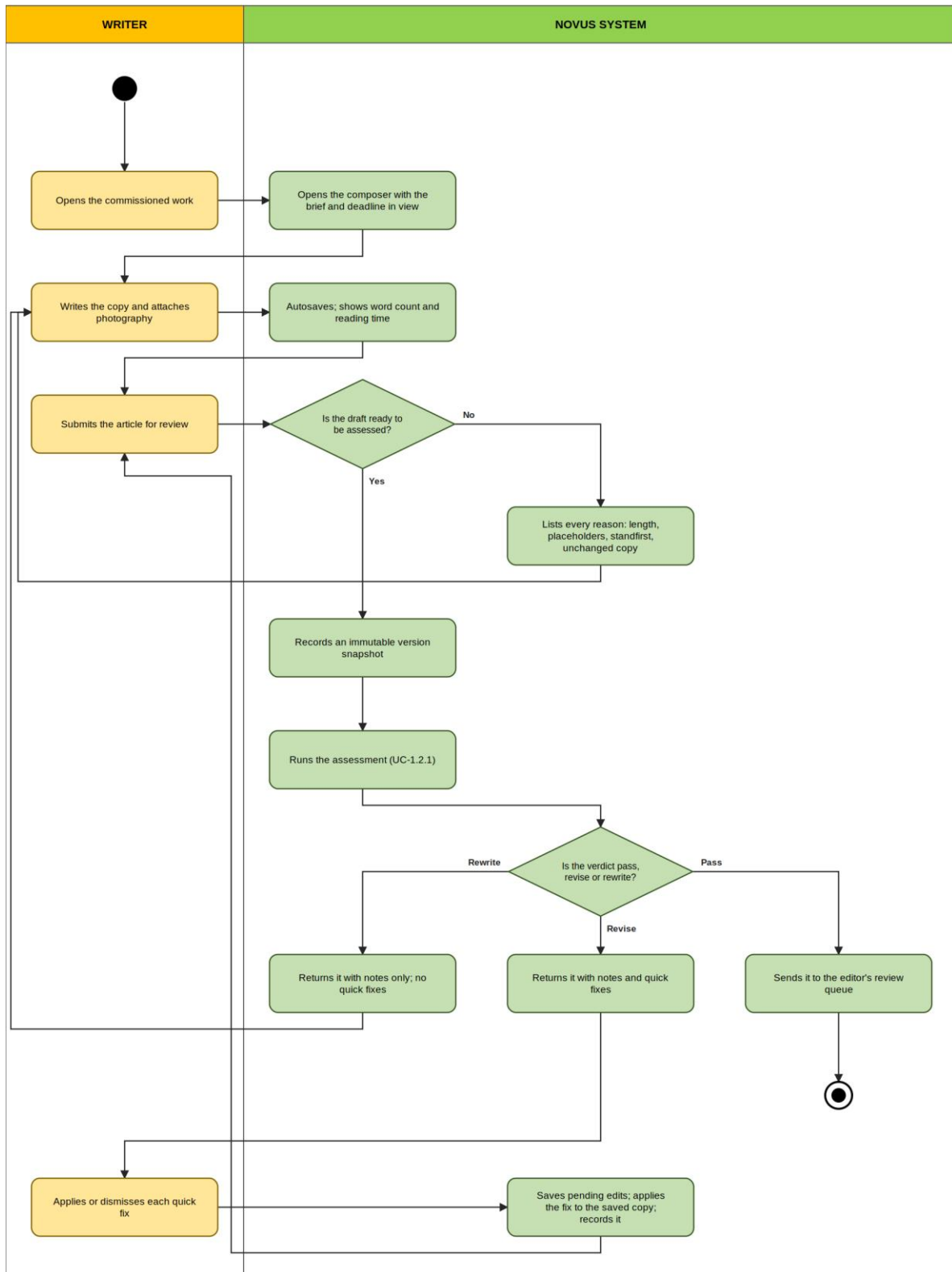


Figure 9 UC-1.2: Produce an Article

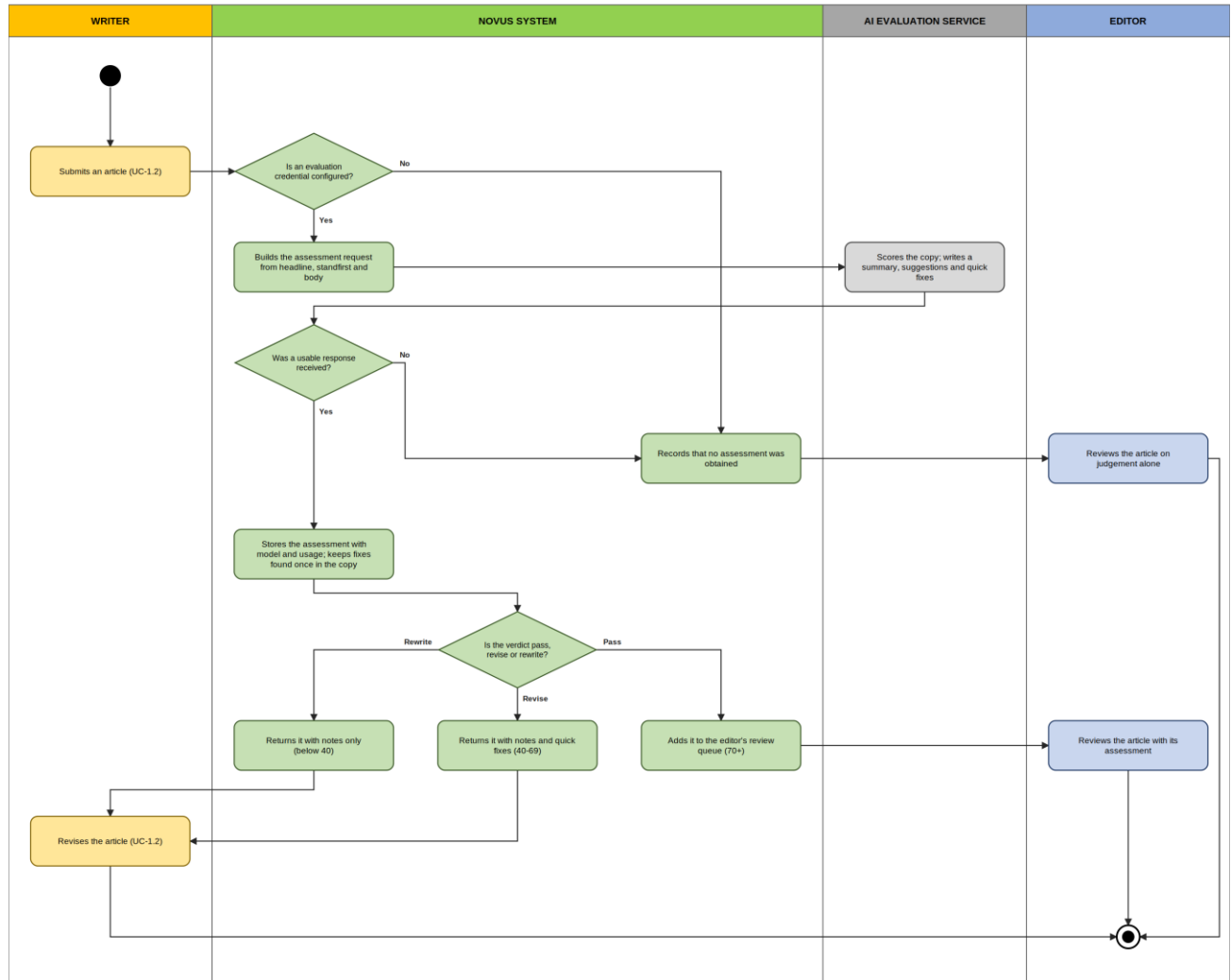


Figure 10 UC-1.2.1: Evaluate a Submitted Article

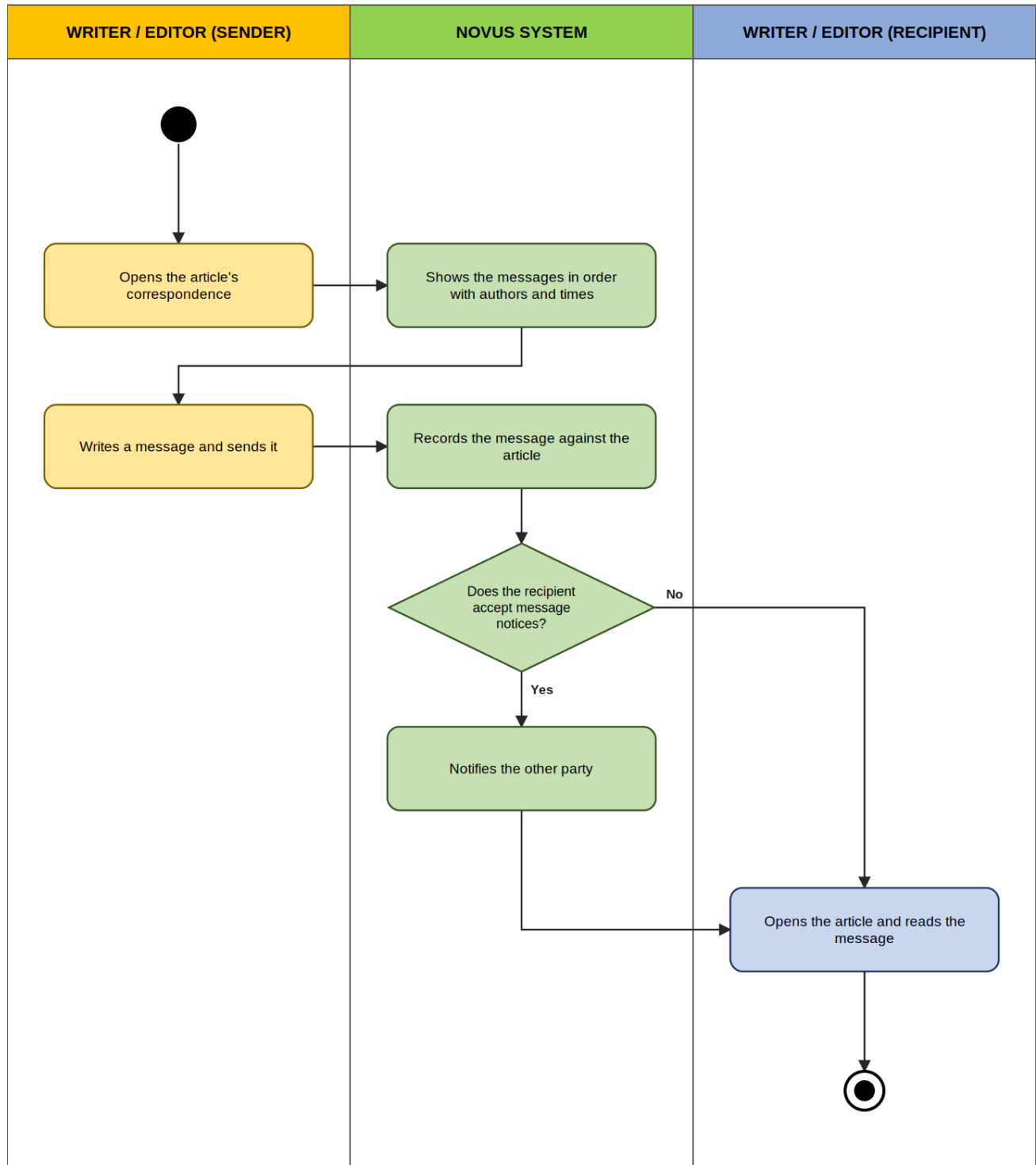


Figure 11 UC-1.3: Communicate About an Article

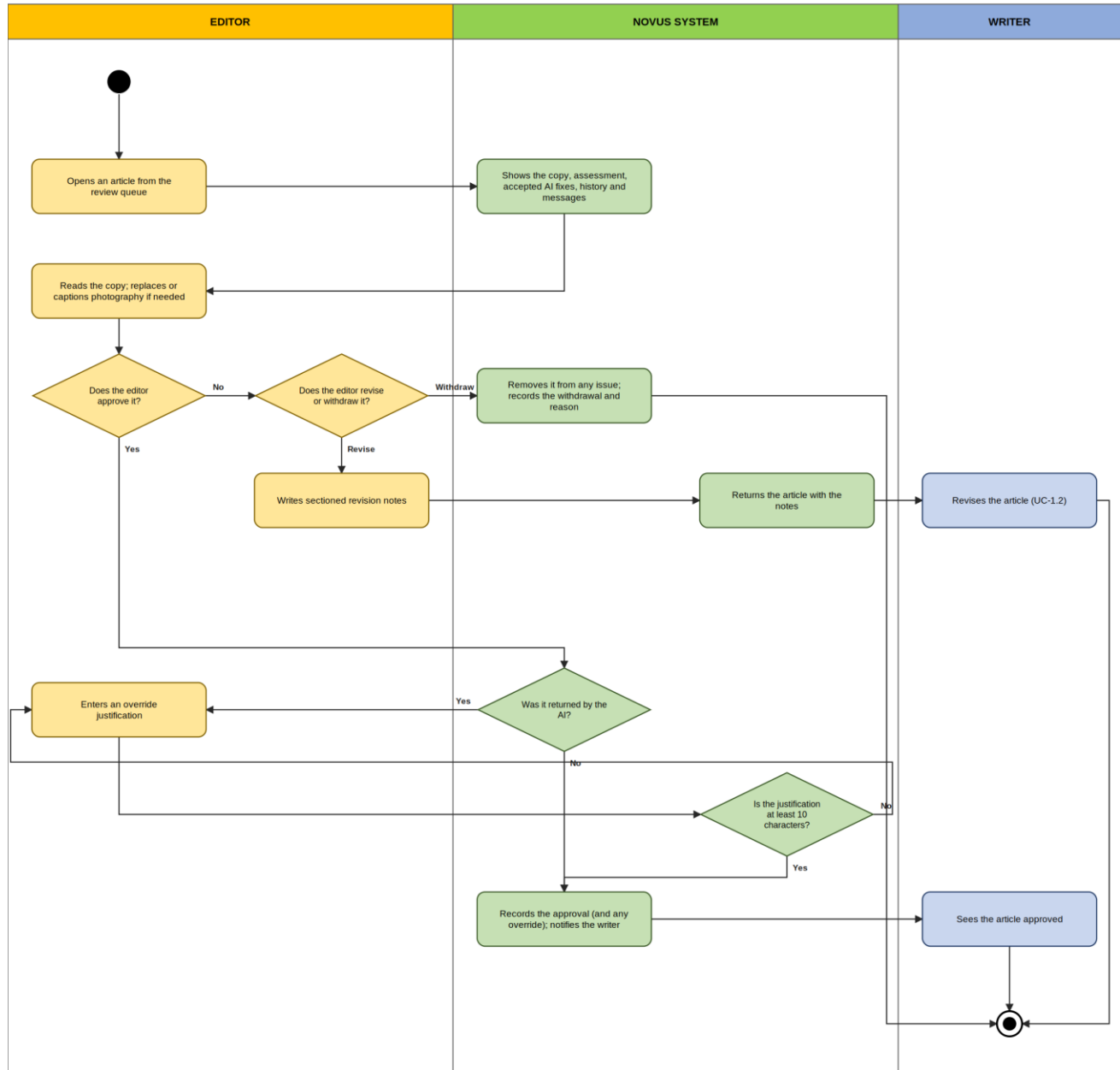


Figure 12 UC-1.3.1: Decide on a Submitted Article

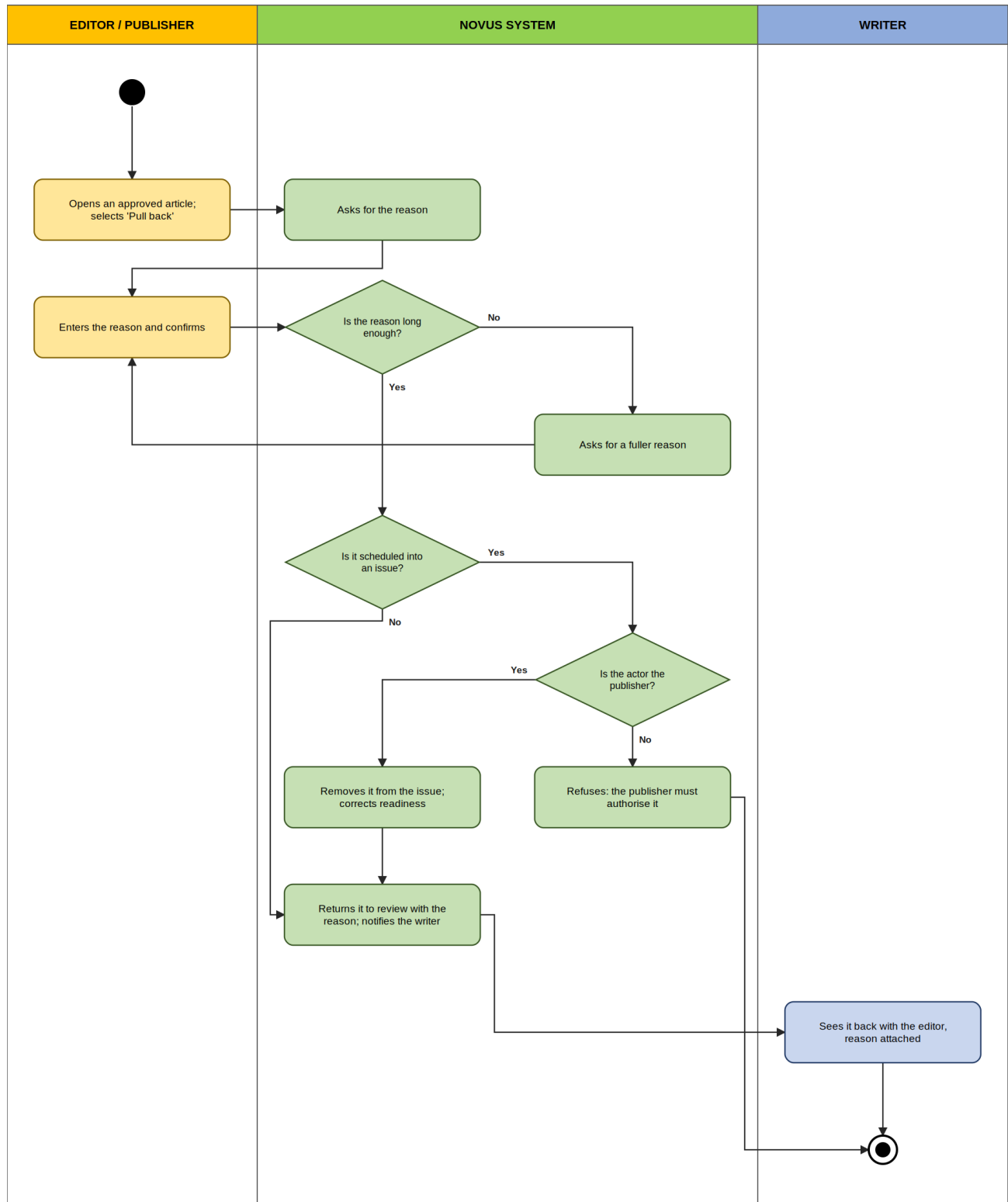


Figure 13 UC-1.4: Correct an Editorial Decision

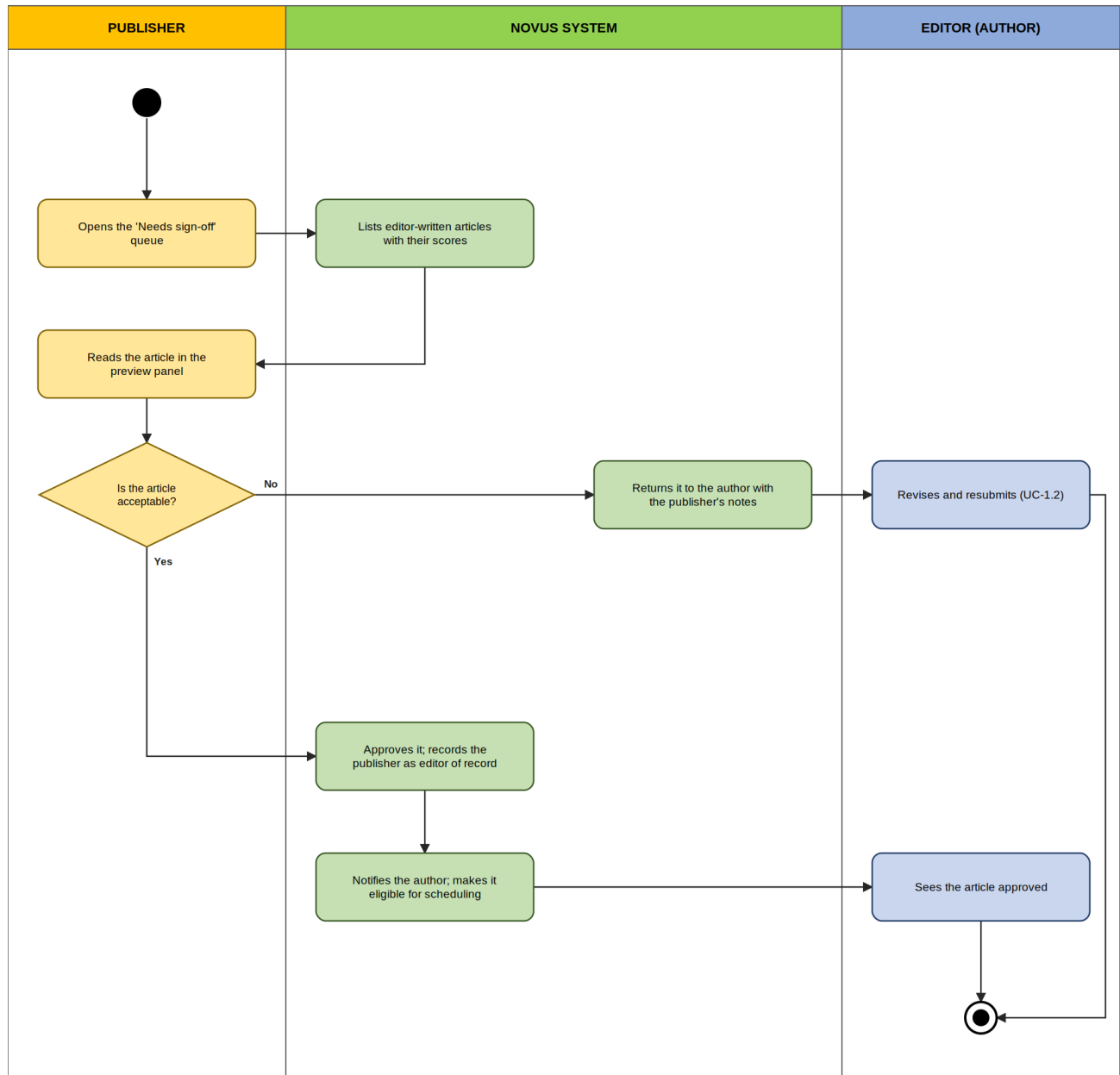


Figure 14 UC-1.5: Sign Off Editor-Authored Copy

ISSUE PRODUCTION AND PUBLICATION

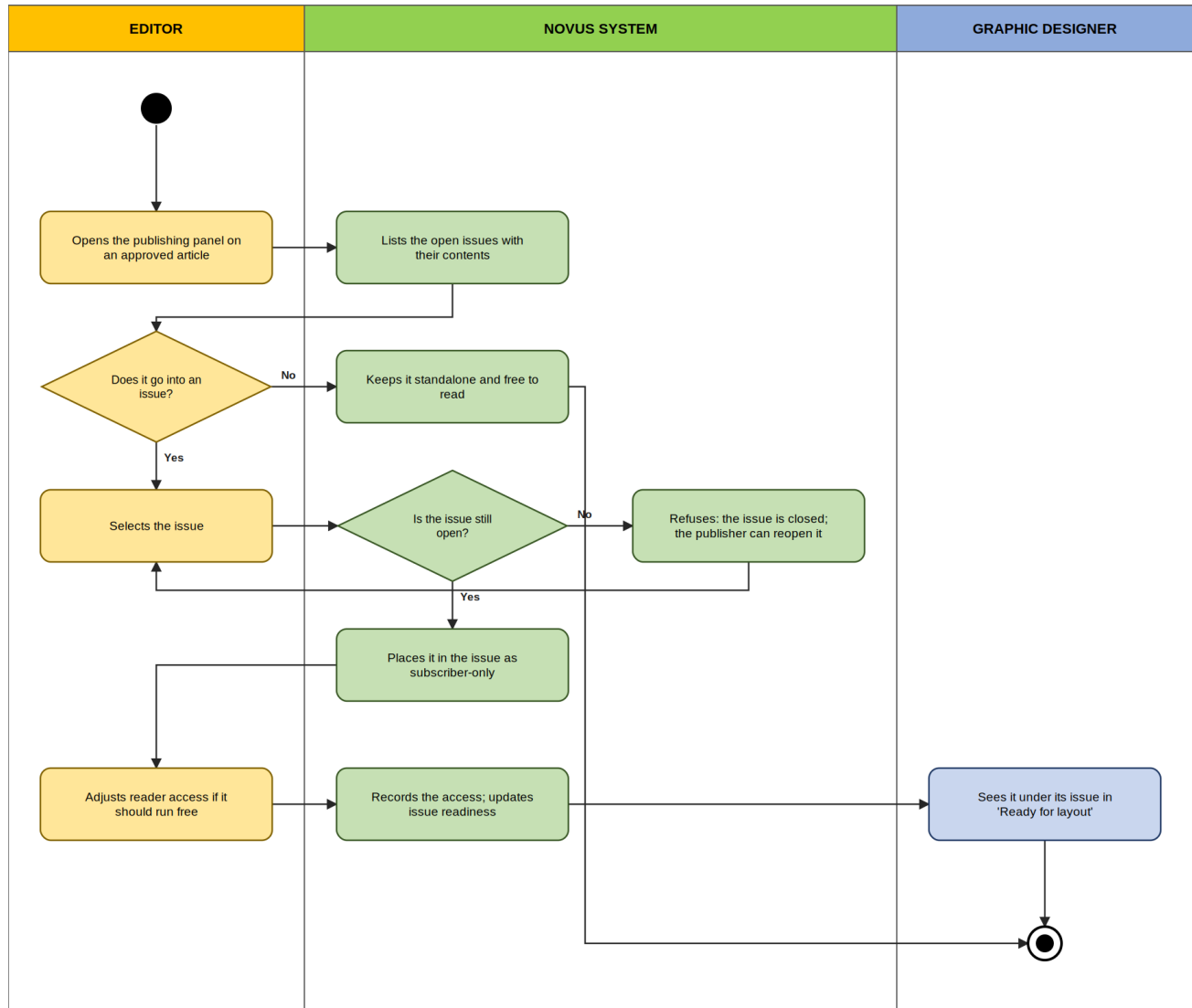


Figure 15 UC-2.1: Schedule Articles into an Issue

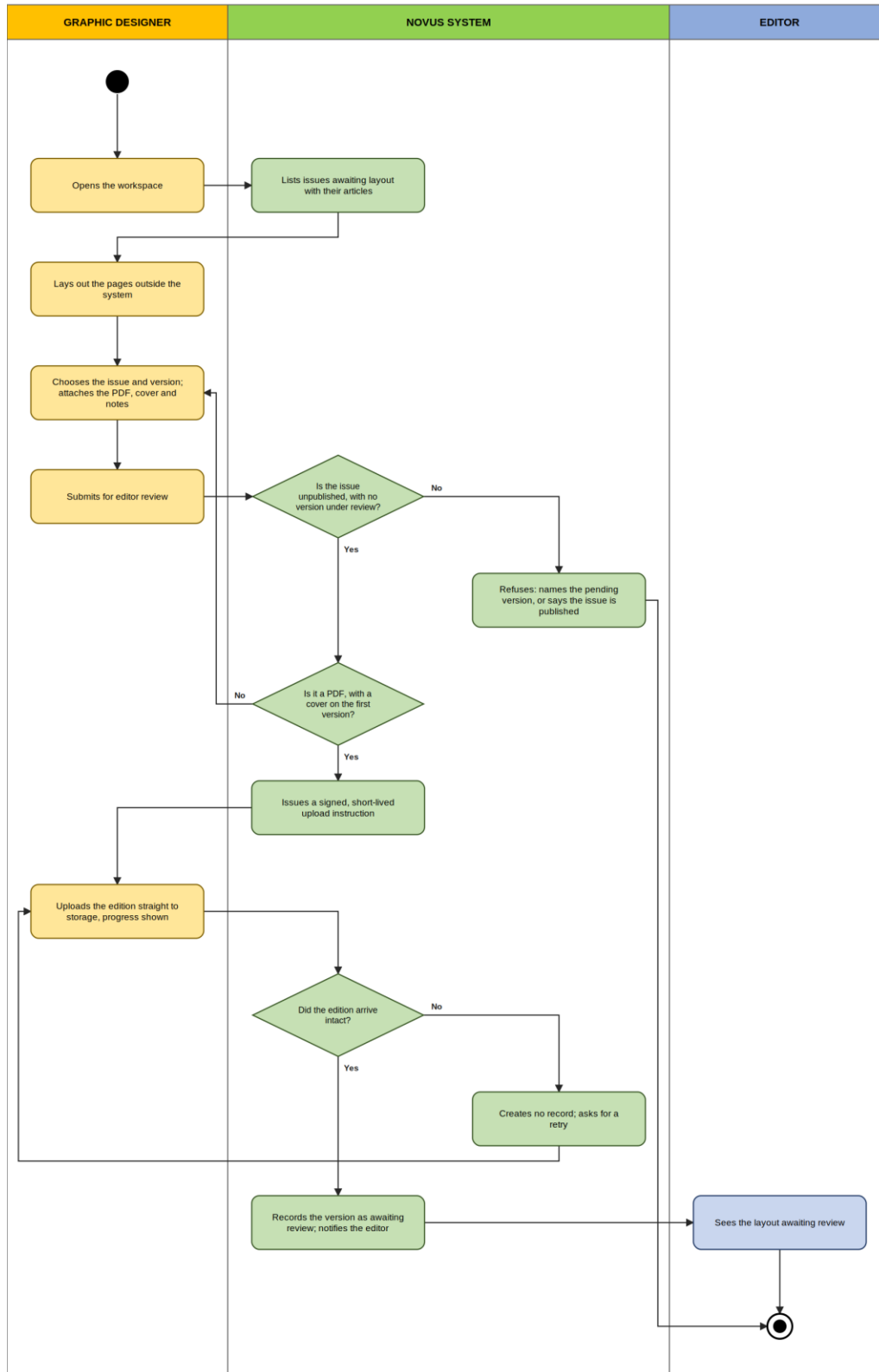


Figure 16 UC-2.2: Produce a Magazine Layout

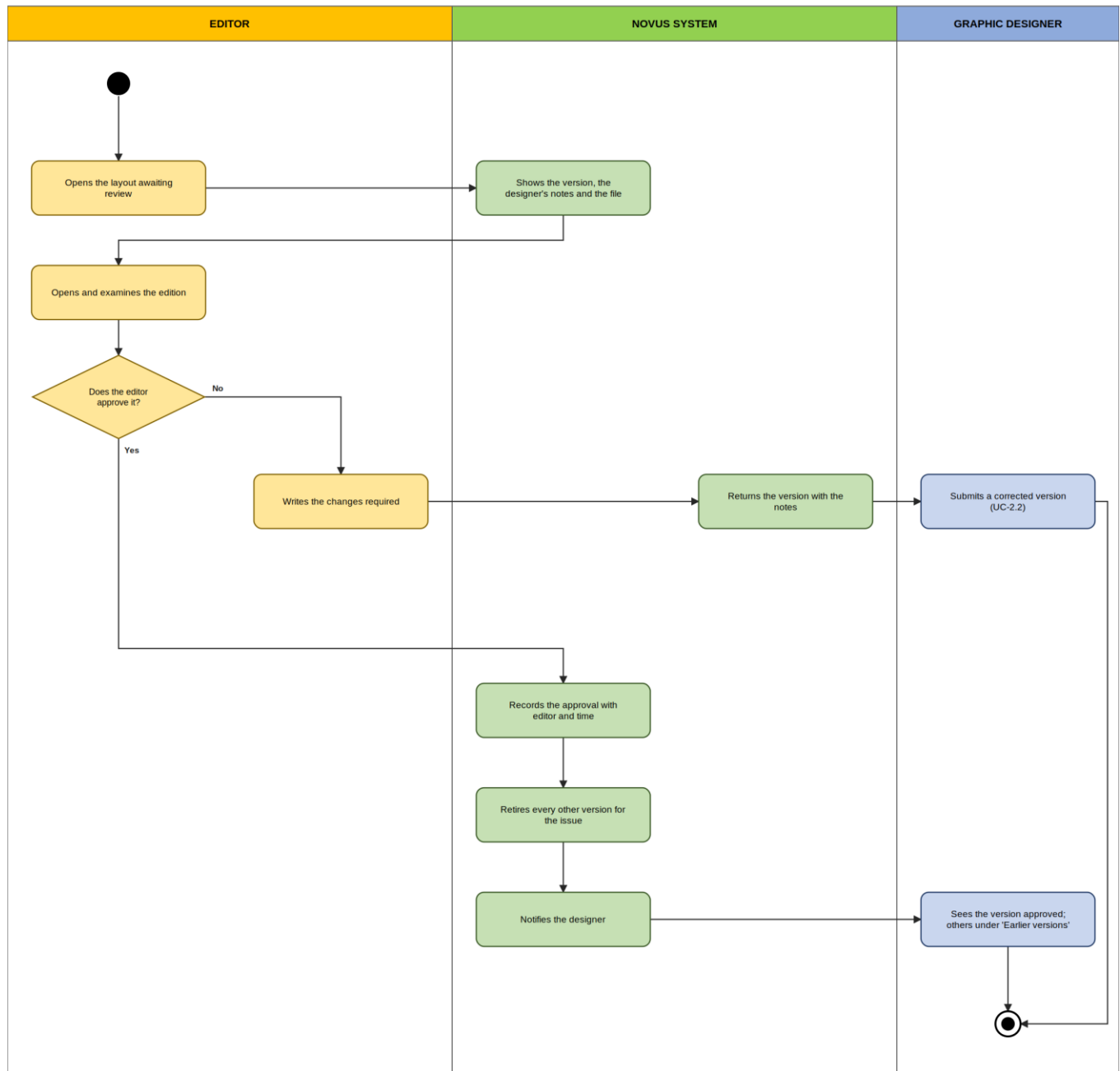


Figure 17 UC-2.3: Approve a Magazine Layout

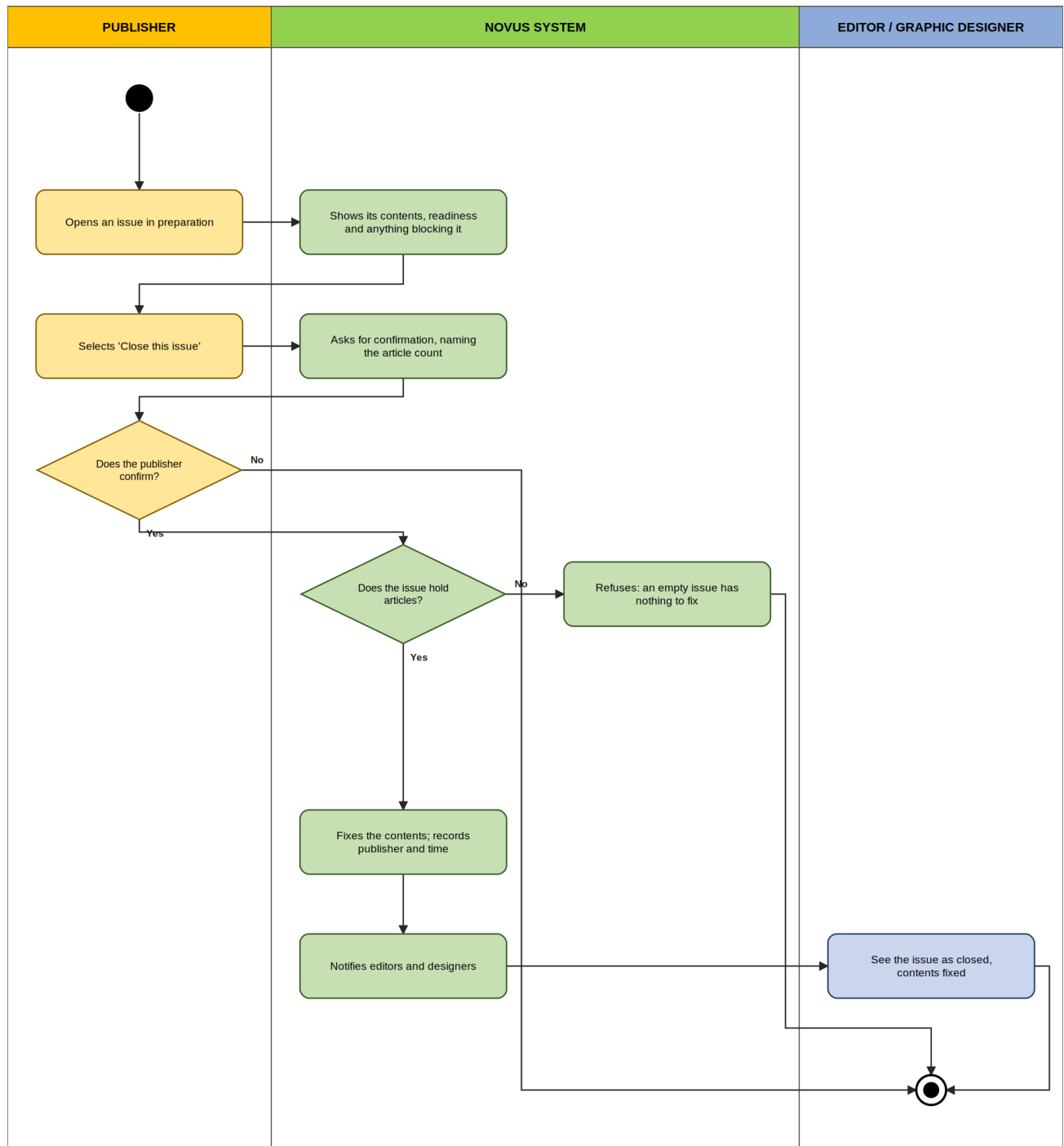


Figure 18 UC-2.4: Close an Issue

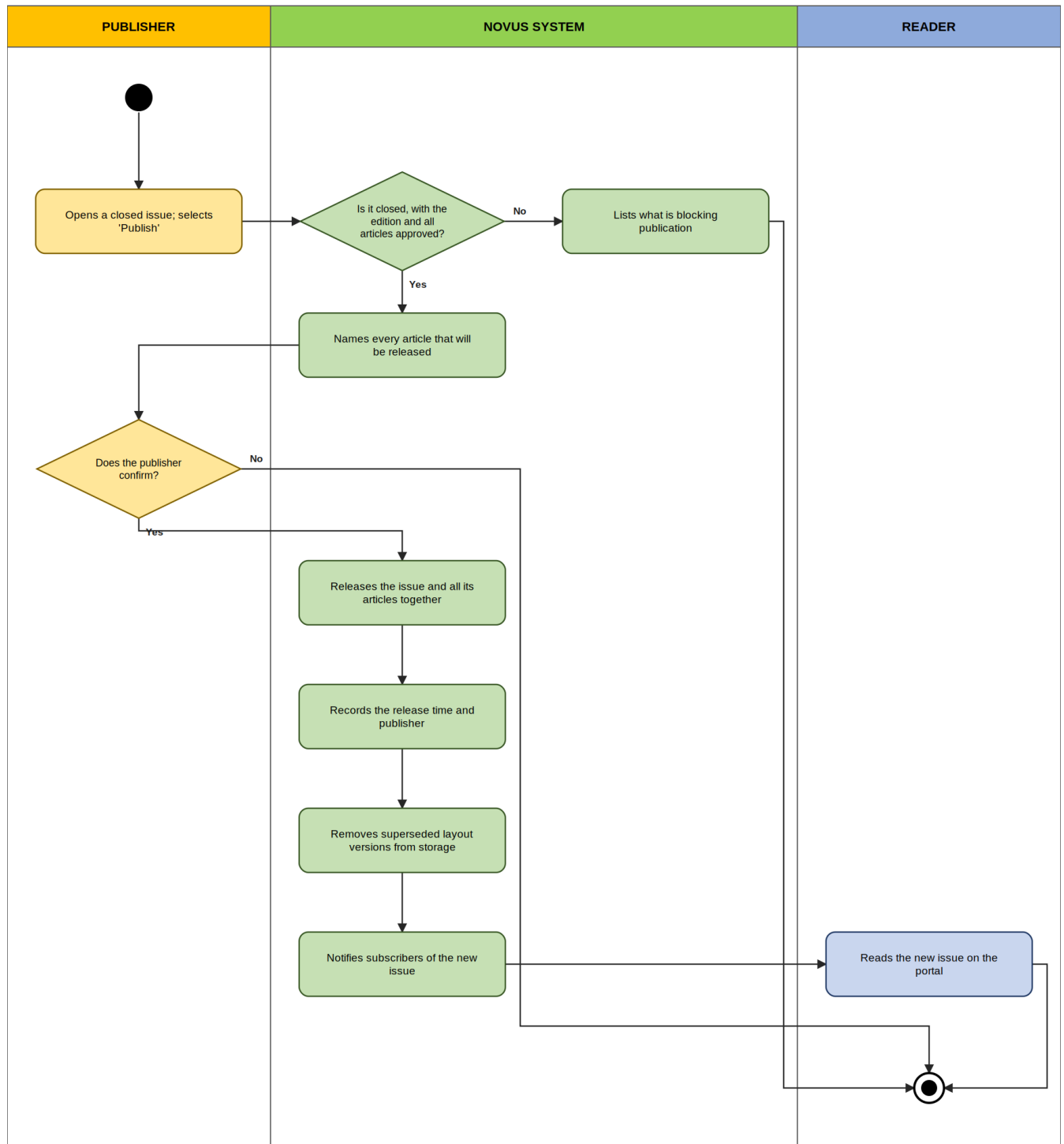


Figure 19 UC-2.5: Publish an Issue

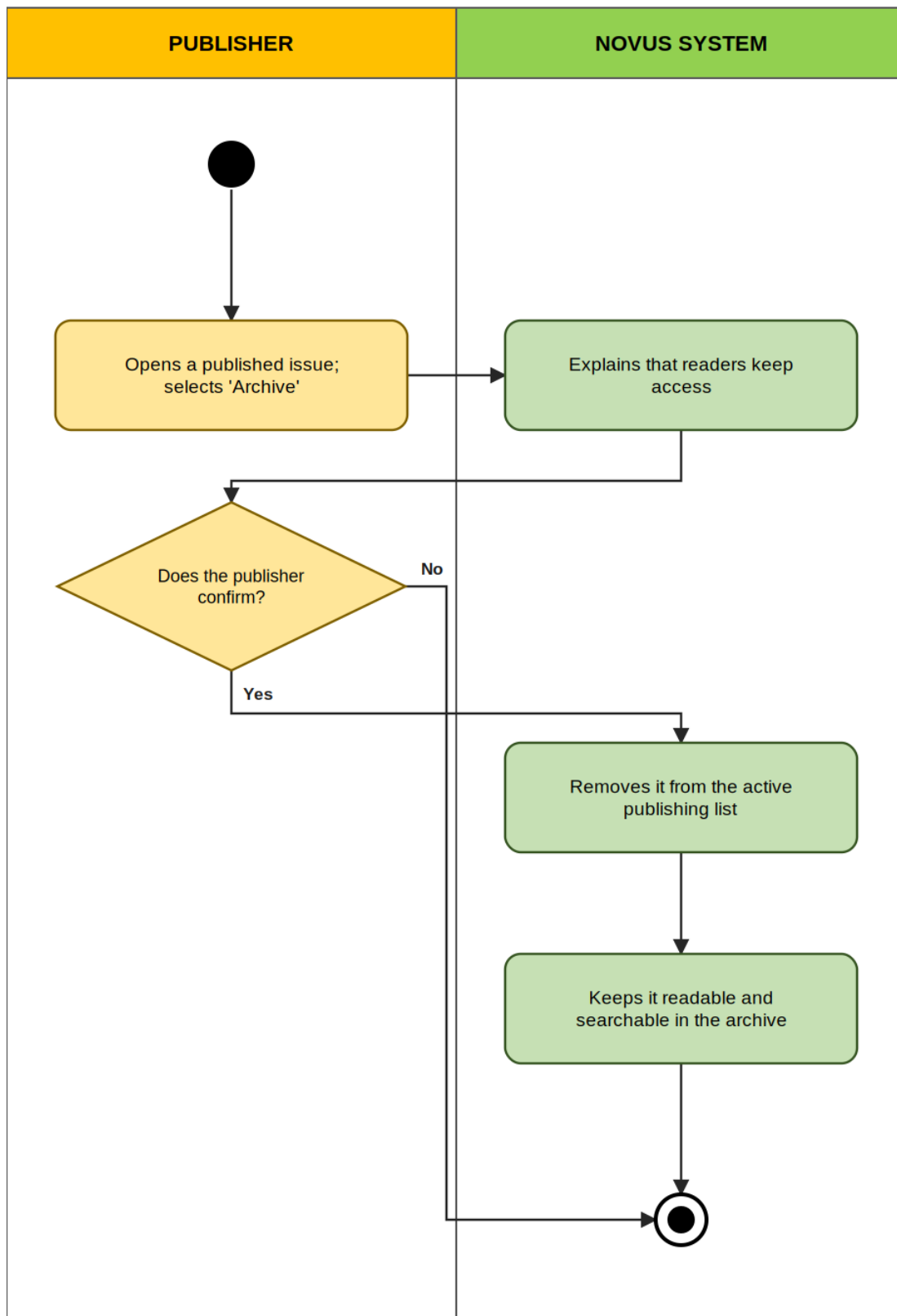


Figure 20 UC-2.6: Retire an Issue

EDITORIAL AND ISSUE OVERSIGHT

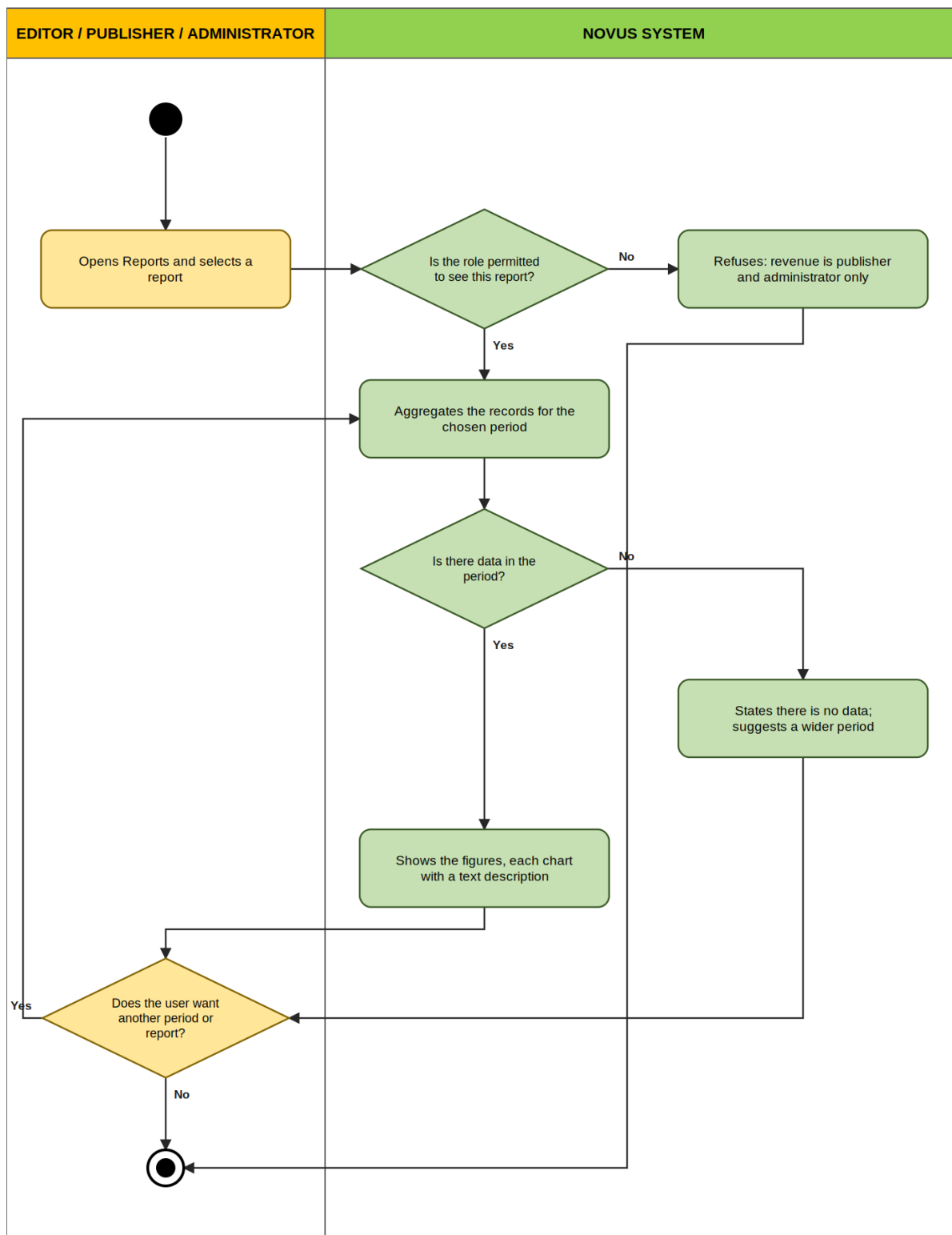


Figure 21 UC-3.1: Review Publication Performance

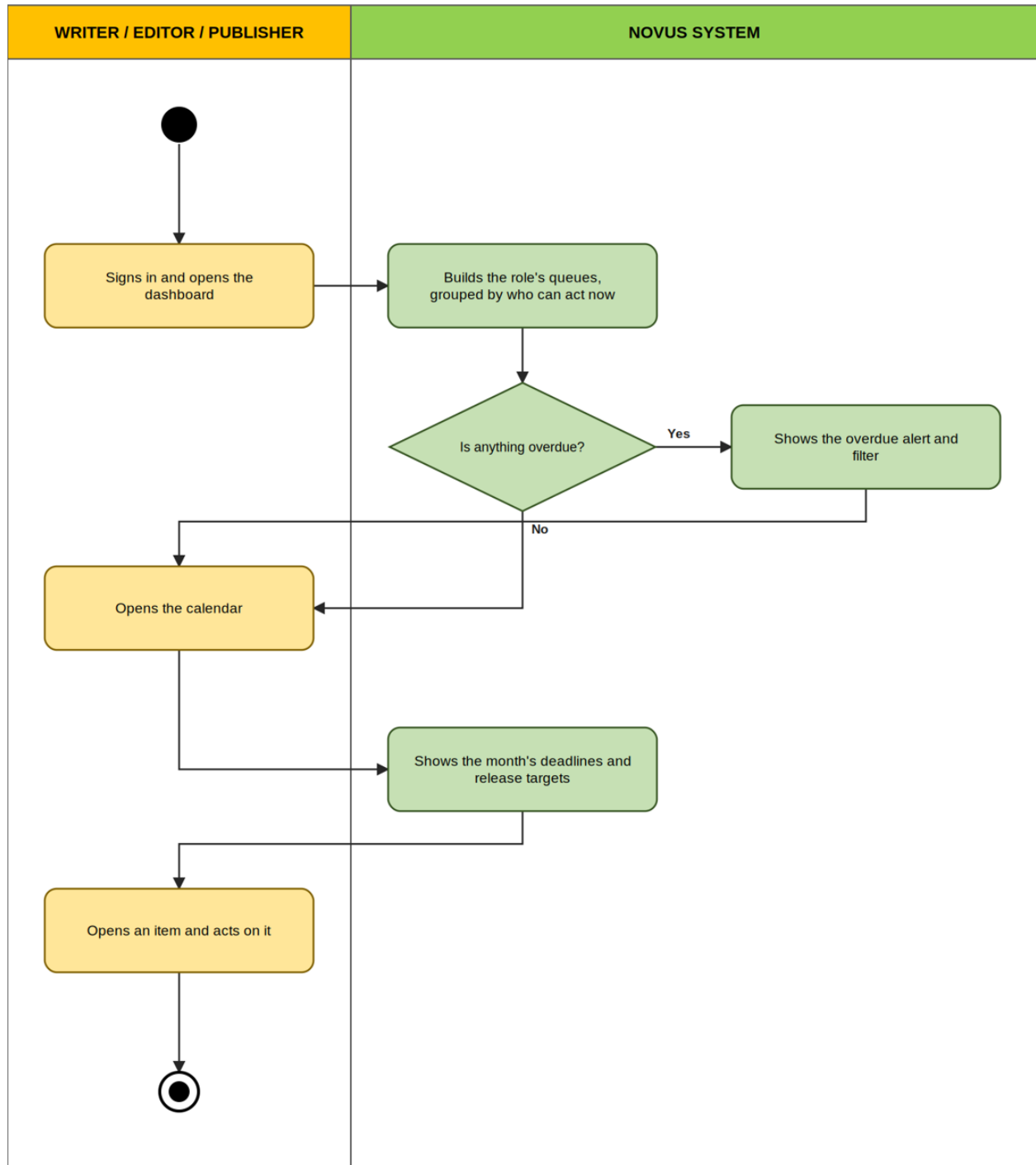


Figure 22 UC-3.2: Monitor Editorial Progress

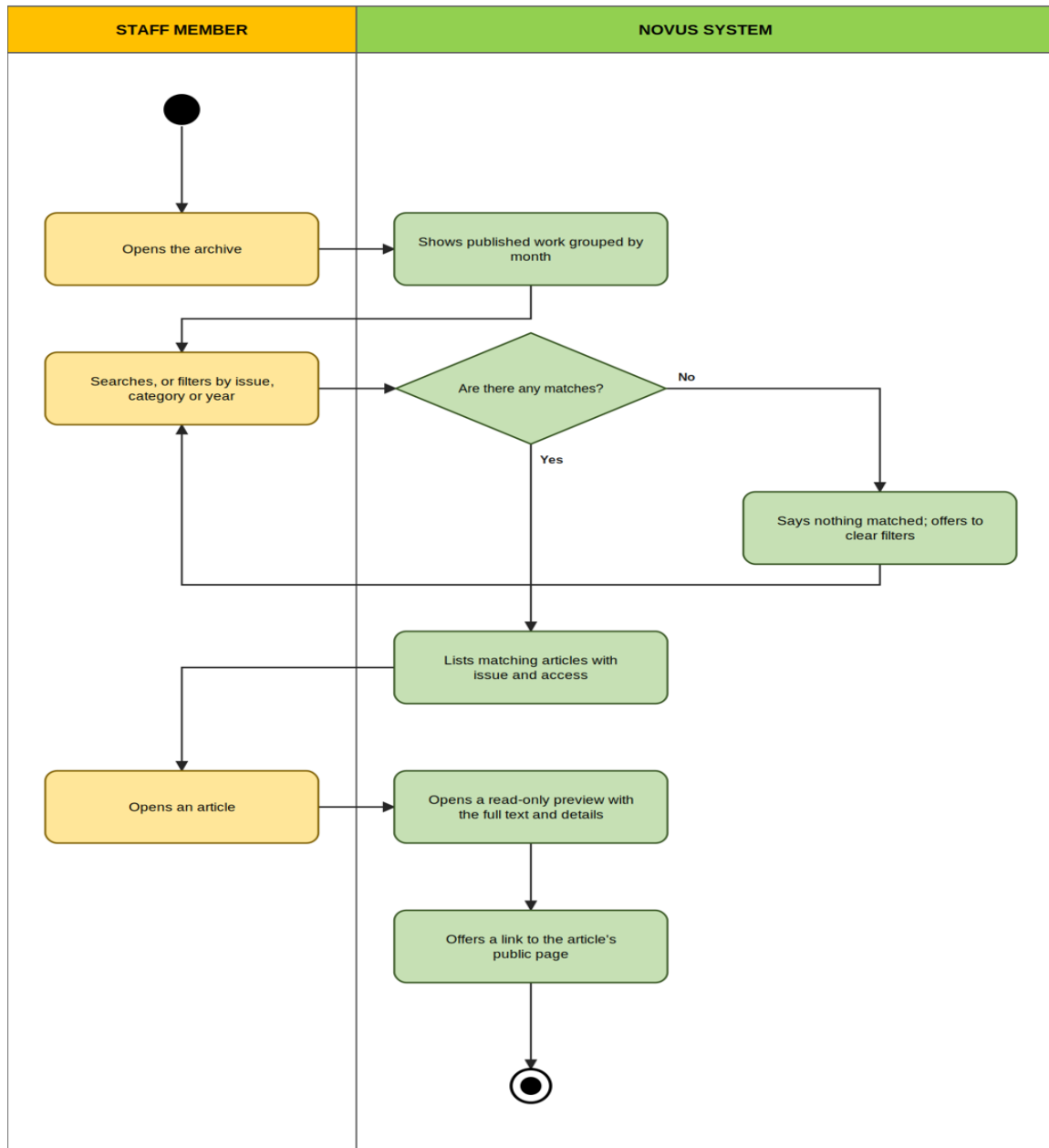


Figure 23 UC-3.3: Consult the Published Archive

READER ENGAGEMENT

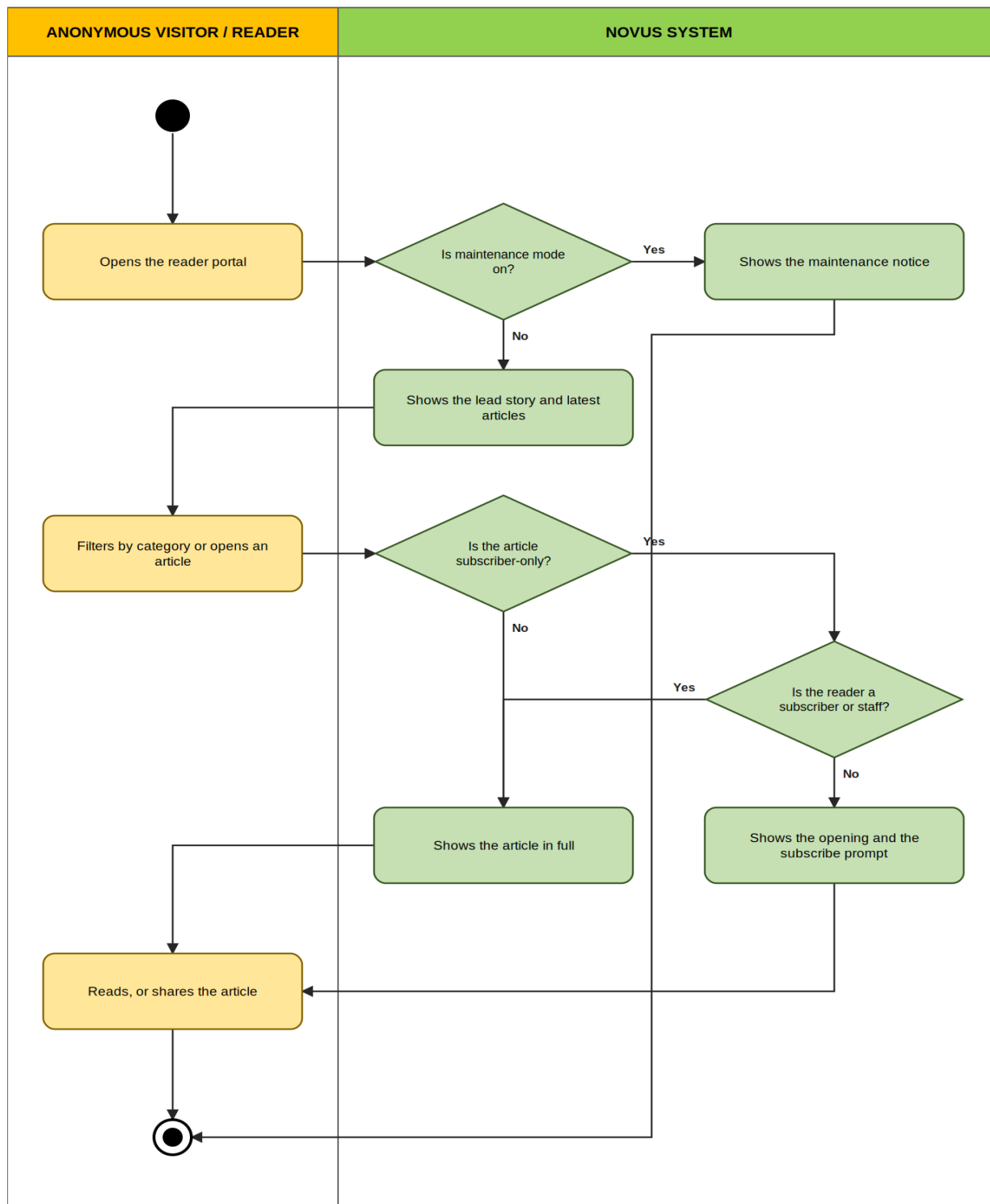


Figure 24 UC-4.1: Read Published Content

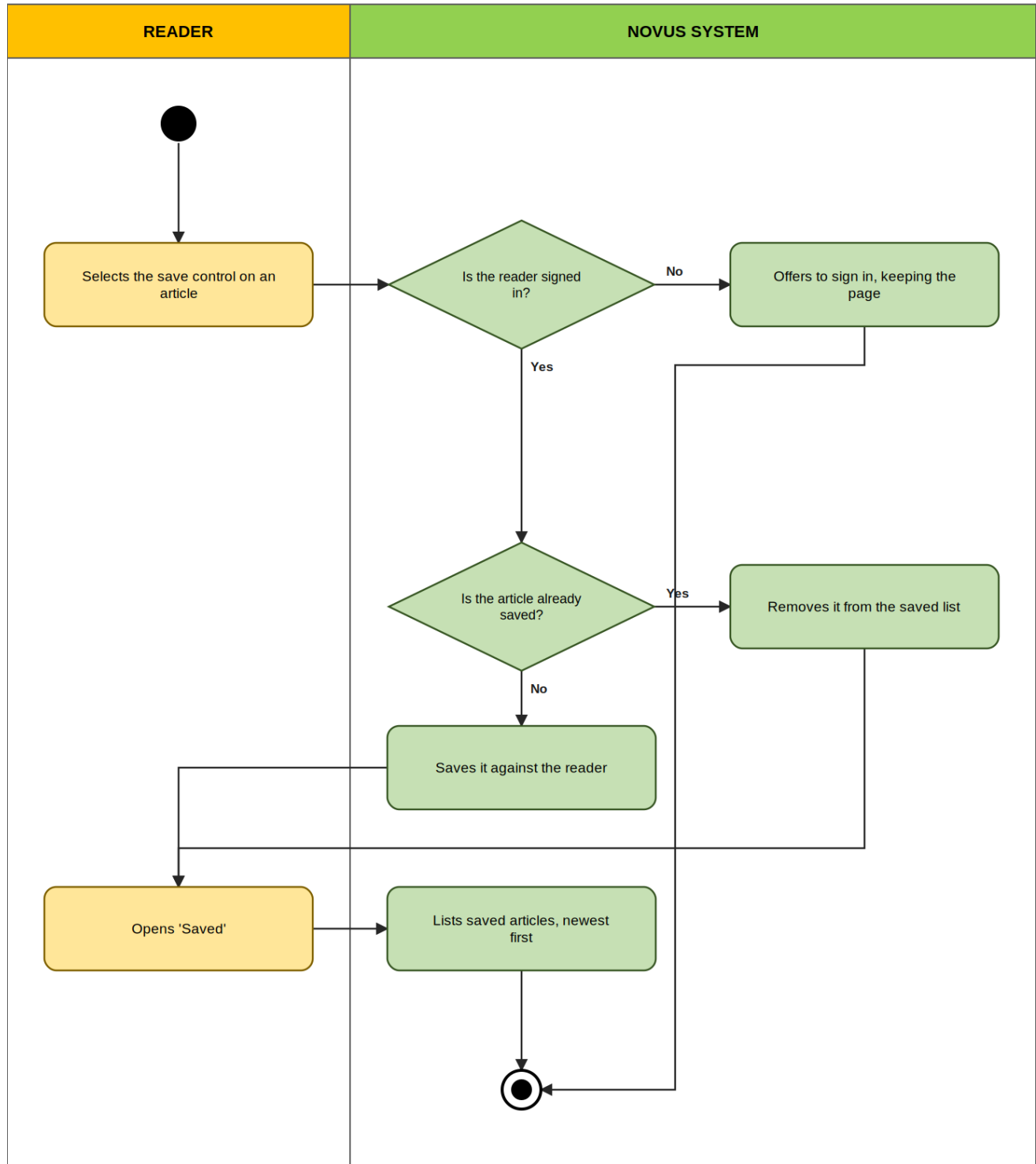


Figure 25 UC-4.2: Save Articles for Later

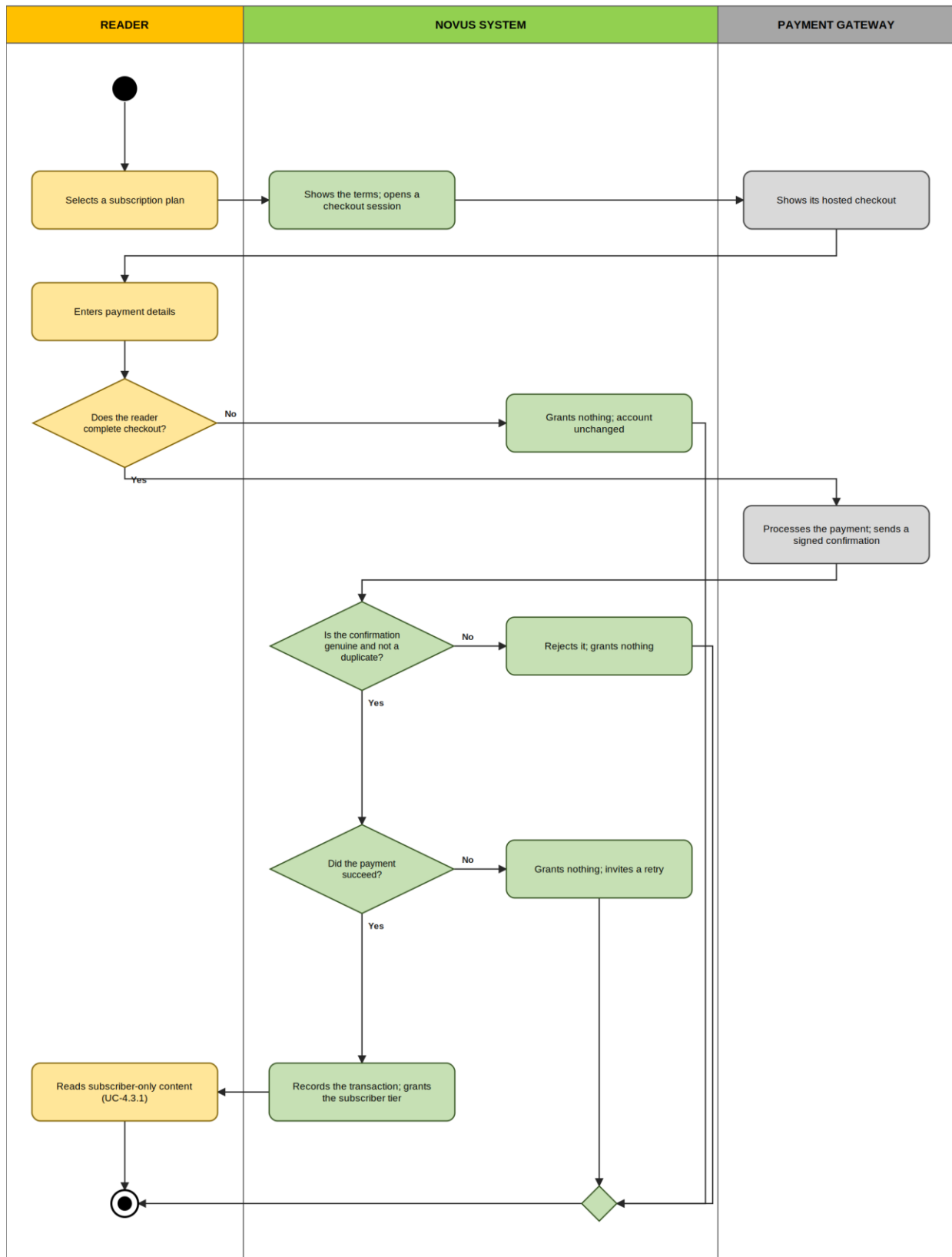


Figure 26 UC-4.3: Subscribe to BOSS Magazine

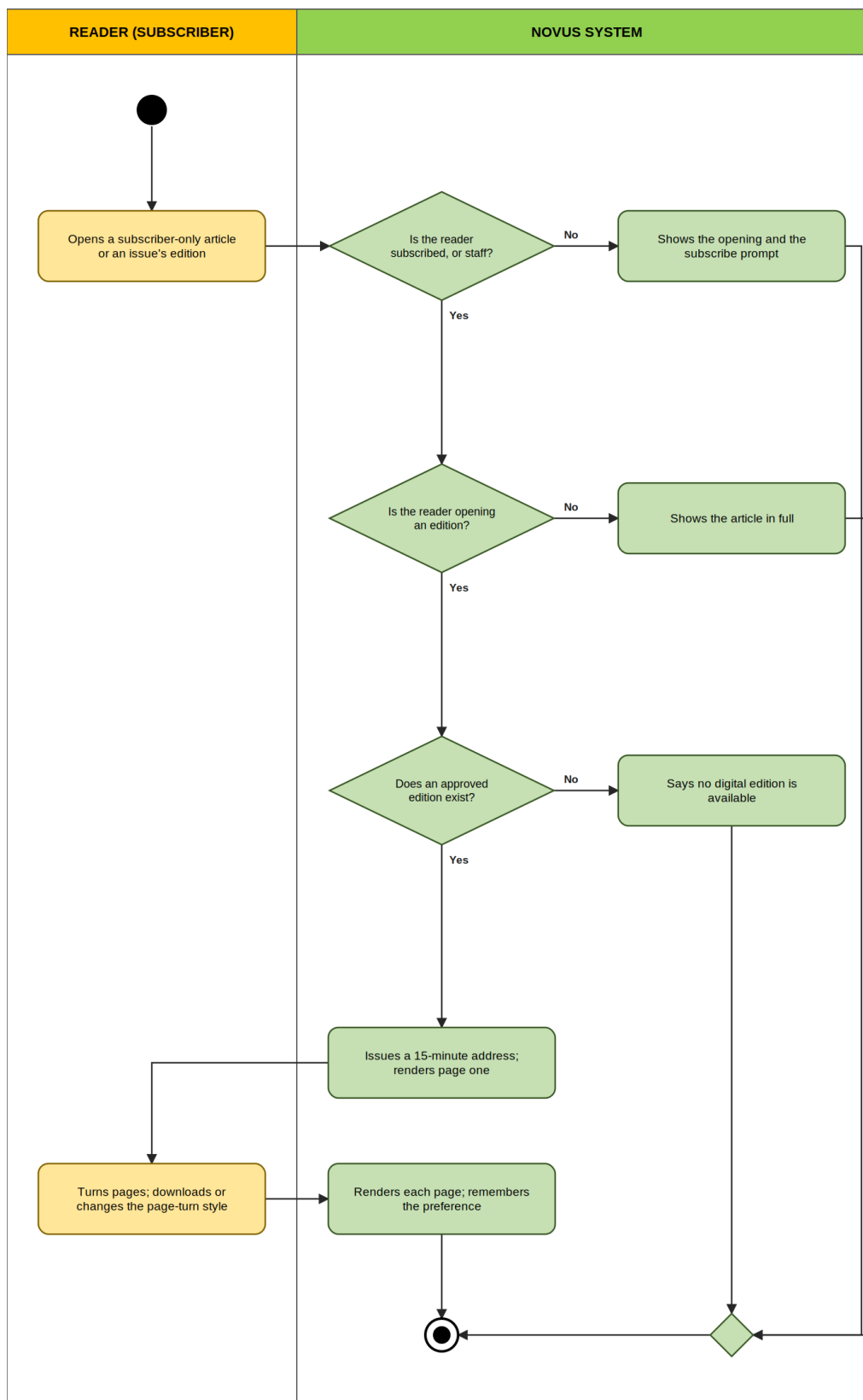


Figure 27 UC-4.3.1: Read Subscriber-Only Content

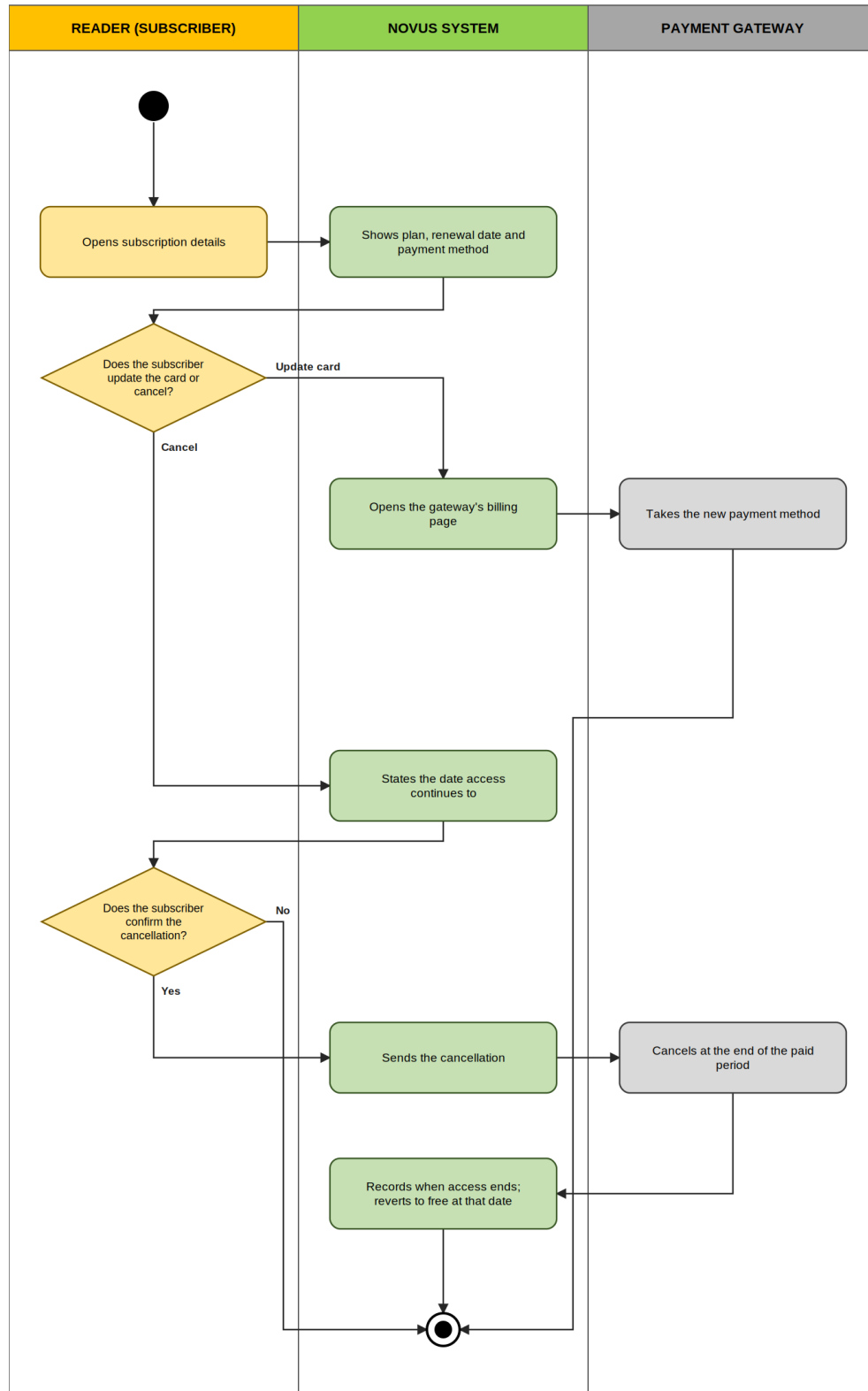


Figure 28 UC-4.4: Manage a Subscription

ACCOUNTS AND PLATFORM ADMINISTRATION

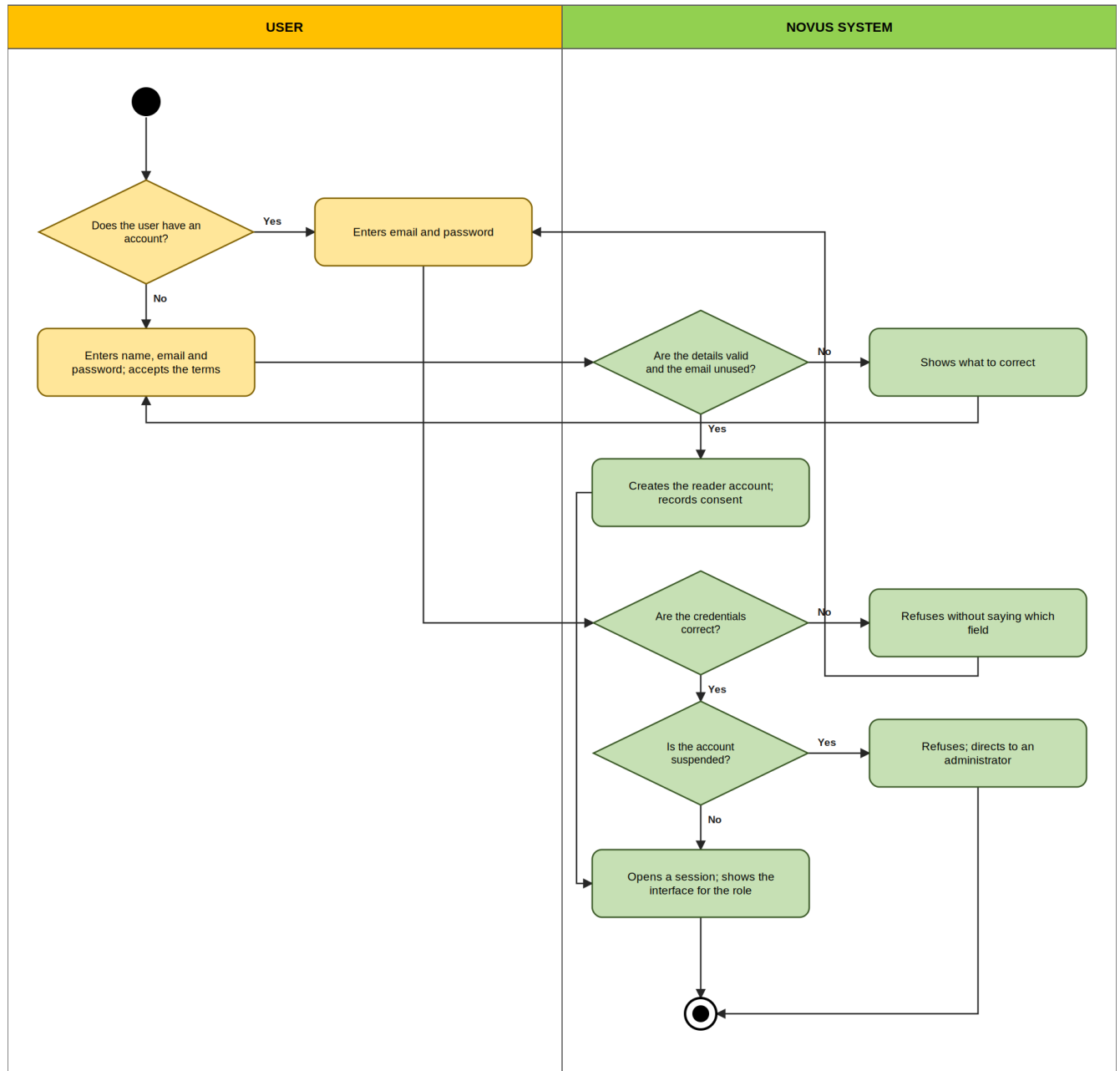


Figure 29 UC-5.1: Register and Authenticate

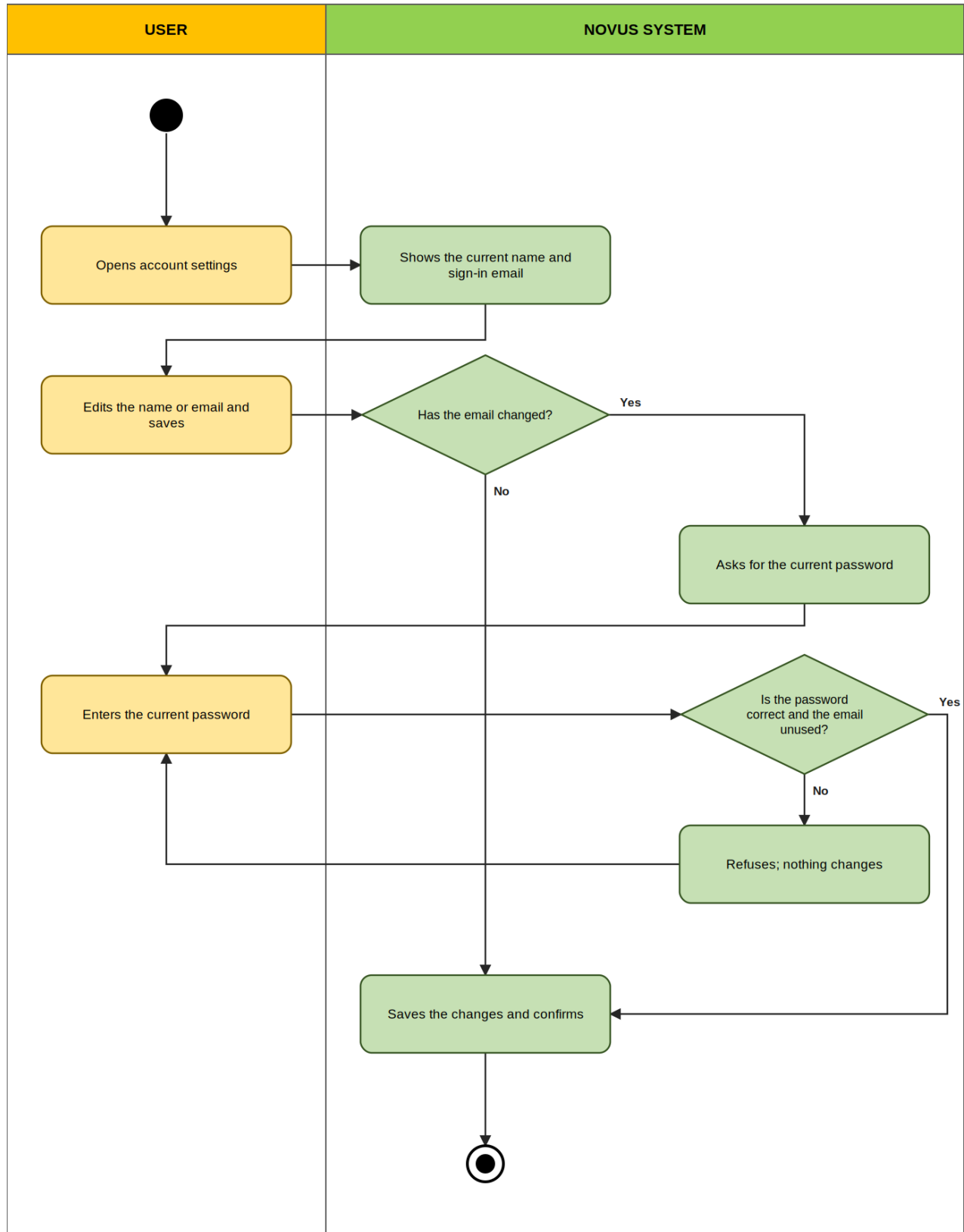


Figure 30 UC-5.2: Manage Own Account

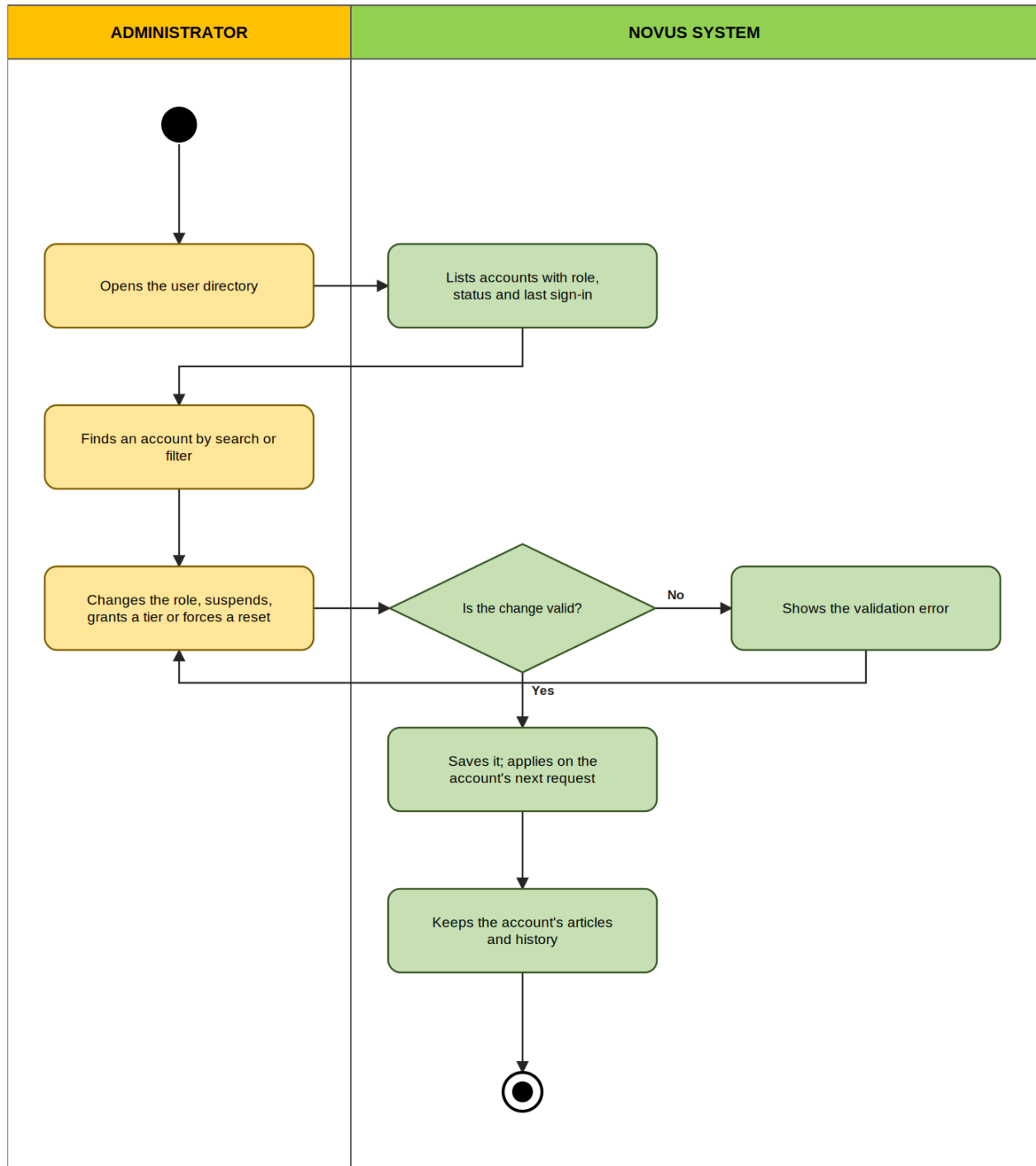


Figure 31 UC-5.3: Administer User Accounts

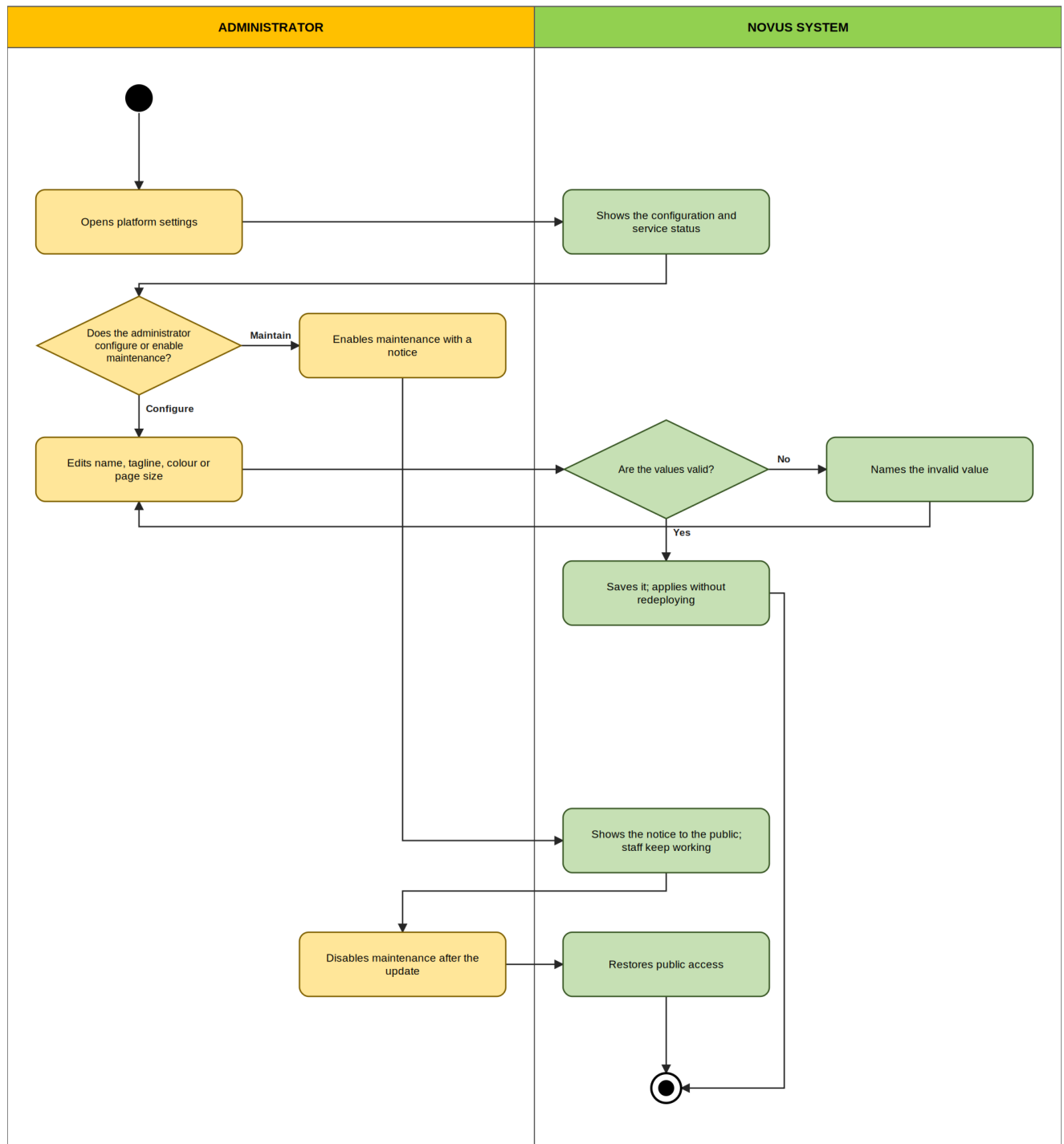


Figure 32 UC-5.4: Configure the Platform

Test Cases based on Fully Dressed Use Case

Access the full test cases through the following link:

[Novus Test Cases with Use Case Traceability](#)

System Analysis and Design

System Context Diagram

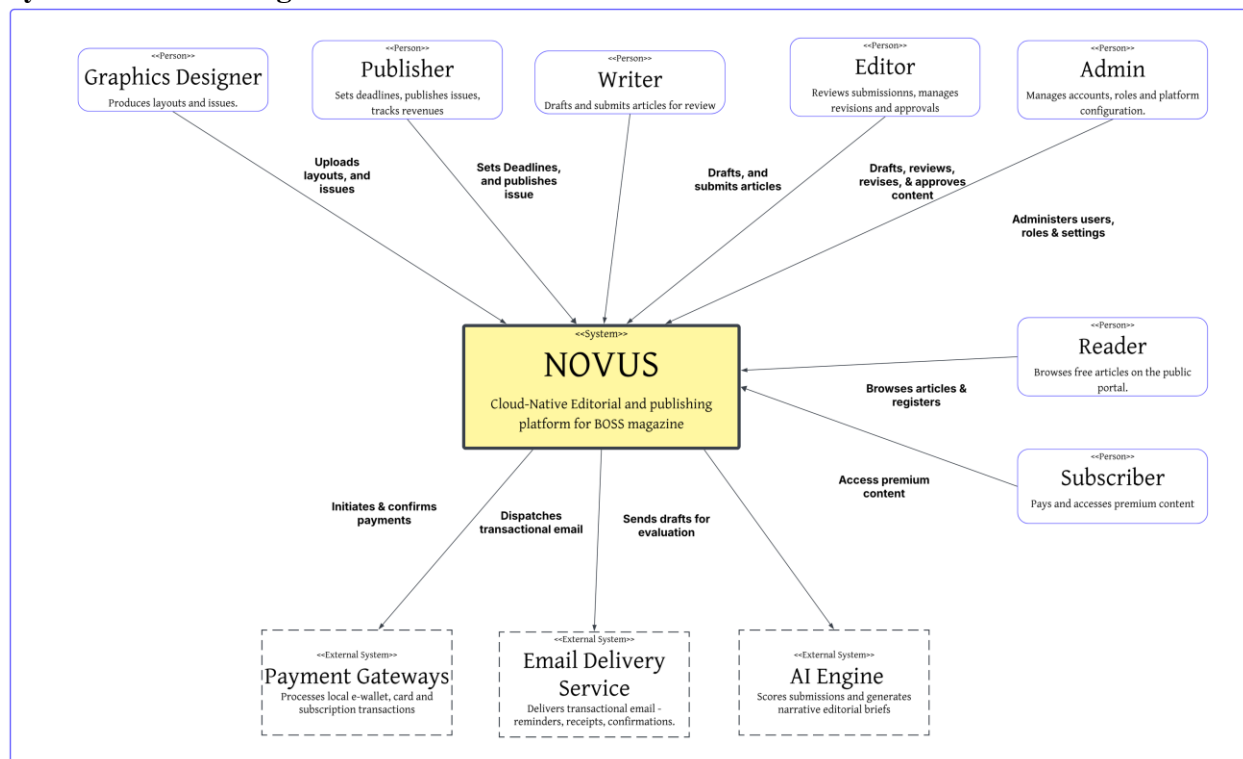


Figure 33 System Context Diagram

The Level 1 C4 diagram shows NOVUS as the central cloud-native publishing platform connecting its main users and external services. Writers, Editors, Publishers, Graphics Designers, and Admins manage content and platform operations, while Readers and Subscribers access published content. NOVUS also integrates payment gateways, an email delivery service, and an AI engine to support payments, notifications, and content evaluation.

Container Diagram

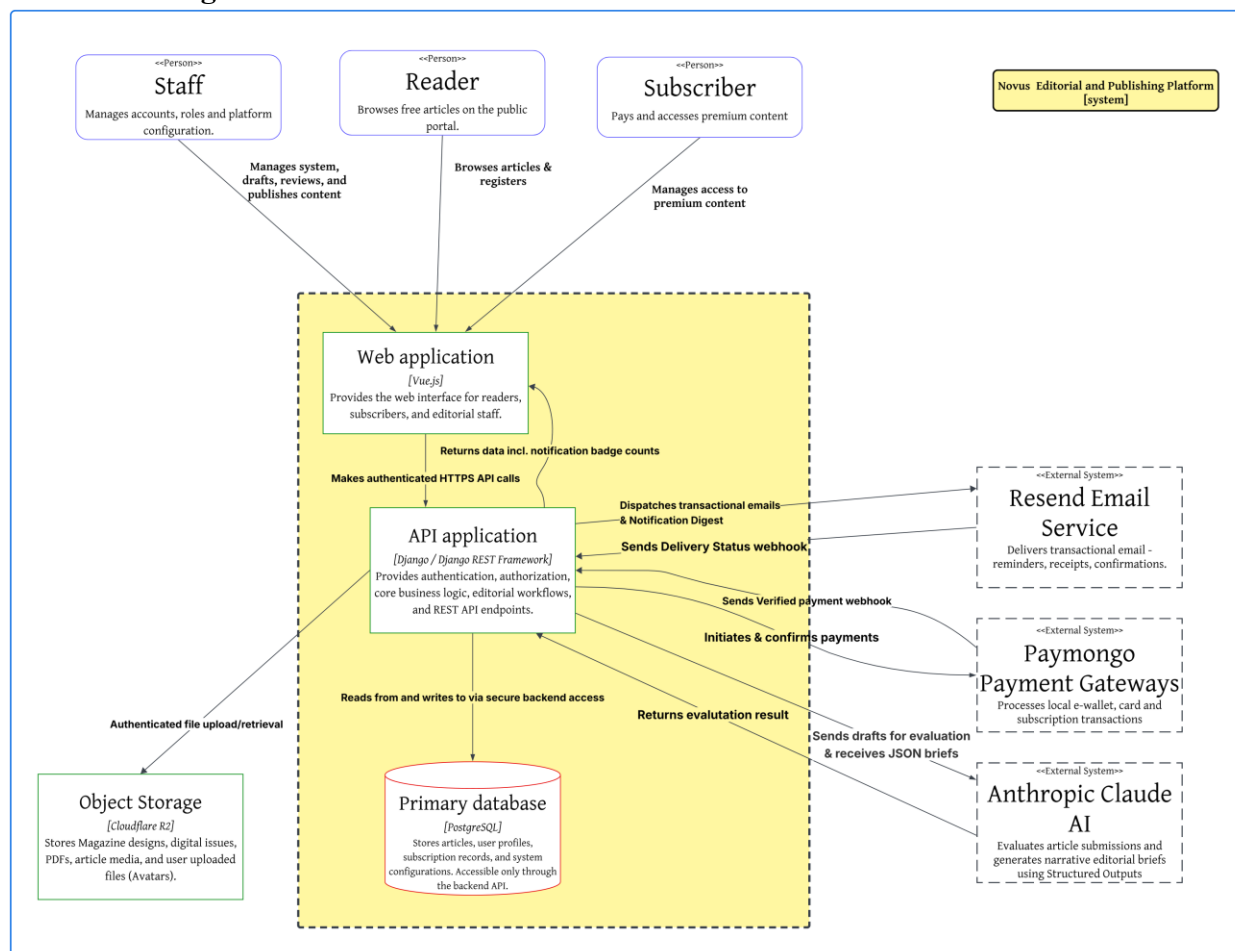


Figure 34 Container Diagram

This Container Diagram maps the high-level architecture of the Novus Editorial and Publishing Platform, demonstrating how Staff, Readers, and Subscribers interact with the system through a central Vue.js Web Application. All client interactions are securely routed via authenticated HTTPS API calls to a core Django API Application, which acts as the central hub for business logic, user authentication, editorial workflows, and REST API endpoints. System data is securely partitioned between a Primary PostgreSQL Database (accessible exclusively through backend channels) for structured records and Cloudflare R2 Object Storage for digital magazine designs, PDF issues, and user-uploaded media. To deliver its complete functional suite, the API application seamlessly integrates across three external services: Resend for automated email dispatches, PayMongo Payment Gateways for secure subscription billing and local e-wallet checkouts, and Anthropic Claude AI for intelligent article evaluations using Structured Outputs.

Component Diagram

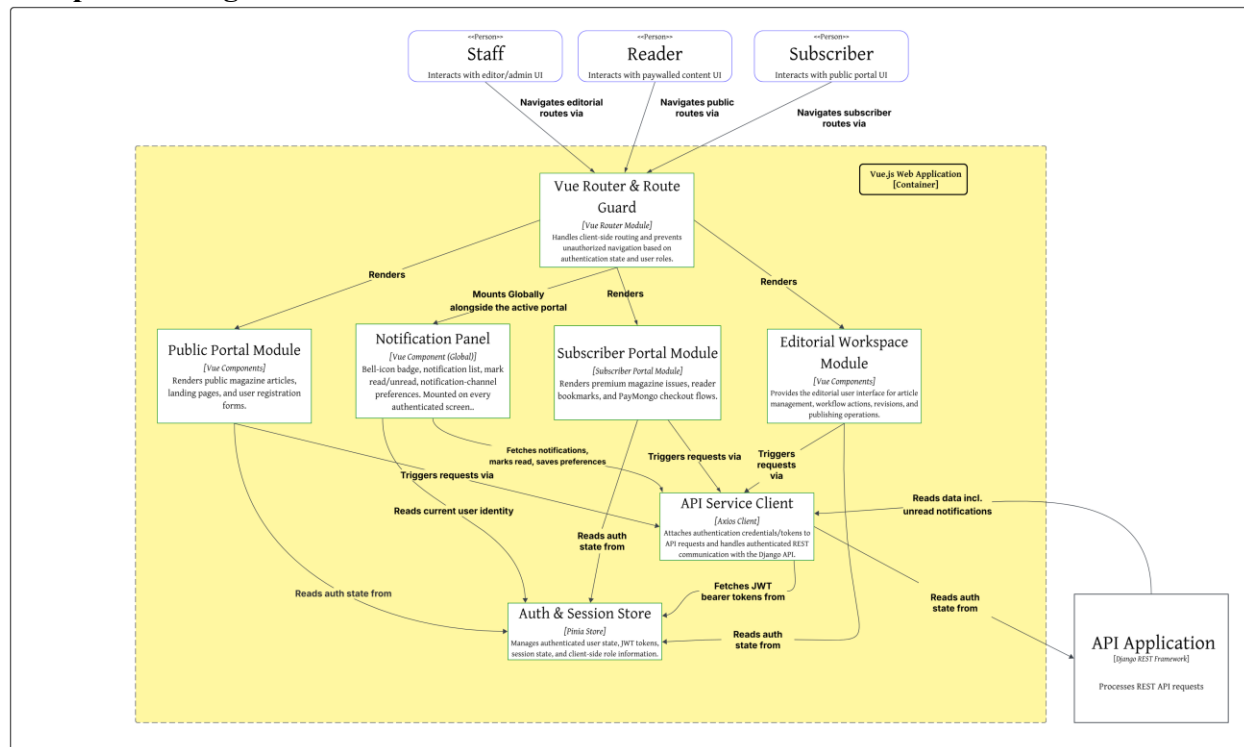


Figure 35 Component Diagram Vue.js Web Application

Vue.js Web Application

This Component Diagram illustrates the frontend architecture of the Vue.js Web Application, detailing how different user roles—including Staff (Writers, Editors, Graphics Designers, Publishers, and Admins), Readers, and Subscribers—interact with the system. User navigation and route permissions are managed by a central Vue Router & Route Guard, which dynamically renders views across role-specific interface modules: the Public Portal Module, the Subscriber Portal Module, and the Editorial Workspace Module, alongside a globally mounted Notification Panel. Internal state, JWT bearer tokens, and user identities are securely managed by a Pinia-based Auth & Session Store, while an Axios-powered API Service Client handles and authenticates all outbound REST requests to the backend Django API Application, driving the interactive web experience for BOSS Magazine PH.

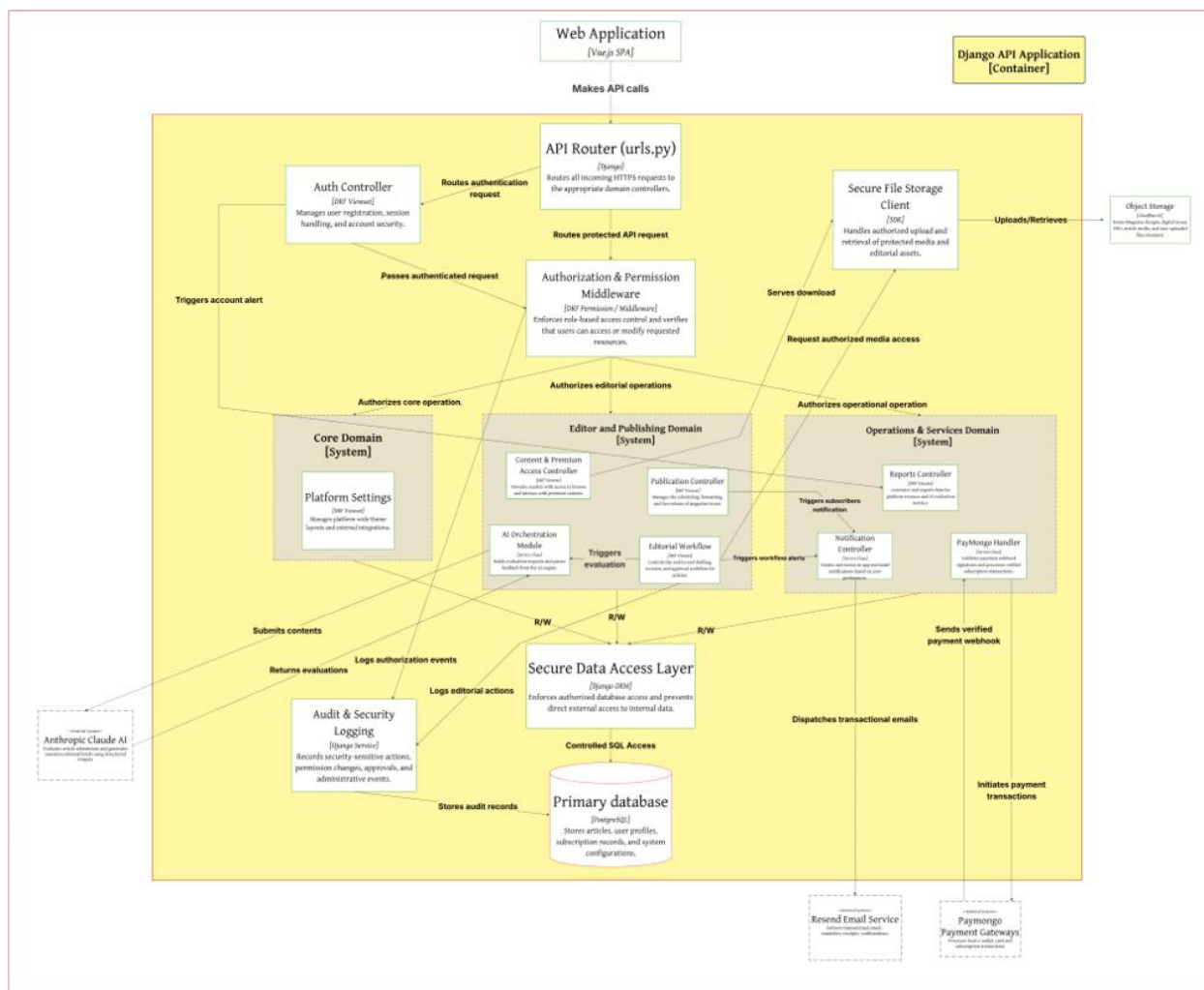


Figure 36 Component Diagram Django API Application

Django API Application

This Component Diagram outlines the internal architecture of the Novus Django API application, illustrating how the backend securely processes and routes requests from the Vue.js frontend. At its entry point, an API Router dispatches incoming traffic through perimeter authentication and permission middleware before delegating execution to three dedicated domain modules: the Core Domain Module (handling platform settings and integrations), the Editor and Publishing Domain Module (managing article drafting, workflow transitions, publication scheduling, paywalls, and Claude AI evaluations), and the Operations & Services Domain Module (overseeing reporting analytics, notification dispatch, and PayMongo payment webhooks). All domain modules communicate through a Secure Data Access Layer to manage encrypted SQL persistence for an encapsulated PostgreSQL primary database, while utilizing specialized clients for Cloudflare R2 object storage and dedicated audit logging, creating a robust, unified backend to drive the editorial workflow for BOSS Magazine PH.

Entity Relationship Diagram

Link: [ERD](#)



Figure 37 Entity Relationship Diagram

Results and Discussion

This section presents the results and detailed discussion of the human-centered design methodology applied during the development of NOVUS: Smart Editorial and Publishing Platform. It details the execution and outcomes across the five sequential stages of the Design Thinking framework: Empathize, Define, Ideate, Prototype, and Test. Through findings from primary stakeholder interviews, empathetic synthesis models (Empathy Maps, 5 Whys, Pain-Gain Analysis), and technical system designs, this section demonstrates how user insights directly informed the platform's architecture, functional capabilities, and operational solutions

Design Thinking Stages

Design Thinking is a human-centered problem-solving process with five stages: empathizing with users, defining the problem, generating ideas, creating prototypes, and test solutions. Allowing continuous improvement until the best solution is reached

Stage 1: Empathize

The first phase of the design thinking process is to have a deep understanding of the target users and to identify the problems they have at hand. The team lay aside all their assumptions about the problem, and the target users. This allows us to objectively consider all possibilities for the customers and their needs. As design thinkers, we immerse ourselves in their environment and experiences through research, interviews, and observations. The goal here is to build empathy by viewing situations from their perspective,, understanding not only what they say, but also how they act, what they think, and how they feel.



Figure 38 Design Thinking Stage 1 Meeting with our Client

To achieve this, we conducted interviews as a group. Our aim was to create a comfortable atmosphere that would encourage them to provide answers without hesitation. By practicing active listening and asking thoughtful, open-ended questions, we were able to gather authentic insights. This process allowed us to uncover their true needs, motivations, and pain points, helping us identify the underlying challenges that require attention.

Client Interviews

The following are the questions the group has made for our client:

Table V Client Interviews

Question	Answer
On average, how many <i>articles/stories</i> or pages are submitted per publishing cycle?	More than 10 articles/Stories
How long does it typically take (<i>in days</i>) from when an article is submitted to when you actually open and begin reviewing it?	It takes around 1-2 days from being submitted to being actually opened and reviewed
How often do multiple article submissions arrive on the same day?	Frequently more than half the time
On average, how many revision rounds does an article go through before it is approved?	It varies widely per writer
On average, how many days pass from when an article is submitted to when a final decision (approved or rejected) is made?	It normally takes around 1-2 days

5 Whys Simplified

Why does BOSS Magazine PH experience workflow delays, limited revenue, and difficulty scaling?

- Because its editorial, publishing, and business activities are handled mostly through manual processes and disconnected platforms.

Why are the processes manual and disconnected?

- Because BOSS Magazine PH **does not have a centralized digital platform** that manages content, editorial workflows, publishing, and digital business functions.

Why does the magazine not have a centralized digital platform?

- Because its current operations have relied primarily on social media, messaging applications, email, and other general-purpose tools rather than a dedicated publishing infrastructure.

Why is relying on these general-purpose tools becoming a problem?

- Because the magazine's content volume and operational responsibilities have grown, while its existing tools do not provide sufficient workflow tracking, automation, collaboration, analytics, or monetization capabilities.

Why can't the current setup support the magazine's growth and sustainability?

- Because BOSS Magazine PH lacks a dedicated digital infrastructure designed to integrate its editorial, publishing, audience reach, and revenue processes into one scalable system.

Root Cause

BOSS Magazine PH lacks a dedicated and centralized digital infrastructure that integrates editorial workflow, publishing operations, audience reach, and revenue generation, resulting in manual work, operational inefficiencies, limited scalability, and missed digital business opportunities.

Theoretical Personas

PERSONA 1: MARIA

Magazine editor

Manages multiple writers and article submissions

Struggles with tracking articles, deadlines, revisions, and communication

PERSONA 2: DANIEL

Magazine writer

Writes and submits articles for publication

Struggles with unclear instructions, fragmented communication, and delayed feedback

PERSONA 3: ROBERT

Magazine publisher

Oversees the quality, production, and publishing of magazine issues

Struggles with missed deadlines, late problems, and lack of visibility into the publishing process

PERSONA 4: ALEX

Graphic designer

Creates magazine layouts and visual content

Struggles with late content changes, unclear design directions, and repeated layout revisions

Empathy Map

Say	Thinks
• "I don't have a formal publishing schedule — I just try to keep track in my head" • "When a deadline is missed, it takes more than 2 weeks to recover" • "One late article can delay the whole issue" • "I noticed some submissions were heavily AI-generated — I could tell just from reading it" • "The more writers we add, the more work falls on me" • "We need to start making money from our content digitally — we can't just keep publishing for free."	• "I'm the only one reviewing everything — if I fall behind, the whole issue falls behind" • "I can't tell which articles are genuinely in progress and which ones writers have forgotten about" • "Some of these submissions feel like they were written by AI — I had to catch it myself with no tool to help me" • "The magazine produces quality content but has no way to earn from it. We are doing the work of a real publication without any of the revenue that should come with it." • "I spend more time managing messages and chasing people than I do actually editing" • "If one writer misses their deadline, it's not just their article that's late — it's everyone else's too" • "I'm reviewing the same types of basic errors over and over — there has to be a better way" • "I don't even know how many hours I'm putting in until I look back at the week"
Does	Feels
• Receives 10+ articles per publishing cycle across 4+ communication platforms (Messenger, email, Viber, etc.) • Spends 1–2 days reviewing each submitted article • Conducts multiple revision rounds per article; writers take 3–5 days to resubmit • Makes final approval decisions in 1–2 days per article • Reviews at least 5 articles per publishing cycle — only 3–4 out of 10 need no revision • Spends 1–2 hours daily chasing writers for updates or reminders • Spends 2+ hours daily writing and explaining revision notes to writers • Handles 21–49 messages per working day related to article management alone • Commits 31–40 hours per week on editorial tasks • Takes 2+ weeks to complete a full publishing cycle	• Overwhelmed — 31–40 hours of editorial work per week with no system, no automation, no support • Frustrated — spending 2+ hours a day just chasing writers and explaining revisions that could be standardized • Anxious — one missed deadline doesn't just affect one article, it cascades into the entire issue • Unsupported — she is the only editor, and the workload scales with every new writer added • Vigilant but exhausted — having to manually detect AI-generated content on top of everything else • Scattered — managing 21–49 messages a day across 4+ platforms with no single source of truth • Responsible but isolated — she cares deeply about the magazine's quality but carries the entire editorial burden alone • She recognizes that the magazine needs to generate digital revenue to be sustainable, but has no platform, no subscription system, and no mechanism to act on that ambition under the current setup. Commercially aware but stuck.

Figure 39 Empathy Map Editor

1. EDITOR

SAY

- I don't have a formal publishing schedule I just try to keep track in my head.
- When a deadline is missed, it takes more than 2 weeks to recover.
- One late article can delay the whole issue.
- I noticed some submissions were heavily AI-generated, I could tell just from reading it.
- The more writers we add, the more work falls on me.
- We need to start making money from our content digitally we can't just keep publishing for free

THINKS

- I'm the only one reviewing everything if I fall behind, the whole issue falls behind.
- I can't tell which articles are genuinely in progress and which ones writers have forgotten about.
- Some of these submissions feel like they were written by AI I had to catch it myself with no tool to help me.
- The magazine produces quality content but has no way to earn from it. We are doing the work of a real publication without any of the revenue that should come with it.
- I spend more time managing messages and chasing people than I do actually editing.
- If one writer misses their deadline, it's not just their article that's late it's everyone else's too.
- I'm reviewing the same types of basic errors over and over there has to be a better way.
- I don't even know how many hours I'm putting in until I look back at the week.

DOES

- Receives 10+ articles per publishing cycle across 4+ communication platforms (Messenger, email, Viber, etc.).
- Spends 1–2 days reviewing each submitted article.
- Conducts multiple revision rounds per article; writers take 3–5 days to resubmit.
- Makes final approval decisions in 1–2 days per article.
- Reviews at least 5 articles per publishing cycle only 3–4 out of 10 need no revision.
- Spends 1–2 hours daily chasing writers for updates or reminders.
- Spends 2+ hours daily writing and explaining revision notes to writers.
- Handles 21–49 messages per working day related to article management alone.
- Commits 31–40 hours per week on editorial tasks.
- Takes 2+ weeks to complete a full publishing cycle.

FEELS

- Overwhelmed 31–40 hours of editorial work per week with no system, no automation, no support.
- Frustrated spending 2+ hours a day just chasing writers and explaining revisions that could be standardized.
- Anxious one missed deadline doesn't just affect one article, it cascades into the entire issue.
- Unsupported she is the only editor, and the workload scales with every new writer added.
- Vigilant but exhausted having to manually detect AI-generated content on top of everything else.
- Scattered managing 21–49 messages a day across 4+ platforms with no single source of truth.
- Responsible but isolated, she cares deeply about the magazine's quality but carries the entire editorial burden alone.

Say	Thinks
• "I need a dedicated Workspace for my tasks." • "Communication is fragmented across multiple apps." • "I want a more structured workflow and timeline." • "I'm unsure if the editor has even seen my submission." • "Feedback is sometimes somewhat vague or unclear." • "I've had to ask for clarification before proceeding with revisions." • "Personal schedule conflicts make meeting deadlines hard." • "I wait and assume it's fine if I hear nothing back."	• Did my editor miss my message in the 80% Messenger noise? • I wish I knew exactly what "Under Review" actually looks like. • Is the feedback coming today, or in 4–7 days like last time? • I'm probably guessing what they want in this revision round. • Why do I have to check Gmail, Viber, AND Messenger for one brief? • I'd be more productive if I didn't have to 'check in' myself. • I hope I'm not writing something that will be rejected later. • There has to be a better way to brainstorm than this.
Does	Feels
• Switches between 3+ communication platforms daily. • Starts writing even when the brief is only "mostly clear." • Frequently scrolls back through chat history to find buried notes. • Manually checks in with the editor to find out the article status. • Balances multiple assignments without a central progress tracker. • Resubmits work while still feeling a bit uncertain of requirements. • Uses Google Docs/Sheets as a makeshift organization tool. • Delays work due to waiting on a response from the editorial side.	• Anxious about missed messages getting "buried" in group chats. • Disconnected due to the lack of a "clear acknowledgment" (20%). • Frustrated by the "It varies a lot" feedback timeline (60%). • Uncertain when starting an article without a 100% clear brief. • Pressured by deadlines when the instructions arrive late. • Blind regarding where their work sits in the editorial pipeline. • Overwhelmed by the fragmentation of the current "manageable" system. • Under-supported when brainstorming and expanding on new ideas.

Figure 40 Empathy Map Writers

2. WRITERS

SAY

- I need a dedicated workspace for my tasks.
- Communication is fragmented across multiple apps.
- I want a more structured workflow and timeline.
- I'm unsure if the editor has even seen my submission.
- Feedback is sometimes somewhat vague or unclear.
- I've had to ask for clarification before proceeding with revisions.
- Personal schedule conflicts make meeting deadlines hard.
- I wait and assume it's fine if I hear nothing back.

THINKS

- Did my editor miss my message in the 80% Messenger noise?
- I wish I knew exactly what 'Under Review' actually looks like.
- Is the feedback coming today, or in 4–7 days like last time?
- I'm probably guessing what they want in this revision round.
- Why do I have to check Gmail, Viber, AND Messenger for one brief?
- I'd be more productive if I didn't have to 'check in' myself.
- I hope I'm not writing something that will be rejected later.
- There has to be a better way to brainstorm than this.

DOES

- Switches between 3+ communication platforms daily.
- Starts writing even when the brief is only mostly clear.
- Frequently scrolls back through chat history to find buried notes.
- Manually checks in with the editor to find out the article status.
- Balances multiple assignments without a central progress tracker.
- Resubmits work while still feeling a bit uncertain of requirements.
- Uses Google Docs/Sheets as a makeshift organization tool.
- Delays work due to waiting on a response from the editorial side.

FEELS

- Anxious about missed messages getting "buried" in group chats.
- Disconnected due to the lack of a "clear acknowledgment" (20%).
- Frustrated by the "It varies a lot" feedback timeline (60%).
- Uncertain when starting an article without a 100% clear brief.
- Pressured by deadlines when the instructions arrive late.
- Blind regarding where their work sits in the editorial pipeline.
- Overwhelmed by the fragmentation of the current "manageable" system.
- Under-supported when brainstorming and expanding on new ideas.

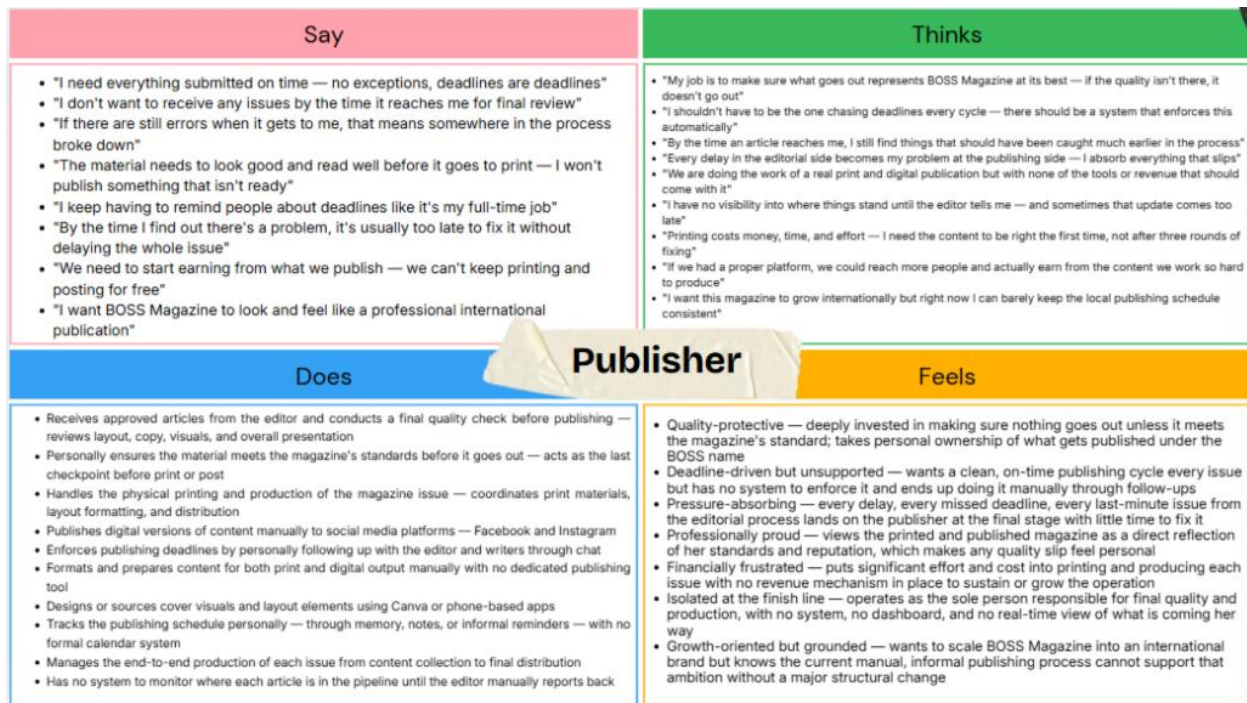


Figure 41 Empathy map Publisher

3. PUBLISHER

SAY

- I need everything submitted on time no exceptions, deadlines are deadlines.
- I don't want to receive any issues by the time it reaches me for final review.
- If there are still errors when it gets to me, that means somewhere in the process broke down.
- The material needs to look good and read well before it goes to print I won't publish something that isn't ready.
- I keep having to remind people about deadlines like it's my full-time job.
- By the time I find out there's a problem, it's usually too late to fix it without delaying the whole issue.
- We need to start earning from what we publish we can't keep printing and posting for free.
- I want BOSS Magazine to look and feel like a professional international publication.

THINKS

- My job is to make sure what goes out represents BOSS Magazine at its best if the quality isn't there, it doesn't go out.
- I shouldn't have to be the one chasing deadline every cycle there should be a system that enforces this automatically.
- By the time an article reaches me, I still find things that should have been caught much earlier in the process.
- Every delay in the editorial side becomes my problem at the publishing side I absorb everything that slips.

- We are doing the work of a real print and digital publication but with none of the tools or revenue that should come with it.
- I have no visibility into where things stand until the editor tells me and sometimes that update comes too late.
- Printing costs money, time, and effort I need the content to be right the first time, not after three rounds of fixing.
- If we had a proper platform, we could reach more people and actually earn from the content we work so hard to produce.
- I want this magazine to grow internationally but right now I can barely keep the local publishing schedule.

DOES

- Receives approved articles from the editor and conducts a final quality check before publishing reviews layout, copy, visuals, and overall presentation.
- Personally, ensures the material meets the magazine standards before it goes out acts as the last checkpoint before printing or post.
- Handles the physical printing and production of the magazine issue coordinates print materials, layout formatting, and distribution.
- Publishes digital versions of content manually to social media platforms Facebook and Instagram.
- Enforces publishing deadlines by personally following up with the editor and writers through chat.
- Formats and prepares content for both print and digital output manually with no dedicated publishing tool.
- Designs or sources cover visuals and layout elements using Canva or phone-based apps.
- Tracks the publishing schedule personally through memory, notes, or informal reminders, with no formal calendar system.
- Manages the end-to-end production of each issue from content collection to final distribution.
- Has no system to monitor where each article is in the pipeline until the editor manually reports back.

FEELS

- Quality-protective deeply invested in making sure nothing goes out unless it meets the magazine's standard; takes personal ownership of what gets published under the BOSS name.
- Deadline-driven but unsupported wants a clean, on-time publishing cycle on every issue but has no system to enforce it and ends up doing it manually through follow-ups.
- Pressure-absorbing every delay, every missed deadline, and every last-minute issue from the editorial process lands on the publisher at the final stage with little time to fix it.
- Professionally proud views the printed and published magazine as a direct reflection of her standards and reputation, which makes any quality slip feel personal.
- Financially frustrated puts significant effort and cost into printing and producing each issue with no revenue mechanism in place to sustain or grow the operation.
- Isolated at the finish line operates as the sole person responsible for final quality and production, with no system, no dashboard, and no real-time view of what is coming her way.
- Growth-oriented but grounded wants to scale BOSS Magazine into an international brand but knows the current manual, informal publishing process cannot support that ambition without a major structural change.

Say	Thinks
• "I need all the final articles in one place before I can start laying out the magazine." • "When the content keeps changing after I've already placed it, I have to redo everything from scratch." • "I don't always know if what I'm designing matches what the editor actually wants." • "Nobody tells me the deadline until it's almost here." • "I spend more time waiting for content than actually designing." • "Sometimes I get feedback that's too vague — 'make it look better' doesn't tell me what to fix." • "I want to know upfront what the theme and feel of the issue is supposed to be." • "If the articles are still being revised, I can't finalize the layout — everything depends on the content being locked."	• Will the editor send me the final versions today, or will I be waiting again? • How am I supposed to design a layout when the article count keeps changing? • I hope what I submit doesn't come back with a completely different direction this time. • Is there a style guide I should be following, or am I just guessing the aesthetic again? • Why is the deadline moved up when the content wasn't even finalized yet? • No one tells me when articles get added or removed — I only find out when I check myself. • I'd work faster if I had the text and photos ready at the same time instead of getting them piece by piece. • Am I supposed to wait for the editor to approve the layout before moving to the next section, or just keep going?
Does	Feels
• Waits for all articles to be finalized by the editor before beginning the magazine layout. • Receives articles and assets through chat or email — no centralized file handoff system. • Manually arranges text, images, and visuals across multiple pages using design tools. • Revises layout multiple times based on feedback from the editor with no version tracking. • Checks in with the editor or publisher to find out if the content count or direction has changed. • Works without a formal design brief — interprets the magazine's visual identity based on past issues. • Adjusts layout from scratch whenever last-minute article changes or additions come in. • Submits completed layout back to the editor through chat for review and confirmation. • Manages multiple layout versions manually with no shared workspace or feedback tool.	• Dependent — cannot start or finish work without the content being locked and delivered first. • Uninformed — rarely receives a proper design brief or issue theme upfront, so a lot of decisions are made by guessing. • Frustrated — redoing layouts from scratch when articles change late in the process wastes significant time and effort. • Disconnected — has no visibility into the editorial pipeline, so changes always come as a surprise. • Pressured — absorbs deadline pressure from the editorial side even when delays were caused upstream. • Undervalued — design feedback is often vague or focuses on content rather than the visual work itself. • Isolated — works independently without a shared tool where the editor can leave layout-specific comments directly. • Motivated but constrained — wants to produce a polished, professional-looking magazine but the disorganized handoff process makes consistent quality hard to achieve.

Figure 42 Empathy Map Graphic Designer

4. GRAPHIC DESIGNER

SAY

- I need all the final articles in one place before I can start laying out the magazine.
- When the content keeps changing after I've already placed it, I have to redo everything from scratch.
- I don't always know if what I'm designing matches what the editor actually wants.
- Nobody tells me the deadline until it's almost here.
- I spend more time waiting for content than actually designing.
- Sometimes I get feedback that's too vague 'make it look better' doesn't tell me what to fix.
- I want to know upfront what the theme and feel of the issue is supposed to be.
- If the articles are still being revised, I can't finalize the layout, everything depends on the content being locked.

THINKS

- Will the editor send me the final versions today, or will I be waiting again?
- How am I supposed to design a layout when the article count keeps changing?
- I hope what I submit doesn't come back with a completely different direction this time.
- Is there a style guide I should be following, or am I just guessing the aesthetic again?
- Why is the deadline moved up when the content wasn't even finalized yet?
- No one tells me when articles get added or removed, I only find out when I check myself.
- I'd work faster if I had the text and photos ready at the same time instead of getting them piece by piece.

- Am I supposed to wait for the editor to approve the layout before moving to the next section, or just keep going?

DOES

- Waits for all articles to be finalized by the editor before beginning the magazine layout.
- Receives articles and assets through chat or email with no centralized file handoff system.
- Manually arranges text, images, and visuals across multiple pages using design tools.
- Revises layout multiple times based on feedback from the editor with no version tracking.
- Checks in with the editor or publisher to find out if the content count or direction has changed.
- Works without a formal design brief interprets the magazine's visual identity based on past issues.
- Adjusts layout from scratch whenever last-minute article changes or additions come in.
- Submits completed layout back to the editor through chat for review and confirmation.
- Manages multiple layout versions manually with no shared workspace or feedback tool.

FEELS

- Dependent cannot start or finish work without the content being locked and delivered first.
- Uninformed rarely receives a proper design brief or issue theme upfront, so a lot of decisions are made by guessing.
- Frustrated redoing layouts from scratch when articles change late in the process wastes significant time and effort.
- Disconnected has no visibility into the editorial pipeline, so changes always come as a surprise.
- Pressure absorbs deadline pressure from the editorial side even when delays were caused upstream.
- Undervalued design feedback is often vague or focuses on content rather than the visual work itself.
- Isolated works independently without a shared tool where the editor can leave layout-specific comments directly.
- Motivated but constrained wants to produce a polished, professional-looking magazine, but the disorganized handoff process makes consistent quality hard to achieve.

Pain-Gain Analysis Summary

Pains

- Fragmented communication, unclear workflows, and lack of a centralized system cause delays, confusion, and repeated follow-ups.
- Manual tracking, revisions, deadline chasing, and late content changes create heavy workloads and unnecessary rework.
- Limited visibility, unclear feedback, and lack of standardized processes make it difficult for writers, editors, publishers, and Graphic designers to coordinate effectively.
- The current manual process makes it difficult to maintain consistent quality, meet publishing schedules, scale operations, and generate digital revenue.

Gains

- Achieves a centralized and structured publishing workflow where articles, files, feedback, deadlines, and progress are managed in one place.
- Reduces delays, repetitive work, missed deadlines, and unnecessary revisions through automation, notifications, tracking, and standardized feedback.
- Improves collaboration and transparency by giving writers, editors, publishers, and Graphic designers clear visibility of article status, responsibilities, and deadlines.
- Improves content quality, publishing efficiency, scalability, and opportunities for digital monetization while reducing workload and operational stress.

Analysis

- They share a common need for better coordination, visibility, efficiency, and centralized workflow management.
- The root cause of their pains is the reliance on fragmented communication channels, manual processes, unclear responsibilities, and lack of a centralized publishing system.
- A centralized, user-friendly platform with workflow tracking, automated reminders, structured feedback, file management, version control, and clear article statuses can reduce delays and workload.
- Improving the publishing process benefits the entire team by increasing productivity, reducing errors and rework, improving collaboration, maintaining quality, and supporting the magazine's long-term digital growth and revenue goals.

Stage 2: Define

The second phase of the design thinking process is by putting together all the information gathered from the previous phase. This stage is to define the problem statements clearly which should be based on human-centered needs, rather than being focused on the business goal.

Clustered Problem

Core Problem Statement BOSS Magazine PH is a comprehensive Filipino lifestyle publication, covering a broad spectrum of topics including fashion, culture, modern trends, business, and society, that currently operates without any dedicated digital infrastructure to support its editorial, publishing, or revenue functions.

Cluster 1: Workflow & Operational Inefficiencies

- **Fragmented Editorial Workflow:** There is no centralized system; submissions come from multiple platforms, making tracking and management difficult.
- **Time-Consuming Manual Reviews:** All articles are reviewed manually, taking hours daily and increasing editor workload.

Cluster 2: Business & Scalability Constraints

- **Zero Revenue Generation:** There are no subscriptions, paywalls, or monetization strategies in place despite continuous content production.
- **Inability to Scale Operations:** The growing workload causes delays, inefficiency, and difficulty in handling an increased volume of content.

Cluster 3: Infrastructure & Brand Limitations

- **No Dedicated Digital Platform:** The current reliance on Facebook limits the publication's reach, discoverability, and professional credibility.

Release Plan

Updated Release Plan and Progress. The Novus release plan follows the three-phase academic delivery schedule. Release 1, completed during MNTSDEV, delivered the system requirements, Product Backlog, use cases, system design, and prototypes. Release 2 under MSYSADD will focus on detailed architecture and development while Release 3 under MCSPROJ will deliver the coded and integrated system, followed by testing, UAT, and production deployment.

The project is currently on schedule. The major deliverables for the MNTSDEV phase have been completed

Access the full release plan through the following link:

[Novus Release plan - .docx](#)

Prototype

Click the following link to see the deployed prototype.

Reader facing website - <https://novusdeploy.vercel.app/read>

Staff Workspace website - <https://novusdeploy.vercel.app/staff/login>

Wireframes

The wireframes of the proposed system were developed to visualize the interface layout and navigation flow for each user role prior to implementation.

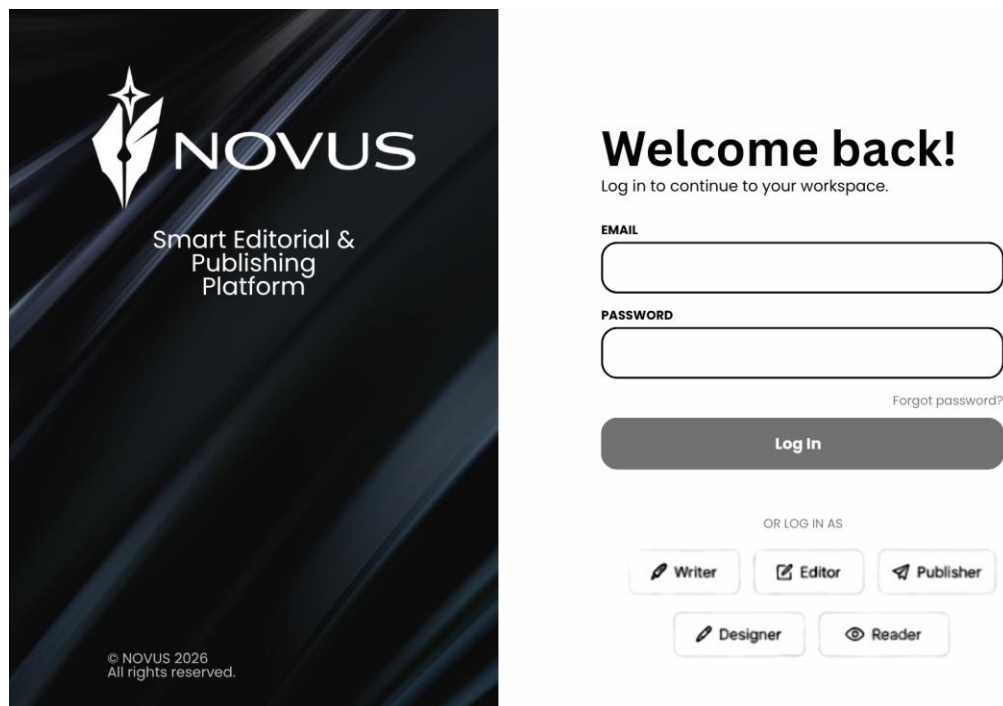


Figure 43 Log in Page

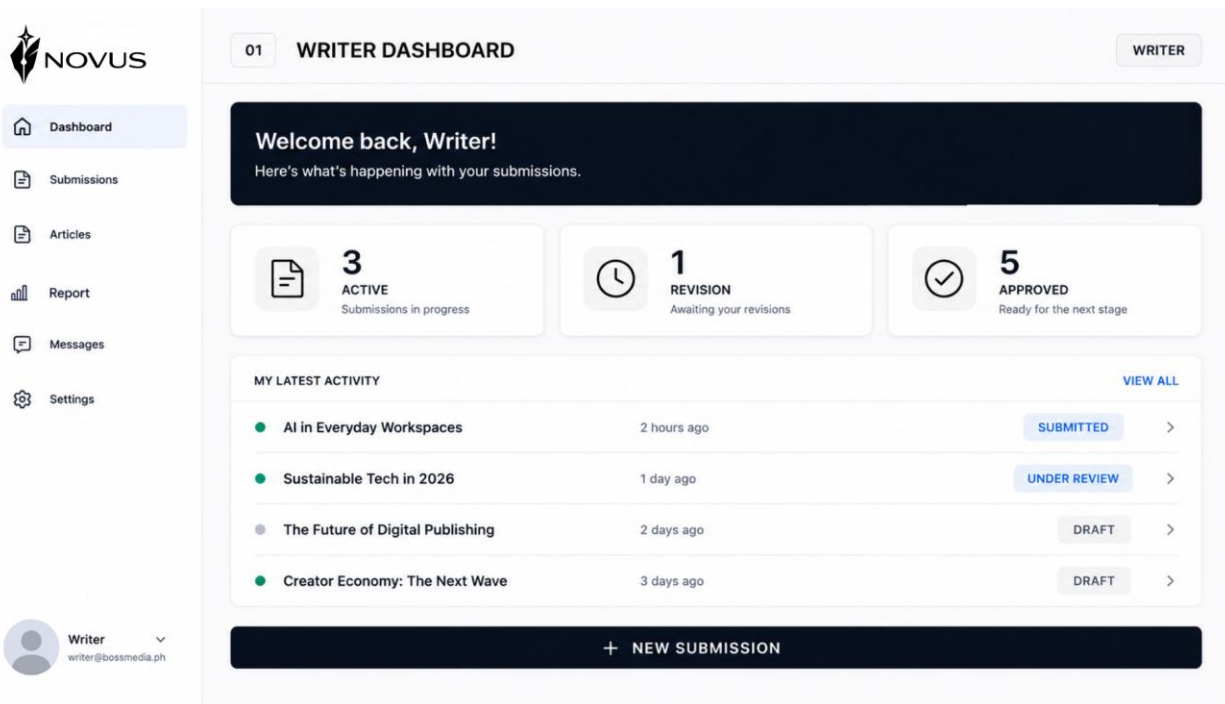


Figure 44 Writer Dashboard



Dashboard

Submissions

Articles

Report

Messages

Settings

Writer

writer@bossmedia.ph

02

ARTICLE SUBMISSION FORM

WRITER

ARTICLE TITLE

Enter article title

SECTION / CATEGORY

Select category

WORD COUNT TARGET

Enter word count

Suggested range: 600 - 1,500 words

ARTICLE BODY

Paragraph

B I U

☰ ☷

“ ”

🔗

🖼️

Write or paste your article here...

🛡️ We've got your back!

Your article will be automatically checked for readability and grammar before review.

TIPS FOR A GREAT ARTICLE

✔️ Use clear and concise language

✔️ Structure your content with headings

✔️ Include relevant images and links

✔️ Proofread before submitting

SAVE DRAFT

SUBMIT FOR REVIEW

🛡️ Articles will be automatically checked for readability and grammar.

Figure 45 Article Submission Form



Dashboard

Submissions

Articles

Report

Messages

Settings

Writer

writer@bossmedia.ph

Studio > Draft Review

Mode: Grammar & Read

Article Editor

Paragraph

B I U

☰ ☷

“ ”

🔗

🖼️

As organizations adapt to lean operations, AI is no longer just a tool—it is becoming the core engine.

❗ Leaders must evaluate how...

(Grammar: Passive voice detected)

Words: 132 Characters: 782

🟢 All changes saved

AI Optimization Panel

📊 Readability Score

Grade Level: 10.2 (Good)

● Flesch Reading Ease: 65

📄 Grammar & Syntax (2)

• Issue #1: Passive voice

Suggest Active Fix

• Issue #2: Typo found

Accept Correction

🔧 Apply All AI Fixes

Figure 46 Article Editor

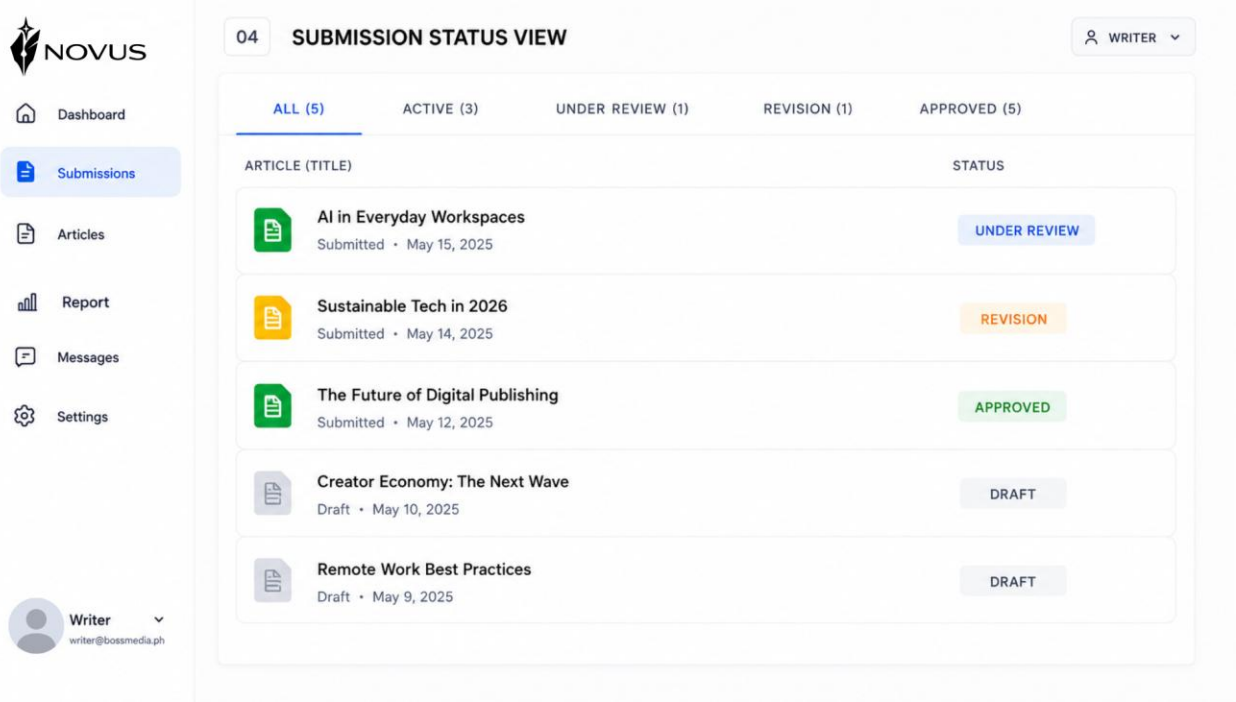


Figure 47 Submission status view

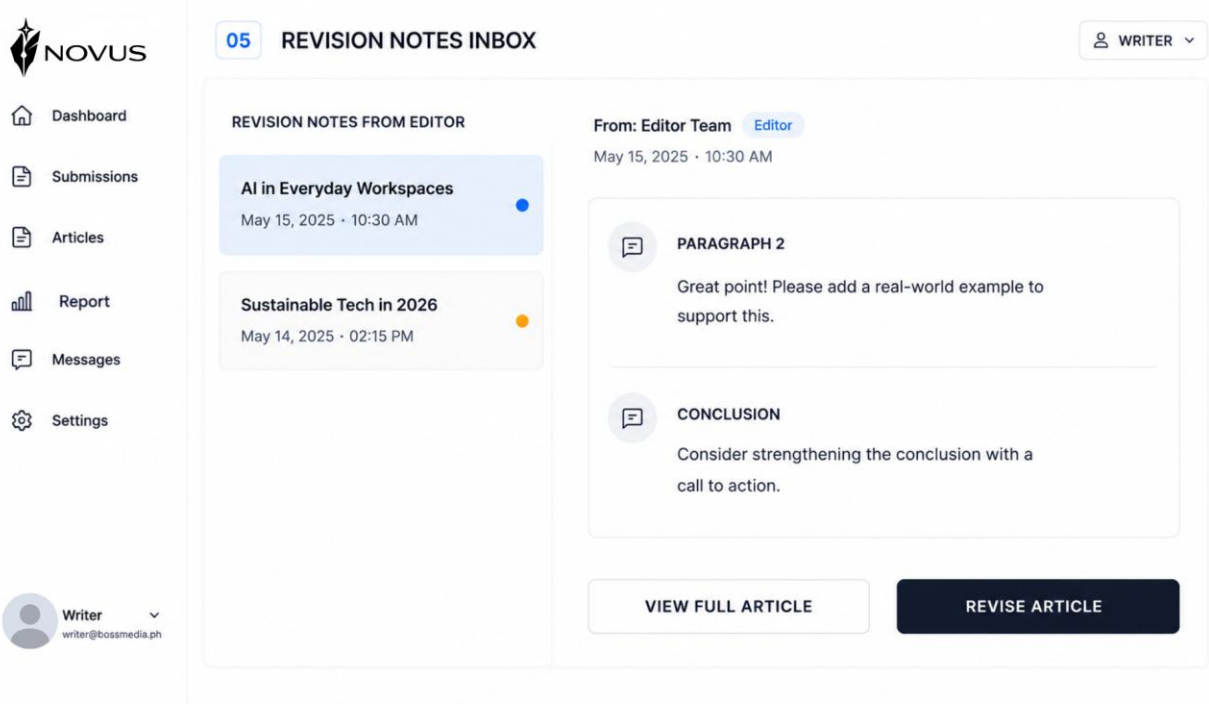


Figure 48 Revisions Inbox

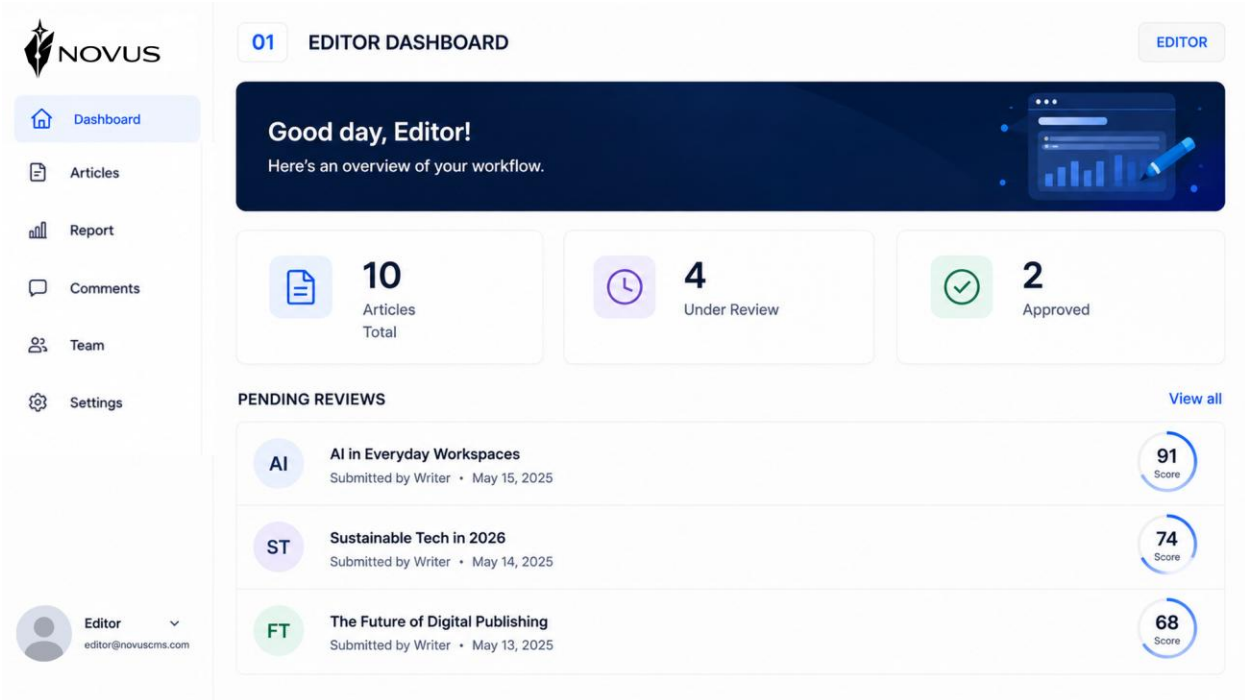


Figure 49 Editor Dashboard

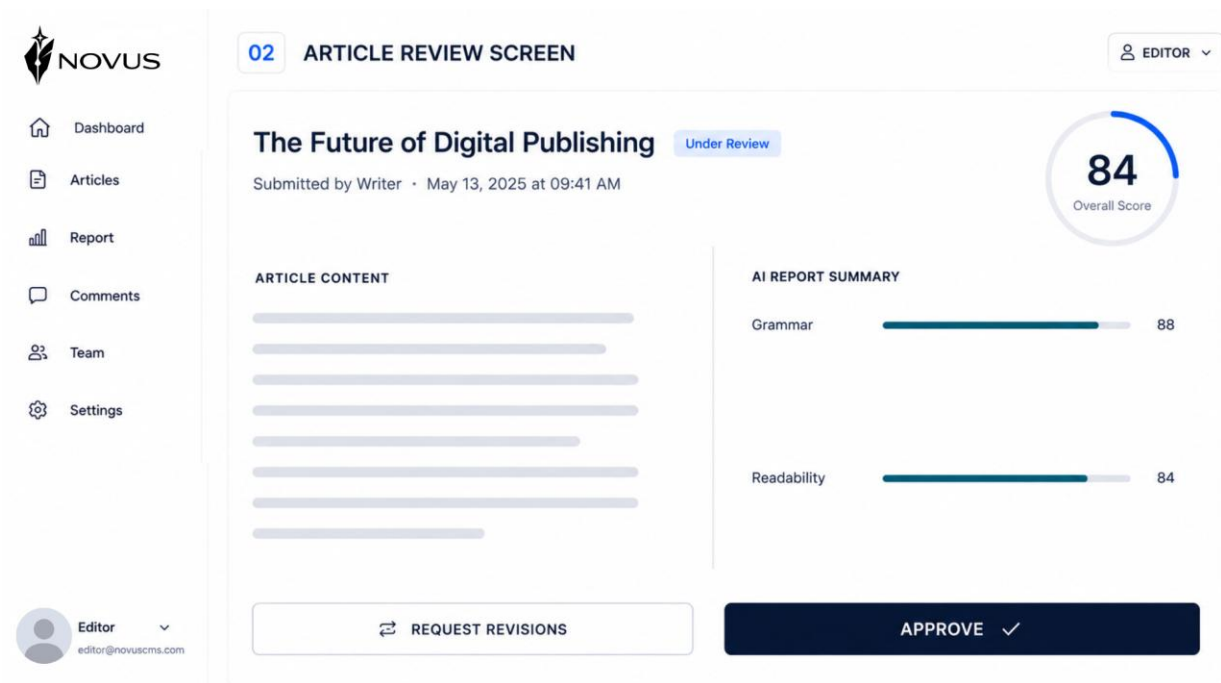


Figure 50 Article Review Screen



- Dashboard
- Articles
- Report
- Comments
- Team
- Settings


Editor
editor@novuscms.com

03

REVISION NOTE COMPOSER

EDITOR

SEND REVISION NOTE

SELECT SECTION

Select section to focus on...

NOTE TYPE

GRAMMAR TONE **STRUCTURE** FACTUAL

REVISION INSTRUCTION

Provide specific feedback or instructions for the writer...

0/500

PRIORITY

Select priority level...

+ ADD ANOTHER NOTE

SEND TO WRITER

Figure 51 Revision notes composer



- Dashboard
- Articles
- Report
- Comments
- Team
- Settings


Editor
editor@novuscms.com

04

APPROVAL SCREEN

EDITOR

 **ARTICLE APPROVED**

This article is approved and ready for the next steps.

ARTICLE PREVIEW

AI SCORE

91
Excellent

Readability 91

Grammar 93

REVISION HISTORY

1 revision round • May 15, 2025 at 10:30 AM

SEND TO PUBLISHER
Notify publisher to review and publish

SEND TO GRAPHIC DESIGNER
Notify designer for visuals and layout

Figure 52 Approval Screen



- Dashboard
- Articles
- Report
- Comments
- Team
- Settings

05 DEADLINE MANAGEMENT PANEL

EDITOR

DEADLINE PRESET

Select deadline template...

FOR ARTICLES ASSIGNED

AI in Everyday Workspaces	MAY 18, 2025	3 DAYS LEFT
Sustainable Tech in 2026	MAY 16, 2025	1 DAY LEFT
The Future of Digital Publishing	MAY 20, 2025	5 DAYS LEFT
Creator Economy: The Next Wave	MAY 22, 2025	7 DAYS LEFT

AUTO REMINDER

Send reminders 24 hours before the deadline.

☒

Editor editor@novuscms.com

SAVE DEADLINES

Figure 53 Deadline management panel



- Dashboard
- Articles
- Report
- Comments
- Team
- Settings

06 PIPELINE OVERVIEW

EDITOR

SEARCH & FILTER PIPELINE

Search articles...

All Status

DRAFT REVIEW APPROVED

ARTICLE	STATUS
AI in Everyday Workspaces	APPROVED
Sustainable Tech in 2026	APPROVED
The Future of Digital Publishing	APPROVED
Remote Work Best Practices	APPROVED
Creator Economy: The Next Wave	APPROVED

5 of 10 articles approved

Editor editor@novuscms.com

Figure 54 Pipeline Overview



- Dashboard
- Articles
- Team
- Layout Handoff**
- Settings


Designer
designer@novuscms.com

01 LAYOUT HANDOFF SCREEN

DESIGNER


NOVUS • DESIGNER • [NAME]
Welcome back! Here are the approved layouts ready for handoff.

APPROVED ARTICLES — READY FOR LAYOUT



AI in Everyday Workspaces
Last updated: May 15, 2025 · Approved v2
All content final and approved

ARTICLE

PHOTOS: 8

DOWNLOAD



Sustainable Tech in 2026
Last updated: May 14, 2025 · Approved v1
All content final and approved

ARTICLE

PHOTOS: 12

DOWNLOAD


All files are final approved versions.
Download the full article package including content brief, assets, and guidelines.

Figure 55 Layout handoff screen



- Dashboard
- Articles
- Team
- Layout Submission**
- Settings


Designer
designer@novuscms.com

02 LAYOUT SUBMISSION SCREEN

DESIGNER

SUBMIT COMPLETED LAYOUT

ISSUE / ARTICLE

Issue #12

UPLOAD LAYOUT FILE



Drag and drop your file here
PDF or INDD files (Max 100MB)

LAYOUT VERSION

v1.0

NOTES TO EDITOR

Add notes about the layout...

0/500


SUBMIT LAYOUT FOR EDITOR REVIEW


You will be notified once the editor reviews your submission.

Figure 56 Layout submission

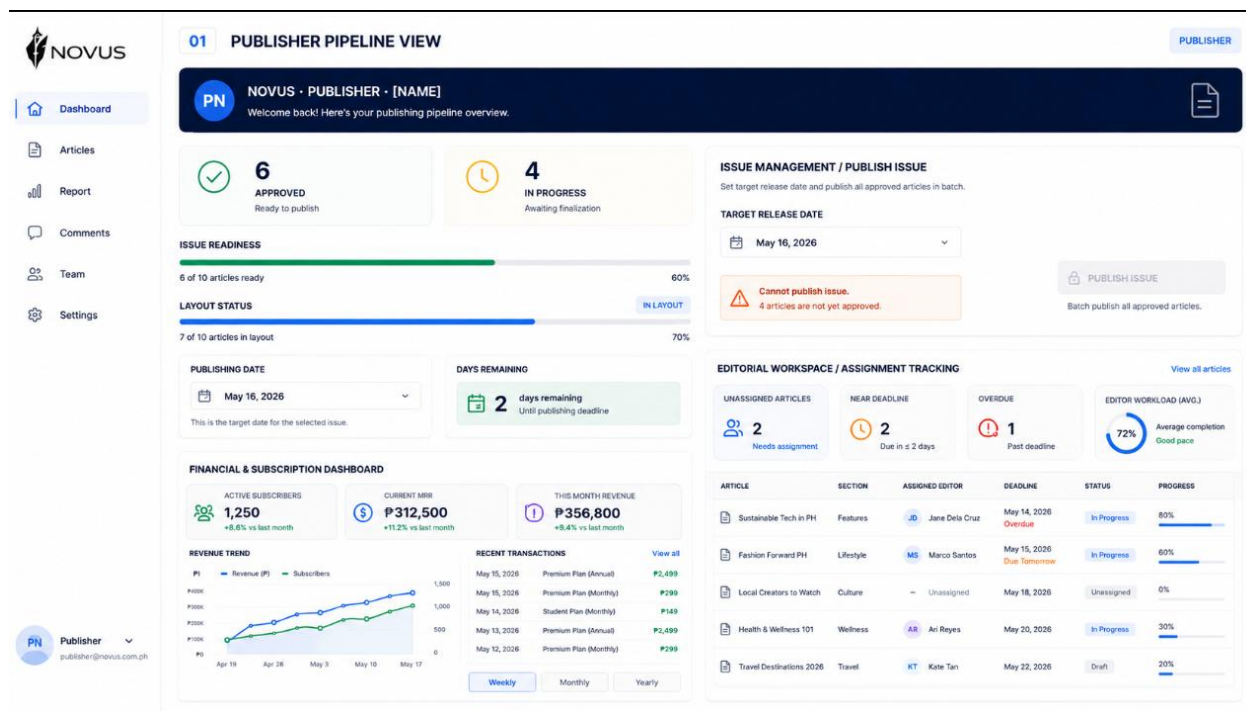


Figure 57 Publisher pipeline view

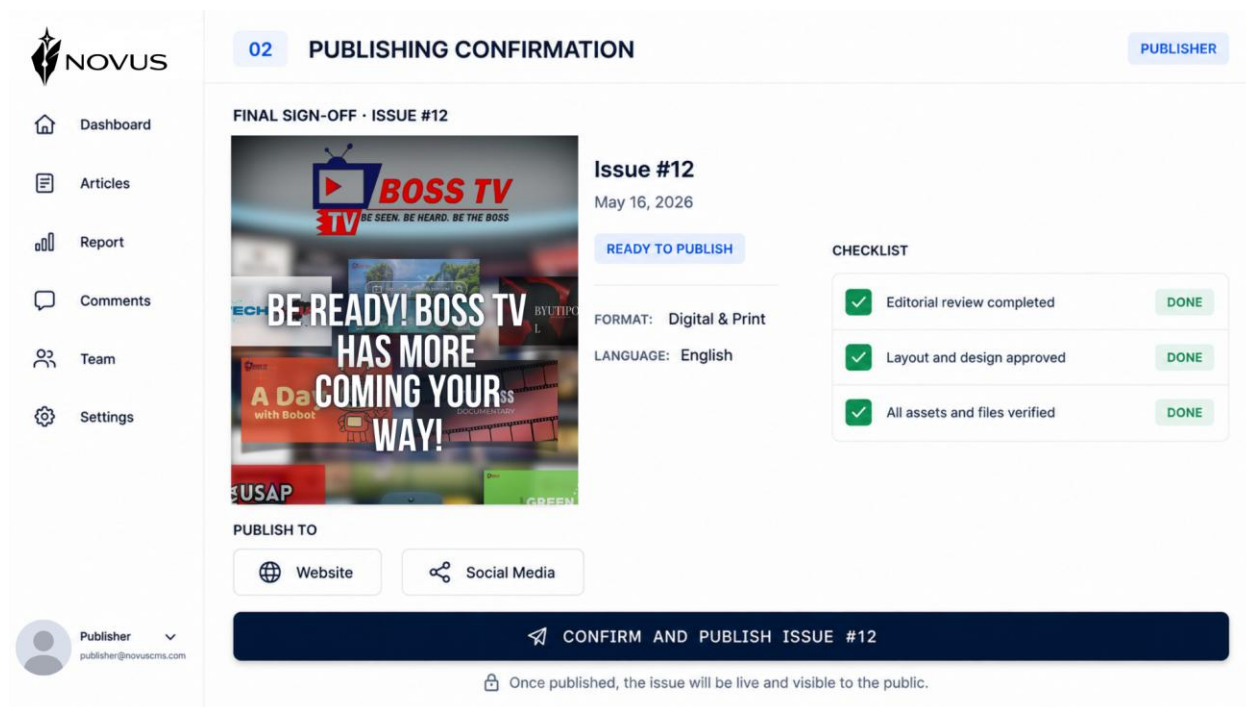


Figure 58 Publishing confirmation

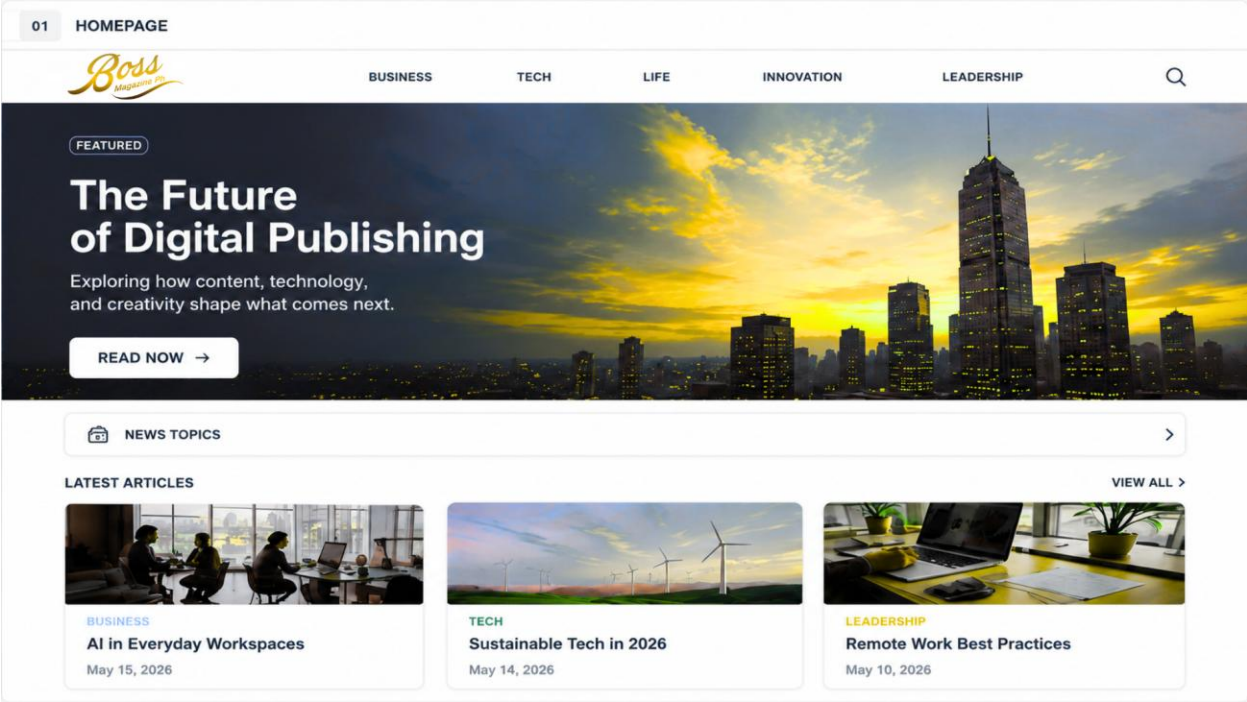


Figure 59 User homepage

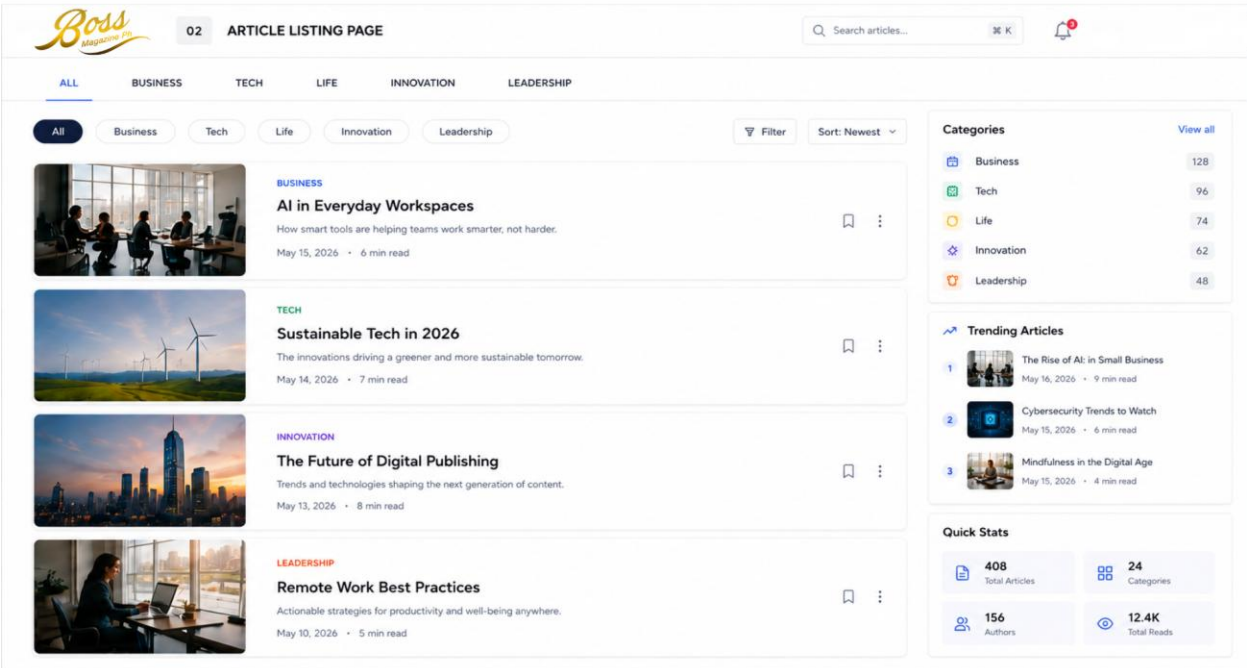


Figure 60 Article listing page

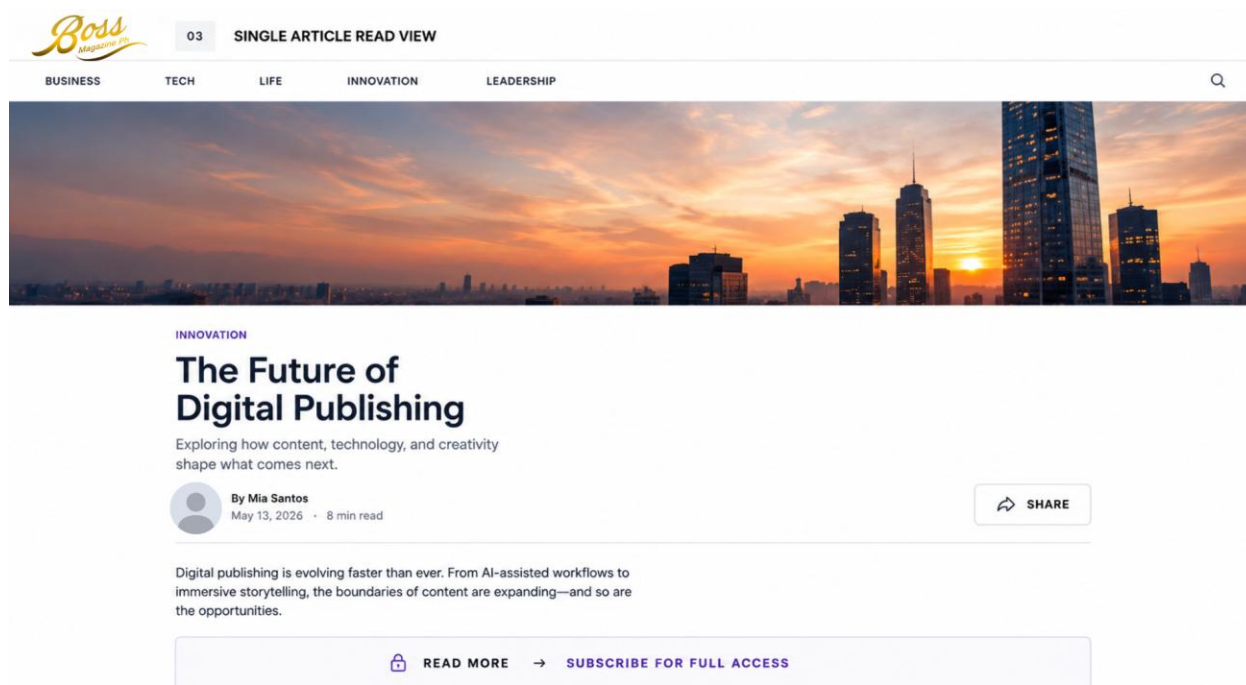


Figure 61 Article read review

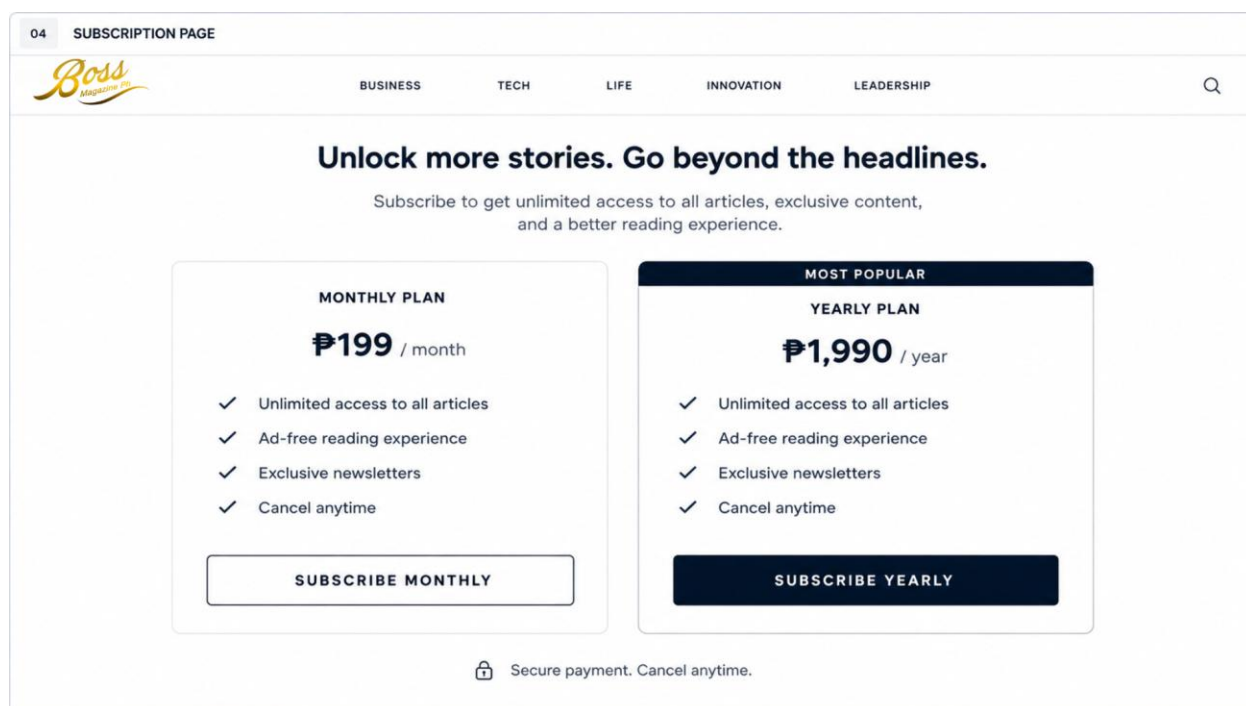


Figure 62 Subscription page

05 PAYMENT CHECKOUT



BUSINESSTECHLIFEINNOVATIONLEADERSHIP

Q

Order Summary

Plan	Yearly Plan
Billing	₱1,990.00 / year
Total	₱1,990.00

Why subscribe?

- ✓ Unlimited access to all articles
- ✓ Ad-free reading experience
- ✓ Exclusive newsletters
- ✓ Cancel anytime

 Secure & encrypted payment
Your payment information is safe with us.

Payment Method

 GCash
 Maya
 Credit Card
 Debit Card

GCash Payment

GCash Number

11-digit mobile number

Account Name

Account name will appear in your GCash app

 Please make sure to send the exact amount
₱1,990.00 to confirm your payment.

Scan to Pay

 GCash
Scan this QR code using
your GCash app



 I HAVE MADE THE PAYMENT

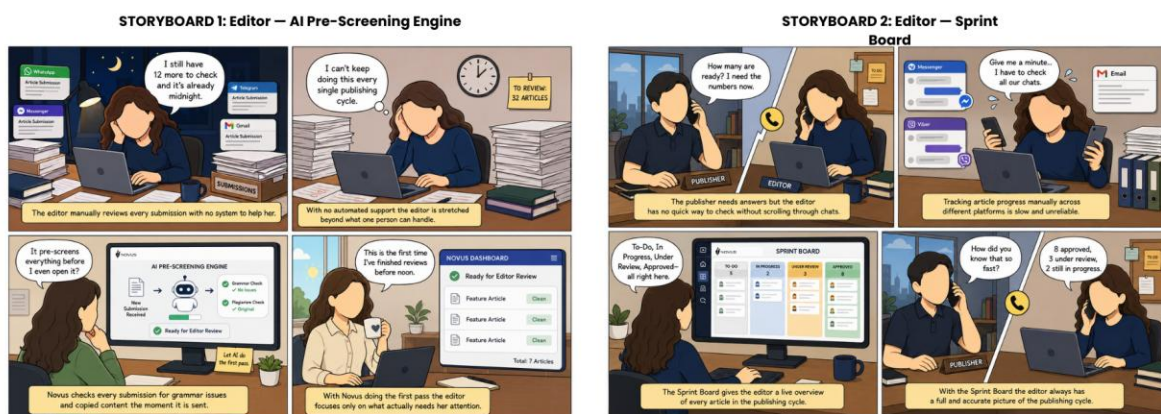
By completing your purchase, you agree to our Terms of Service and Privacy Policy.

Figure 63 Payment checkout

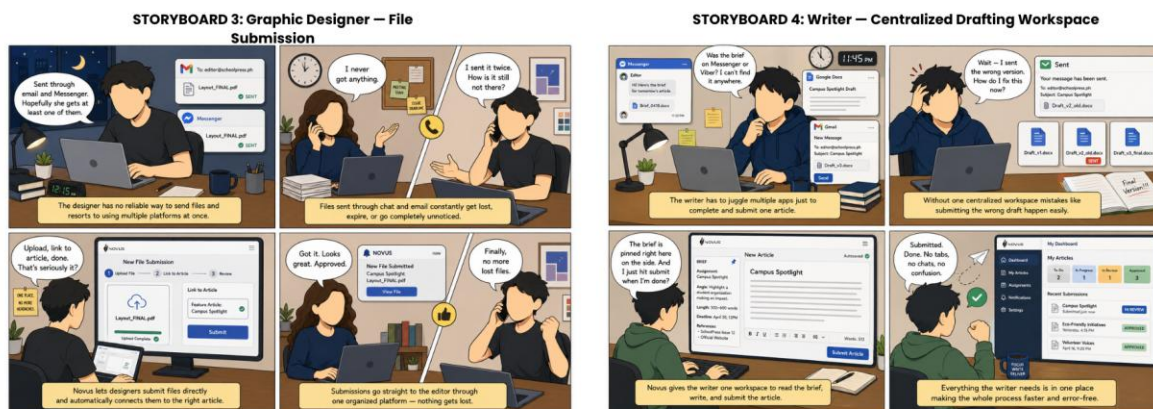
Storyboard

To illustrate the problems addressed by the system and how each feature resolves them, storyboards were developed depicting real-world user scenarios. Fig. 22 presents the storyboards for the editor's AI pre-screening engine and sprint board. Fig. 23 shows the storyboards for the graphic designer's file submission and the writer's centralized drafting workspace. Fig. 24 illustrates the storyboards for the writer's deadline tracking and contextual revision notes features.

NOVUS - STORYBOARDS (PROTOTYPE)



NOVUS - STORYBOARDS (PROTOTYPE)



NOVUS - STORYBOARDS (PROTOTYPE)

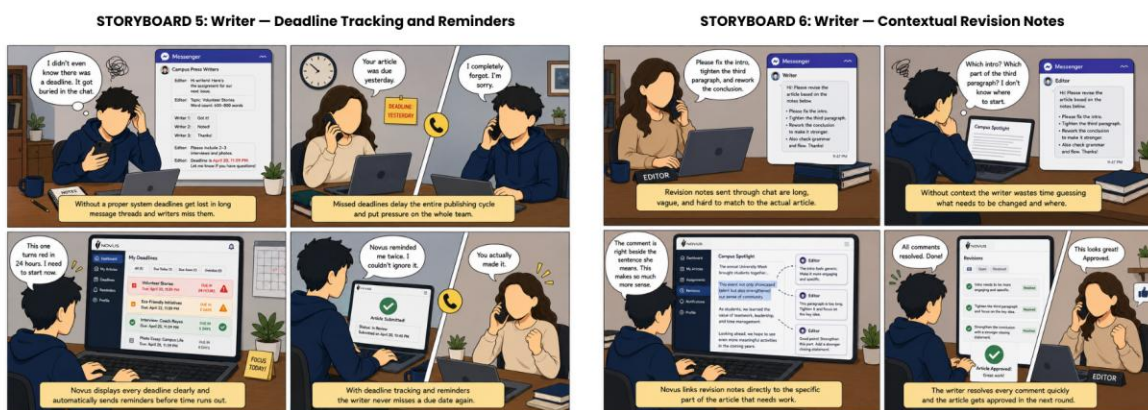


Figure 66 Storyboard Writer deadline tracking & Writer Revision notes

Use Classes and Characteristics

Table VI User classes and characteristics

<i>Roles</i>	<i>Description</i>
<i>Admin</i>	The Administrator user class represents platform managers responsible for system governance, user account management, and role-based access control. Admins maintain oversight of the centralized editorial workflow and access administrative dashboards to monitor subscription revenue, cloud infrastructure costs, and newsroom productivity metrics.
<i>Publisher</i>	The publisher oversees the entire editorial and publishing operations of BOSS Magazine PH. This user has the highest level of access within the internal system, responsible for setting issue deadlines, publishing approved content to the public website, and monitoring subscription revenue and web analytics.
<i>Editor</i>	The editor manages the editorial workflow end-to-end. This user creates and assigns article briefs to writers, reviews submitted drafts, requests revisions or approves final articles, and forwards approved content to the client for sign-off before publication.
<i>Writer</i>	The writer is a content creator assigned to produce articles for the magazine. This user receives assignment briefs through the platform, drafts and submits articles for editorial review, and responds to revision notes until the article is approved.
<i>Graphics Designer</i>	The graphics designer is responsible for the visual production of the magazine. This user receives approved articles and their materials, uploads layout previews and asset versions, and submits the final PDF bundle for editorial and client review.

<i>Reader</i>	A reader is a member of the public who accesses the BOSS Magazine PH public portal. This user can browse articles by category, search by keyword, and opt into the email newsletter. Readers have access to free-tier content only.
<i>Subscriber</i>	A subscriber is a reader who has completed registration and payment to access premium content. This user inherits all capabilities of a standard reader and additionally gains access to paywalled articles, full digital issues, bookmarking, and subscription management through their profile.

Conclusion

BOSS Magazine PH produces consistent, quality content, but its editorial, publishing, and revenue work runs through consumer messaging apps and manual follow-ups. That leaves its sole editor carrying 31 to 40 hours of editorial work a week, stretches each publishing cycle past two weeks, confines its reach to social media, and earns the publication no direct revenue from its content. Through requirement elicitation with BOSS Media Philippines Inc., the team traced these symptoms to a single root cause: the absence of a centralized digital infrastructure. NOVUS was designed to address that cause directly.

In this phase, the team completed the design of NOVUS. Each specific objective now has a defined design that it will be built and measured against:

- **Workflow and capacity.** A centralized pipeline gives every article an explicit status from assignment to publication, records every transition, and groups each dashboard by who can act next. It is designed to handle a minimum of 50 submissions per month.
- **Workload reduction.** An AI pre-screening service scores each draft from 0 to 100. Only drafts that reach the passing mark of 70 go to the editor; the rest return to the writer with revision notes. A Human-in-the-Loop override, with a recorded justification, keeps the final decision with the editor. Success will be measured as at least a 50% reduction in the editor's active review time against a baseline recorded during the pilot phase.
- **Portal performance.** A reader-facing portal gives the magazine an owned, searchable presence independent of social media algorithms. It will be evaluated against standard web performance benchmarks at launch.
- **Monetization readiness.** Subscriptions and a paywall are designed around a local payment gateway that supports GCash, Maya, and cards. Paid digital editions are delivered only through short-lived signed links. The integration will be validated through simulated transactions in a sandbox environment.

The main deliverables of this phase are:

- a product backlog of role-based user stories with acceptance criteria;
- fully dressed use cases with alternate and exception flows, traced to test cases;
- C4 context, container, and component diagrams;
- an entity relationship diagram;
- high-fidelity wireframes for every role.

The review of related literature supports the core design choices: structured workflows over chat-based coordination, AI as a first screen under human control rather than a replacement for editorial judgment, and owned platforms with locally payable subscriptions for independent publishers. The comparison with WordPress, Ghost, and Substack shows that no existing platform combines these capabilities in one system for a small Philippine publisher.

Through NOVUS, BOSS Magazine PH moves from informal, chat-based operations toward a structured, trackable, and revenue-ready publishing platform. That shift supports fairer working conditions for its

team (SDG 8) and gives an independent Filipino publication the digital infrastructure it has so far lacked (SDG 9).

References

- [1] Reuters Institute for the Study of Journalism, "Digital News Report 2026," Univ. of Oxford, Oxford, U.K., Jun. 2026. Accessed: Sep. 26, 2026. [Online]. Available: <https://reutersinstitute.politics.ox.ac.uk/digital-news-report/2026/dnr-executive-summary>
- [2] Cflow, "Editorial workflow: How to streamline content creation for maximum efficiency," Cflow. Accessed: Sep. 26, 2026. [Online]. Available: <https://www.cflowapps.com/editorial-workflow/>
- [3] Amnet, "From manuscript to publication — Why editorial workflows matter more than ever," Amnet, Apr. 28, 2026. Accessed: Sep. 26, 2026. [Online]. Available: <https://amnet.com/from-manuscript-to-publication-why-editorial-workflows-matter-more-than-ever/>
- [4] E. Bastianello, C. Tomlinson, and A. Adamou, "PubLink: Editorial workflow for digital scholarly publications in the humanities," in *Proc. 35th ACM Conf. Hypertext and Social Media (HT '24)*, Poznań, Poland, 2024, pp. 318–322, doi: 10.1145/3648188.3677051.
- [5] E. Brechner, *Agile Project Management with Kanban*. Redmond, WA, USA: Microsoft Press, 2015.
- [6] M. O. Ahmad, J. Markkula, and M. Oivo, "Kanban in software development: A systematic literature review," in *Proc. 39th Euromicro Conf. Software Engineering and Advanced Applications (SEAA)*, Santander, Spain, 2013, pp. 9–16, doi: 10.1109/SEAA.2013.28.
- [7] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track*, New Orleans, LA, USA, 2023.
- [8] M. D. Shermis, "State-of-the-art automated essay scoring: Competition, results, and future directions from a United States demonstration," *Assessing Writing*, vol. 20, pp. 53–76, 2014, doi: 10.1016/j.asw.2013.04.001.
- [9] E. Pitler and A. Nenkova, "Revisiting readability: A unified framework for predicting text quality," in *Proc. 2008 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Honolulu, HI, USA, 2008, pp. 186–195.
- [10] E. Dale and J. S. Chall, "A formula for predicting readability," *Educational Research Bulletin*, vol. 27, no. 1, pp. 11–20, 28, 1948.
- [11] R. Parasuraman and D. H. Manzey, "Complacency and bias in human use of automation: An attentional integration," *Human Factors*, vol. 52, no. 3, pp. 381–410, 2010, doi: 10.1177/0018720810376055.
- [12] Reuters Institute for the Study of Journalism, "Generative AI and news report 2025: How people think about AI's role in journalism and society," Univ. of Oxford, Oxford, U.K., 2025. Accessed: Sep. 26, 2026. [Online]. Available: <https://reutersinstitute.politics.ox.ac.uk/generative-ai-and-news-report-2025-how-people-think-about-ais-role-journalism-and-society>
- [13] N. Newman, R. Fletcher, C. T. Robertson, A. Ross Arguedas, and R. K. Nielsen, "Reuters Institute Digital News Report 2024," Reuters Inst. for the Study of Journalism, Univ. of Oxford, Oxford, U.K., 2024.

Accessed: Sep. 26, 2026. [Online]. Available: <https://reutersinstitute.politics.ox.ac.uk/digital-news-report/2024>

[14] Deployteq, "Challenges facing the publishing and media industry in 2024," Deployteq, Mar. 11, 2024. Accessed: Sep. 26, 2026. [Online]. Available: <https://deployteq.com/challenges-facing-the-publishing-and-media-industry-in-2024/>

[15] Worldpay, "Global Payments Report 2026," Global Payments Inc., Apr. 2026. Accessed: Sep. 26, 2026. [Online]. Available: <https://worldpay.com/en/global-payments-report>

[16] Recurly, "Manually retrying payments? There's a better way," Recurly, Mar. 30, 2021. Accessed: Sep. 26, 2026. [Online]. Available: <https://recurly.com/blog/manually-retrying-payments-theres-a-better-way/>

[17] J. Hamari, N. Hanner, and J. Koivisto, "Service quality explains why people use freemium services but not if they go premium: An empirical study in free-to-play games," *Int. J. Inf. Manage.*, vol. 37, no. 1, pp. 1449–1459, 2017, doi: 10.1016/j.ijinfomgt.2016.09.004.

[18] WordPress.org, "Post status," WordPress Documentation. Accessed: Sep. 26, 2026. [Online]. Available: <https://wordpress.org/documentation/article/post-status/>

[19] PublishPress, "PublishPress Statuses – Custom post status and workflow," WordPress.org Plugin Directory. Accessed: Sep. 26, 2026. [Online]. Available: <https://wordpress.org/plugins/publishpress-statuses/>

[20] Ghost Foundation, "Alternative payment gateways," Ghost Help. Accessed: Sep. 26, 2026. [Online]. Available: <https://ghost.org/help/alternative-payment-gateways/>

[21] Substack, "How do I set up my Stripe account on Substack to start receiving payments?," Substack Support. Accessed: Sep. 26, 2026. [Online]. Available: <https://support.substack.com/hc/en-us/articles/4405482746132-How-do-I-set-up-my-Stripe-account-on-Substack-to-start-receiving-payments>

[22] R. Su and X. Li, "Modular monolith: Is this the trend in software architecture?," in *Proc. 1st Int. Workshop New Trends in Software Architecture (SATrends '24)*, Lisbon, Portugal, 2024, pp. 10–13, doi: 10.1145/3643657.3643911.

[23] Encode OSS, "Django REST framework." Accessed: Sep. 26, 2026. [Online]. Available: <https://www.django-rest-framework.org/>

[24] OWASP Foundation, "OWASP Top 10:2025," 2025. Accessed: Sep. 26, 2026. [Online]. Available: https://owasp.org/Top10/2025/0x00_2025-Introduction/

[25] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Engineering Task Force, RFC 7519, May 2015, doi: 10.17487/RFC7519.

[26] Cloudflare, "Temporary credentials," Cloudflare R2 Documentation. Accessed: Sep. 26, 2026. [Online]. Available: <https://developers.cloudflare.com/r2/api/s3/temporary-credentials/>

[27] World Wide Web Consortium (W3C), "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 5, 2023. Accessed: Sep. 26, 2026. [Online]. Available: <https://www.w3.org/TR/WCAG22/>

[28] *Data Privacy Act of 2012*, Republic Act No. 10173, Philippines, Aug. 15, 2012. Accessed: Sep. 26, 2026. [Online]. Available: <https://privacy.gov.ph/data-privacy-act/>

Appendices

Appendix A: Project Vision

For **BOSS Media Philippines Inc.** and independent publishers struggling with fragmented communication, unsustainable manual review workloads, and zero direct digital revenue, **NOVUS** is a cloud-native, **AI-assisted publishing platform** that unifies the entire media pipeline from initial draft to monetized publication.

Unlike reliance on informal messaging apps, social media algorithms, or general-purpose web builders, **NOVUS** delivers structured editorial tracking, automated AI pre-screening with Human in the Loop overrides, localized PayMongo checkouts, and an owned public portal. This transform disconnected, chat-based operations into an efficient, professional, and commercially sustainable media enterprise.

Appendix B: Schedule

Gantt chart or table containing your project schedule.

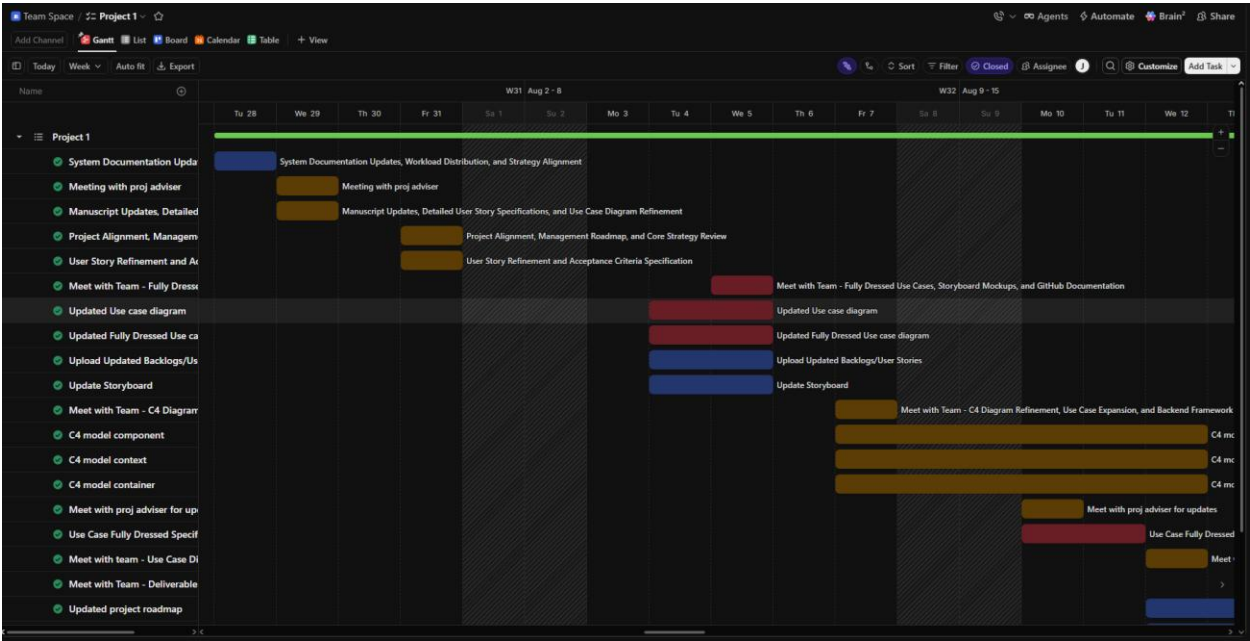


Figure 67 Project schedule

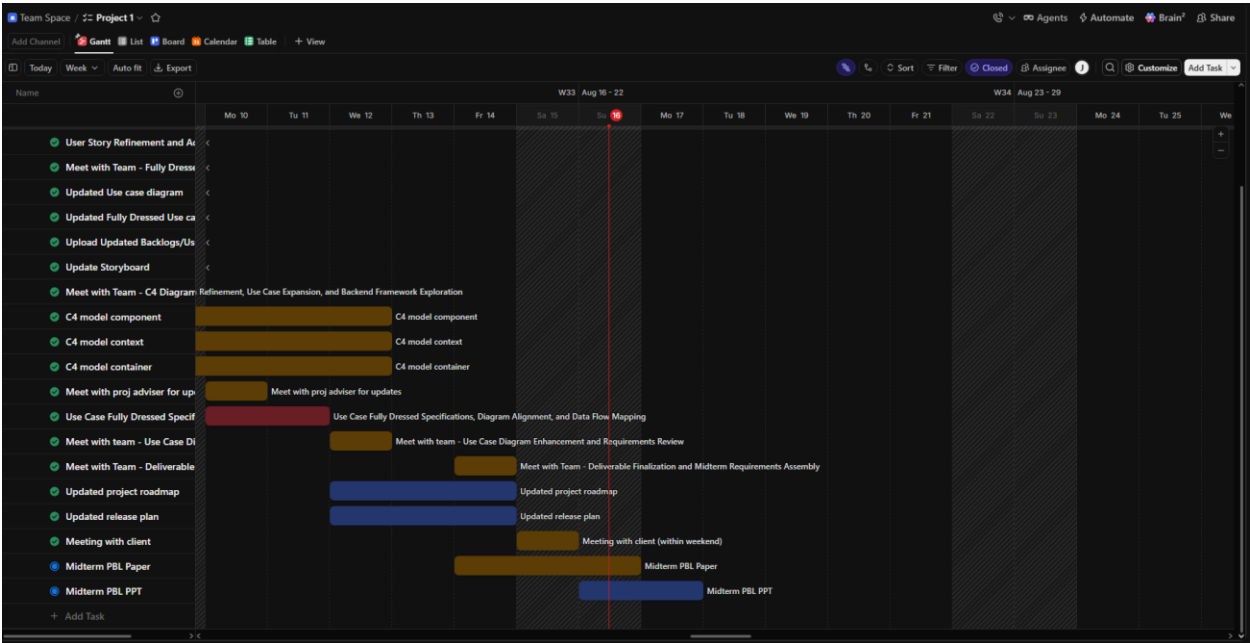


Figure 68 Project schedule II

Appendix C: Product Roadmap

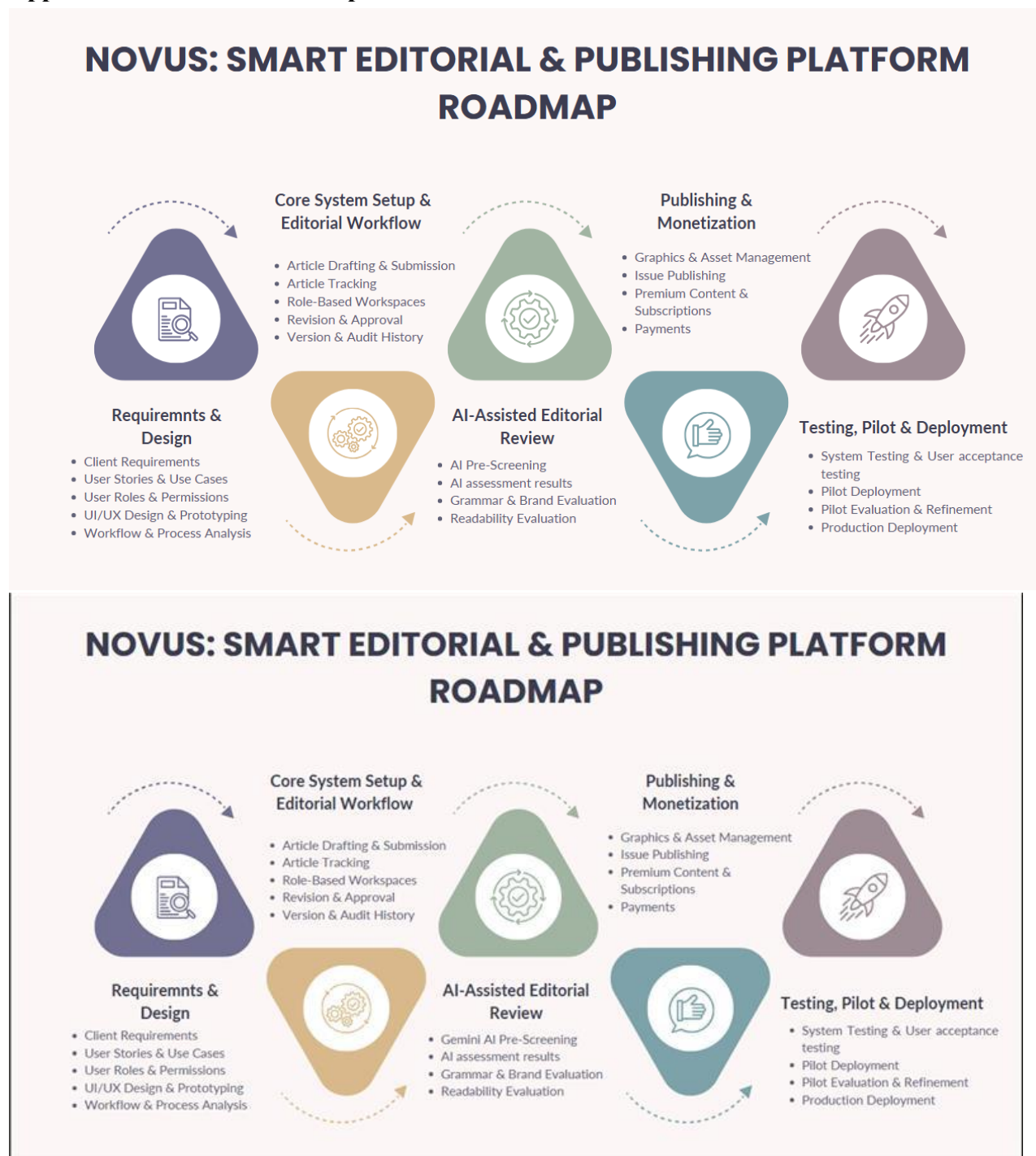


Figure 69 Project Roadmap

Appendix D: Team Meetings

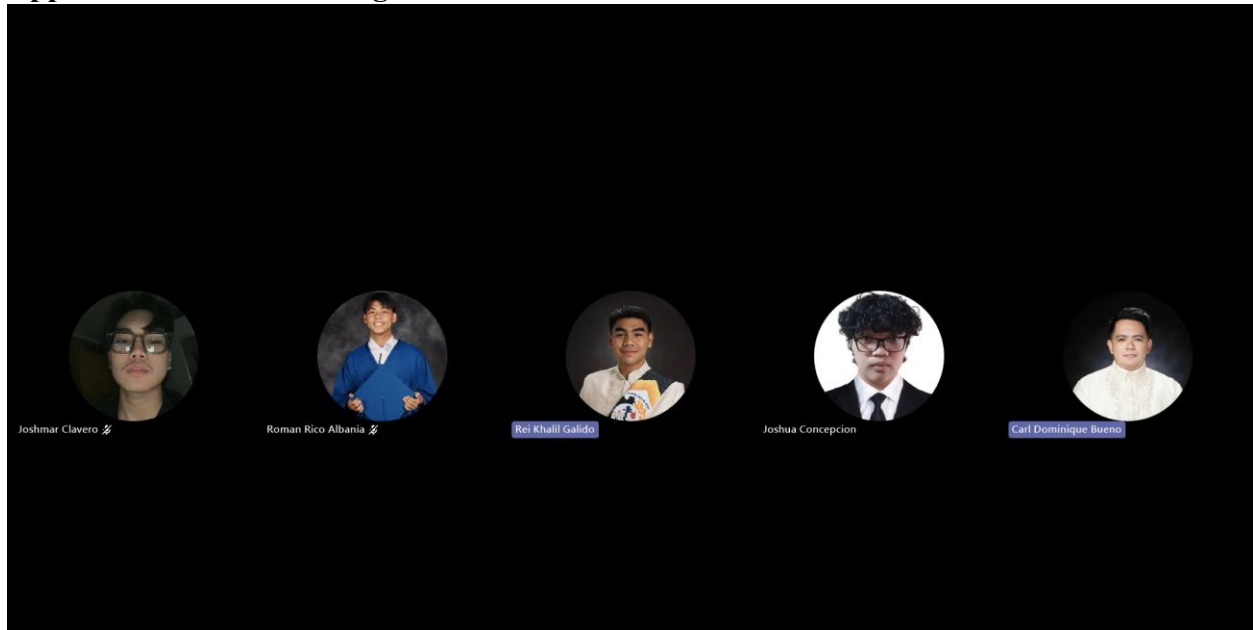


Figure 70 Meeting with team adviser

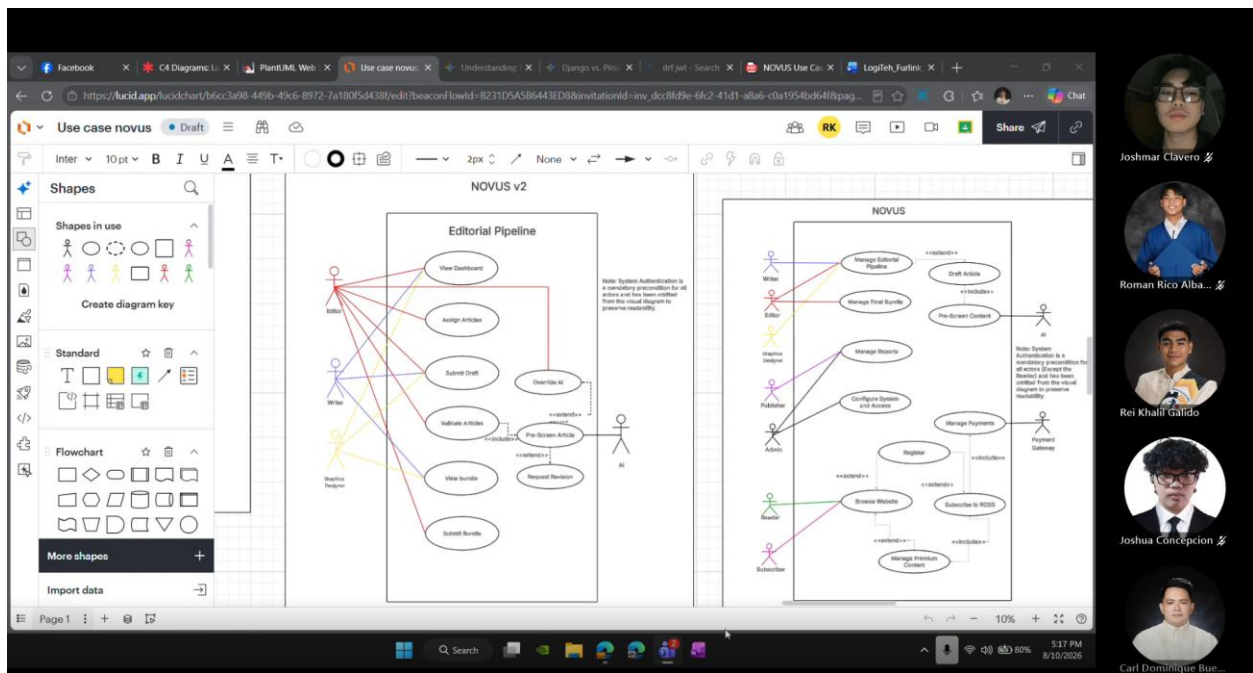


Figure 71 Consultation of Deliverables with team adviser

Recent sprint review.

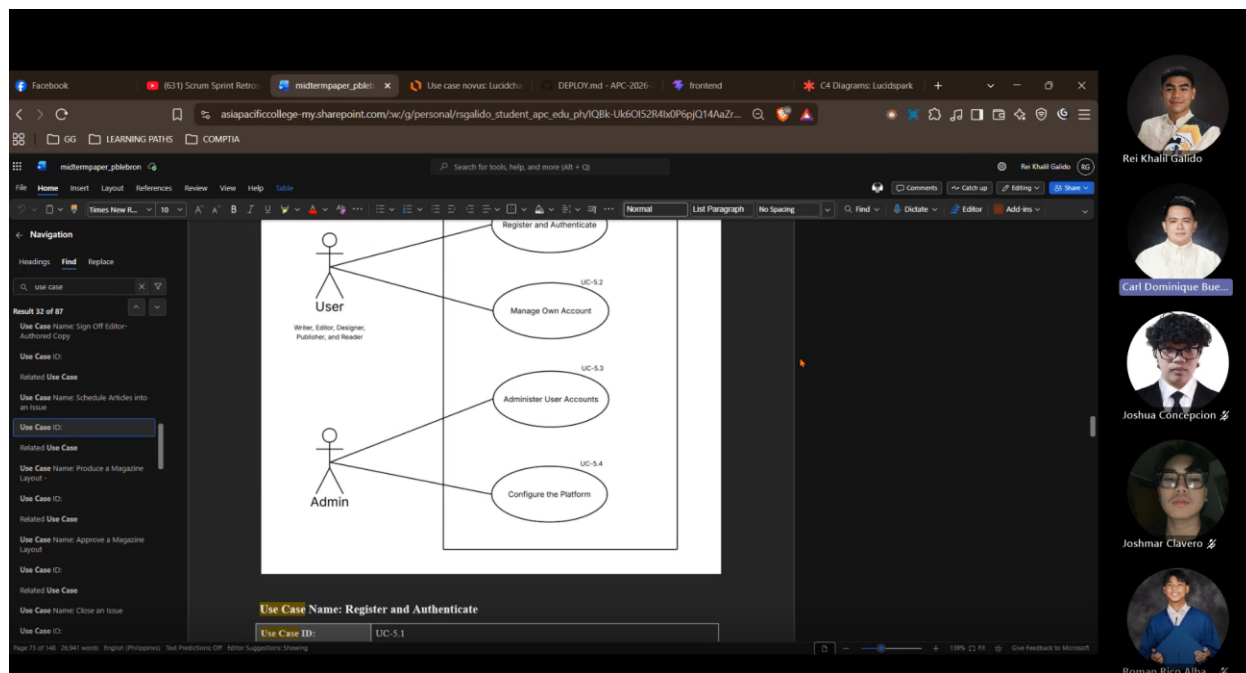


Figure 72 Sprint review with team adviser

Throughout the course of the project, our team conducted a total of 6 sessions of formal meetings, consultations, and progress updates with our Team Adviser, **Sir Carl Dominique Bueno**. While these scheduled sessions provided us with valuable insights and guidance.

Also recently we also conducted a sprint review with him that lasted almost an hour. It consisted of us updating with him with the current progress, review of deliverables, and more tips/guide on what to focus on next.

With this he has helped us a lot in organizing on which task should take priority and which we need to polish more.

Appendix E: Source Code

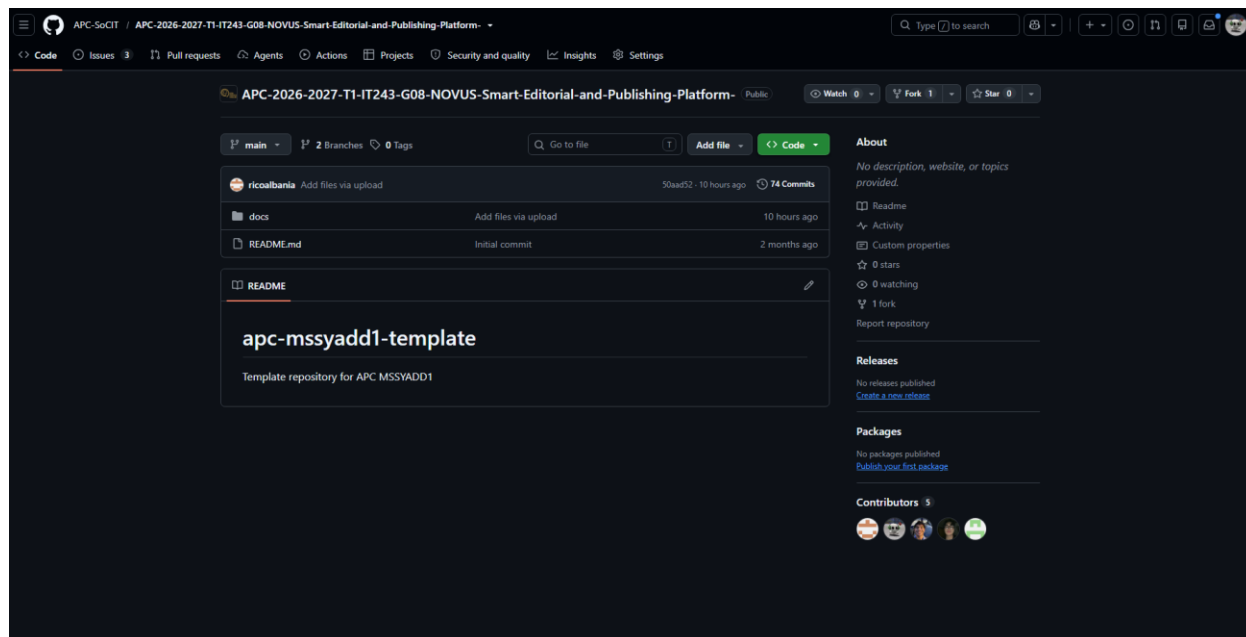


Figure 73 GitHub repository

<https://github.com/APC-SoCIT/APC-2026-2027-T1-IT243-G08-NOVUS-Smart-Editorial-and-Publishing-Platform->